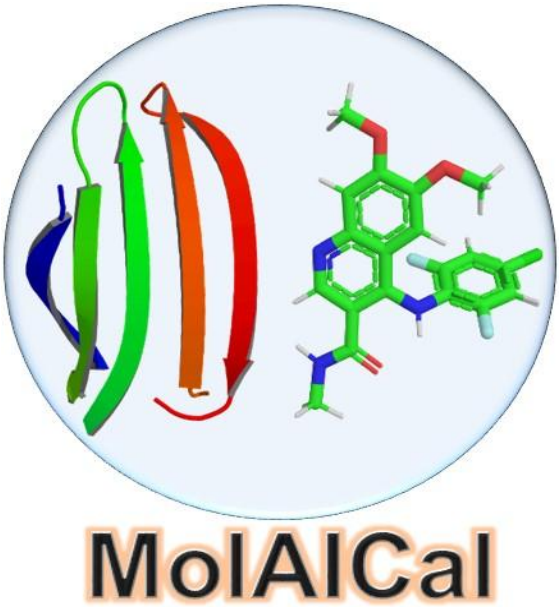
MolAICal

A drug design software combined by artificial intelligence and classical programming

Manual



V2.0a

Official site: https://molaical.github.io
Official site mirror in China: https://molaical.gitlab.io

Qifeng Bai
Email: molaical@yeah.net
Homepage: https://molaical.github.io/baiqf.html
School of Basic Medical Sciences
Lanzhou University
Lanzhou, Gansu 730000, P. R. China

Contents

Part I. Classical Algorithm

1. General information of MolAICal

1.1. Introduction

Computer-aided drug design (CADD) utilizes computational methodologies for the screening, development, modification, analysis, and design of biologically active compounds[1]. With advancements in technology, artificial intelligence (AI) approaches—most notably deep learning—have permeated the realm of CADD research, gradually evolving into the paradigm of AI-driven drug design (AIDD). Both CADD and AIDD are engaged in a range of applications, including de novo drug design, molecular docking, bioactivity prediction, biological image analysis, and synthesis prediction, among others[2]. Importantly, these methodologies contribute to cost reduction and accelerated drug discovery processes. Against this backdrop, the MolAICal software package has been developed for drug design, leveraging AI-centric deep learning methods—such as large language models (LLMs) and agent-based systems—alongside classical algorithms, including genetic algorithms, etc. Key functionalities of MolAICal are outlined as follows:

1. De novo drug design: MolAICal enables fragment growth within receptor pockets, utilizing either pre-assigned molecular libraries or those generated by deep learning models.
2. Molecular docking and virtual screening: MolAICal performs molecular docking and virtual drug screening based on either known ligand databases or molecular databases generated via deep learning models.
3. Prediction of receptor-ligand binding affinity: Based on NAMD trajectories, employing methodologies such as MM/GBSA, among others.
4. Integration of deep learning models and web server: For drug design tasks encompassing molecular generation, drug modification, binding affinity prediction, and related functionalities.
5. Utility tools for drug discovery
   - ✧ Pore radius measurement for receptors (e.g., ion channel proteins).
   - ✧ Potential of mean force calculation for free energy estimation based on principal components.
   - ✧ Quantitative Structure-Activity Relationship (QSAR) analysis for ligand bioactivity prediction.
   - ✧ Calculation of drug-like properties via Lipinski's Rule of Five[3].
   - ✧ Pan-Assay Interference Compounds (PAINS)[4] filtering and ADMET property computation.
   - ✧ MCE-18 descriptor calculation.
   - ✧ Reasoning capabilities of Large Language Models (LLMs) and agent-based systems for drug discovery.

   …among other functionalities.

MolAICal is freely available for educational, academic, and other non-profit purposes. For any commercial use of MolAICal, contact with the licensors is required (Email: molaical@yeah.net).

1.2. Installation

Download
MolAICal: https://molaical.github.io or https://molaical.gitlab.io

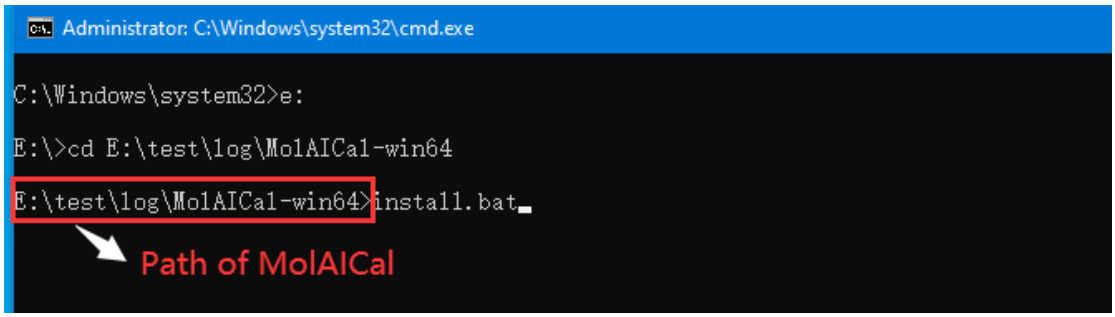
1) For Windows operating system:
Firstly, **decompress** the downloaded MolAICal file. **Secondly**, if MolAICal is placed on E: disk, change into the directory path of MolAICal in DOS or Powershell by below commands (Users should modify them according to the actual path of MolAICal):

```
#> e:
#> cd E:\test\log\MolAICal-win64
```

At last, input the below command and press **Enter** Key for installation:

```
#> install.bat
```

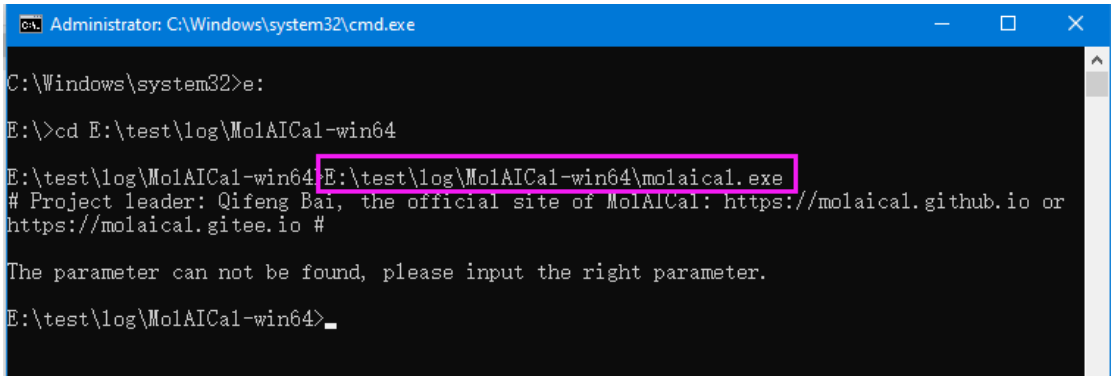
It should be like the below terminal:



MolAICal is a command-line tool. Users can use the absolute path of MolAICal on DOS or Powershell of Windows. To test MolAICal running, for example: if MolAICal is placed on E: disk, then users can run the blow example command (Users should modify it according to the actual path of molaical.exe):

```
#> E:\test\log\MolAICal-win64\molaical.exe
```

Press **Enter** Key, it should show similar message as follows:



Until now, install and test are complete.

2) For Linux operating system:

Using the below commands in Linux terminal console to install MolAICal:

1) tar -xzvf MolAICal-linux64-xxx.tar.gz

Notice: Please replace the corrected characters in MolAICal-linux64-xxx.tar.gz according to your downloaded MolAICal.

2) cd MolAICal-linux64-xxx
3) chmod +x install.sh
4) ./install.sh

Installing MolAICal on a Linux system requires starting (or removing and starting) a container and image, which is relatively time-consuming—please be patient. If the user wishes to install MolAICal in the background or has a limited terminal session time, they can use the command:
./install.sh >& log &
to perform a background installation.

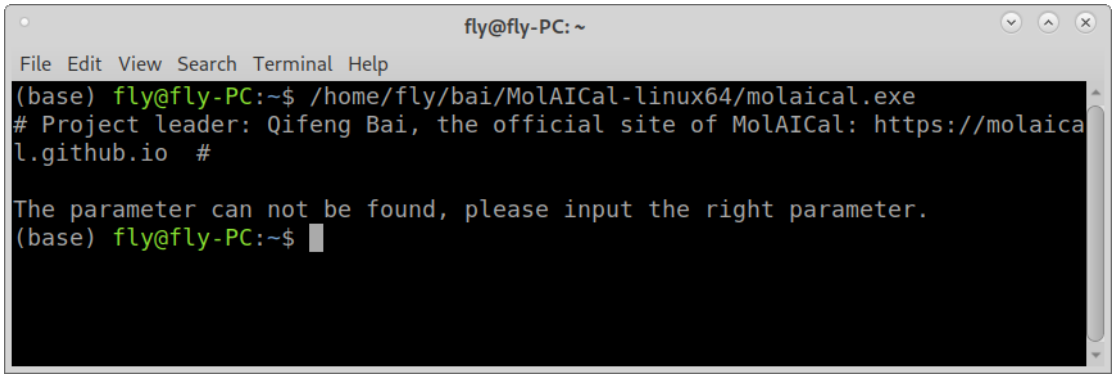
Notice: MolAICal only provides Windows and Linux 64-bit versions. Users should not change the name or path of MolAICal after installation is complete in the Linux System. If users change the name or path of MolAICal, please reinstall MolAICal. To use MolAICal expediently, users can set the system path of MolAICal. So the command molaical.exe can be used without a given path.

Since the Linux version of MolAICal launches a persistent container that runs continuously, users can **remove, uninstall or stop** this container of **MolAICal** using the following commands:
#> molaical.exe -eset sys rmi molaical:v9
#> molaical.exe -eset sys rm molaical

For more detailed tutorials, please go to the MolAICal official website: https://molaical.github.io or the official site mirror in China: https://molaical.gitlab.io.

At last, users can run the blow example command in the Linux terminal console (Users should modify it according to the actual path of molaical.exe):
#> /home/fly/bai/MolAICal-linux64/molaical.exe

Press **Enter** Key, it should show similar messages as follows:



Until now, installation and testing are complete.

Possible install issues (option):

If MolAICal images and containers have already been loaded previously, MolAICal will first remove these images and containers. If similar errors occur as below:

rm: cannot remove '630b0b26-296e-3f17-a1b7-e4effdf77bba/ROOT/usr/bin': Directory not empty

the following command can be executed to forcefully remove the container:

1) Enter P1 mode:

Usage: molaical.exe -eset sys setup --execmode=P1 containerIDNAME

e.g.,

#> molaical.exe -eset sys setup --execmode=P1 molaical

or

#> molaical.exe -eset sys setup --execmode=P1 30754628-67fe-3109-bfd3-00a830163ee5

And then:

#> cd ~/.udocker/containers/

#> rm -rf <container folder>

1.3. How to cite MolAICal

If MolAICal is used in your work, please cite the two papers below:

1. Bai, Q. et al. MolAICal: a soft tool for 3D drug design of protein targets by artificial intelligence and classical algorithm. *Briefings in bioinformatics* 22, bbaa161 (2021). https://doi.org/10.1093/bib/bbaa161

2. Bai, Q., Research and development of MolAICal for drug design via deep learning and classical programming. *arXiv* 2020. doi: https://arxiv.org/abs/2006.09747

1.4. Environment parameters of Linux version MolAICal

The environment parameters of MolAICal start with the parameter '-eset', the usages are below:

- If **'-eset xserver'**, it will check whether X server is reachable
- If **'-eset lib'**, put the path into the file 'file_lib'.
- If **'-eset bin'**, put the path into the file 'file_bin'.
- If **'-eset mount'**, put the path into the file 'file_mount'.
- If **'-eset clean'**, clean the contents of the files 'file_lib', 'file_bin', and 'file_mount'.
- If **'-eset pdclean'**, this command searches for processes associated with files and directories in the current directory (including all subdirectories by default). If associated processes are found, they are terminated. It accepts one optional argument specifying a directory path. When this argument is provided: it terminates processes associated with files/directories in the current

directory only (excluding subdirectories), and it terminates processes associated with files/directories in the specified directory path and all its subdirectories.

- If it contains **"-call set –p <arg> -n <arg>"**, it will set the <arg> path of -p into the files 'file_lib', 'file_bin', and 'file_mount'.
- If **'-eset start env'**, start the container and image.
- If **'-eset stop env'**, stop the container and image.
- If **'-eset restart env'**, restart the container and image.
- If the X Window System (X11) for local inter-process communication between the X server and graphical applications is not found, please set **"x_gui_set"** for molaical.exe. The default value is "/tmp/.X11-unix". For example: input "export vmd=/home/feng/soft/vmd193" in the console and press the 'Enter' key on the keyboard.
- If set "export debug=1", MolAICal will print some information for debug.
- If **'-eset shell <arg>'**, Supported values: **in** | **exit**. **'in'** means to enter into the virtual environment of MolAICal, it will see the file paths and do operations by Linux commands. **'exit'** means to exit the current virtual environment. The **'in'** argument enters the MolAICal virtual environment, where users can view file paths and perform operations using Linux commands. The **'exit'** argument exits the current virtual environment.
- If **'-eset sys'**, it serves for the operation of images, containers, etc by calling udocker (https://github.com/indigo-dc/udocker or https://indigo-dc.github.io/udocker) under Apache-2.0 license (https://www.apache.org/licenses/LICENSE-2.0). The more information is below:

'-eset sys' Syntax:

molaical.exe -eset sys [general_options] <command> [command_options] <command_args>

molaical.exe -eset sys [-h|--help|help] :Display this help and exits

molaical.exe -eset sys [-V|--version|version] :Display tarball version and exits

General options common to all commands must appear before the command:

-D, --debug	:Debug
-q, --quiet	:Less verbosity
--insecure	:Allow insecure non authenticated https
--repo=<directory>	:Use repository at directory
--allow-root	:Allow execution by root NOT recommended
--config=<conf_file>	:Use configuration <conf_file>

Commands:

--help [command]	:Command specific help
showconf	:Print all configuration options
search <repo/expression>	:Search dockerhub for container images
pull <repo/image:tag>	:Pull container image from dockerhub
create <repo/image:tag>	:Create container from a pulled image
run <container_id|name>	:Execute created container
run <repo/image:tag>	:Pull, create and execute container
images -l	:List container images
ps -m -s	:List created containers
name <container_id> <name>	:Give name to container

```
rmname <name>                          :Delete name from container
rename <name> <new_name>               :Change container name
clone <container_id>                   :Duplicate container
rm    <container-id|name>              :Delete container
rmi <repo/image:tag>                   :Delete image
tag <repo/image:tag> <repo2/image2:tag2>     :Tag image
import <tar> <repo/image:tag>          :Import tar file (exported by docker)
import - <repo/image:tag>              :Import from stdin (exported by docker)
export -o <tar> <container>            :Export container directory tree to file
export - <container>                   :Export container directory tree to stdin
load -i <exported-image>               :Load image from file (saved by docker)
load                                   :Load image from stdin (saved by docker)
save -o <imagefile> <repo/image:tag>   :Save image with layers to file

inspect -p <repo/image:tag>            :Print image or container metadata
verify <repo/image:tag>                :Verify a pulled image
manifest inspect <repo/image:tag>      :Print manifest metadata

molaical.exe -eset sys manifest inspect centos/centos8
molaical.exe -eset sys pull --platform=linux/arm64 centos/centos8
molaical.exe -eset sys tag centos/centos8    mycentos/centos8:arm64

protect <repo/image:tag>               :Protect repository
unprotect <repo/image:tag>             :Unprotect repository
protect <container>                    :Protect container
unprotect <container>                  :Unprotect container
mkrepo <top-repo-dir>                  :Create another repository in location
setup --execmode=<mode>                :Change container execution mode
setup --nvidia                         :Setup container to use nvidia GPU
setup --purge                          :clean mountpoints and files created by molaical.exe -eset sys
setup --fixperm                        :attempt to fix file permissions
login                                  :Login into docker repository
logout                                 :Logout from docker repository
```

Examples:

```
molaical.exe -eset sys search expression
molaical.exe -eset sys search quay.io/expression
molaical.exe -eset sys search --list-tags myimage
molaical.exe -eset sys pull myimage:mytag
molaical.exe -eset sys images
molaical.exe -eset sys create --name=mycontainer    myimage:mytag
molaical.exe -eset sys ps -m -s
molaical.exe -eset sys inspect mycontainer
molaical.exe -eset sys inspect -p mycontainer
```

```
molaical.exe -eset sys manifest inspect centos/centos8
molaical.exe -eset sys pull --platform=linux/arm64 centos/centos8
molaical.exe -eset sys tag centos/centos8    mycentos/centos8:arm64

molaical.exe -eset sys run    mycontainer    cat /etc/redhat-release
molaical.exe -eset sys run --hostauth --hostenv --bindhome    mycontainer
molaical.exe -eset sys run --user=root    mycontainer    yum install firefox
molaical.exe -eset sys run --hostauth --hostenv --bindhome mycontainer    firefox
molaical.exe -eset sys run --entrypoint="" mycontainer    /bin/bash -i
molaical.exe -eset sys run --entrypoint="/bin/bash" mycontainer -i

molaical.exe -eset sys clone --name=anotherc mycontainer
molaical.exe -eset sys rm anotherc

molaical.exe -eset sys mkrepo /data/myrepo
molaical.exe -eset sys --repo=/data/myrepo load -i docker-saved-repo.tar
molaical.exe -eset sys --repo=/data/myrepo images
molaical.exe -eset sys --repo=/data/myrepo run --user=$USER    myimage:mytag

molaical.exe -eset sys export -o myimage.tar mycontainer
molaical.exe -eset sys import myimage.tar mynewimage
molaical.exe -eset sys create --name=mynewc mynewimage
molaical.exe -eset sys export --clone -o mycontainer.tar mycontainer
molaical.exe -eset sys import --clone mycontainer.tar
```

Notes:

* by default the binaries, images and containers are placed in
  $HOME/.udocker
* by default the following host directories are mounted in the
  container:
  /dev /proc /sys /etc/resolv.conf /etc/host.conf /etc/hostname
* to prevent the mount of the above directories use:
  run --nosysdirs <container>
* additional host directories to be mounted are specified with:
  run --volume=/data:/mnt --volume=/etc/hosts <container>
  run --nosysdirs --volume=/dev --volume=/proc <container>
* molaical.exe -eset sys provides several execution modes that offer different
  approaches and technologies to execute containers, they
  can be selected using the setup command. See the setup help.
  molaical.exe -eset sys setup --execmode=F3 fedx
  molaical.exe -eset sys setup --execmode=R1 fedx
  molaical.exe -eset sys setup --execmode=S1 fedx
  molaical.exe -eset sys setup --help

```
* molaical.exe -eset sys facilitates the usage of nvidia drivers within containers
      molaical.exe -eset sys setup --nvidia fedx
```

Here, the example commands are listed:

1) Start the X server and run the program if the x server is not set.

#> molaical.exe -eset xserver -pmf -i rmsd-dis.dat

2) Start the X server

#> molaical.exe -eset xserver

3) Set the path for the vmd and external program

#> molaical.exe -call set -p "/home/feng/soft/vmd193/bin/vmd" -n vmd

#> molaical.exe -call set -p "/home/feng/soft/namdcpu/namd3" -n namd

4) Clean the set path

#> molaical.exe -eset clean

5) Delete all processes associated with all files and folders under the current directory

#> molaical.exe -eset pdclean

6) Delete all processes associated with all files and folders under the folder "dresults" including the current directory

#> molaical.exe -eset pdclean dresults

7) Set lib path

#> molaical.exe -eset lib /home/feng/soft/namdcpu

8) Set bin path

#> molaical.exe -eset bin /home/feng/soft/namdcpu

9) Set mount path

#> molaical.exe -eset mount /home/feng/soft/vmd193

10) Start the container and image in MolAICal

#> molaical.exe -eset start env

11) Stop the container and image in MolAICal

#> molaical.exe -eset stop env

12) Restart the container and image in MolAICal

#> molaical.exe -eset restart env

13) Enter and exit the virtual environment in MolAICal

#> molaical.exe -eset shell in
When wanting to get out of this environment, just type the **'exit'** command.

14) Check images and containers
#> molaical.exe -eset sys images
#> molaical.exe -eset sys ps

15) Load images and create containers
#> molaical.exe -eset sys load -i molaicalvx.tar
#> molaical.exe -eset sys create --name=molaical molaical:v9

16) Remove images and containers
Usage: molaical.exe -eset sys rmi <image name>
#> molaical.exe -eset sys rmi molaical:v9

Usage: molaical.exe -eset sys rm <container name>
#> molaical.exe -eset sys rm molaical

Notice: udocker does not have a direct equivalent to the **"docker commit"** command, which creates a new image from a container's changes in Docker.
However, it provides similar functionality through a manual workaround using its export and import commands:

- Use udocker export -o <tar_file> <container> to export the modified container's filesystem to a tar file.
- Then, use udocker import <tar_file> <new/repo/image:tag> to import that tar file as a new image.

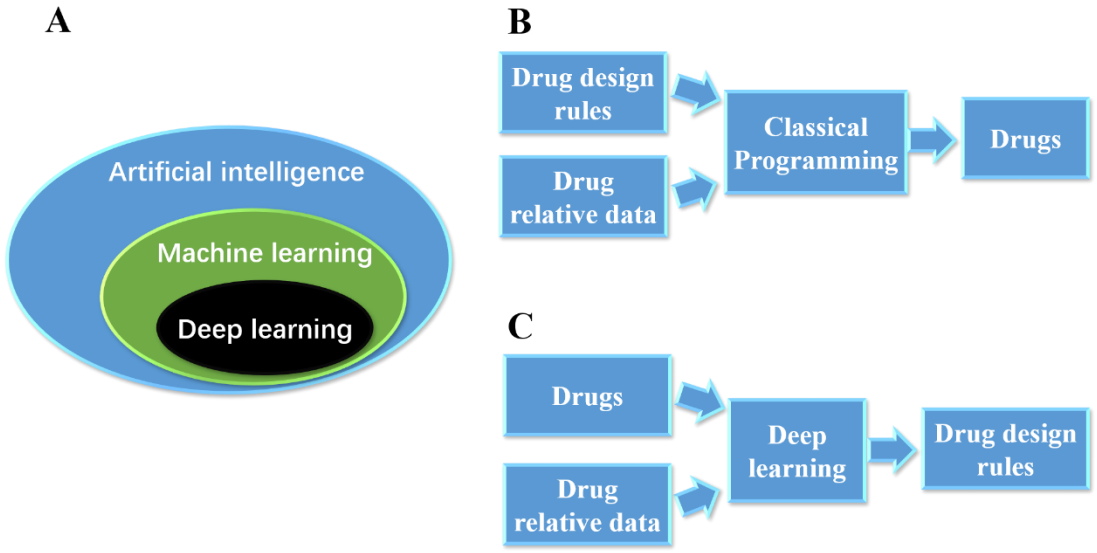
This process allows you to save and reuse the state of a modified container, though it's not as automated as **"docker commit"**. Note that containers in udocker retain changes until explicitly removed, but without exporting, those changes aren't persisted as a new image.

2. *De novo* drug design by MolAICal and DL

2.1. Theory

Rational drug design aims to identify novel therapeutic agents based on the understanding of biological targets, including proteins, DNA, RNA, and the like. Computer-aided drug design (CADD) constitutes an approach within rational drug design, encompassing the discovery, design, optimization, and analysis of biologically active pharmaceutical agents. Fragment-based drug discovery (FBDD) is a de novo drug design strategy aimed at identifying lead compounds. These fragments are small organic molecules characterized by small size and low molecular weight.

Such small chemical fragments may exhibit weak binding affinity to biological targets; however, they can be considered as anchored fragments. Subsequently, novel compounds can be generated through fragment growth based on these anchored fragments within the receptor pocket.



Part I Figure 2.1 The deep learning and algorithm for drug design.

Deep learning, which utilizes multiple layers to progressively extract higher-level features from raw input data, is a subset of machine learning and artificial intelligence (see Part I Figure 2.1A). Deep learning has been successfully applied in drug discovery[2] and medical research[5]. It enables the training of molecular generative models capable of generating molecules with 1D sequences or 2D structures via generative adversarial networks (GANs)[6-7]. Traditionally, classical programming generates new pharmaceutical agents by inputting drug design rules and drug-related data (e.g., molecular weight, logP) (see Part I Figure 2.1B). Input drug data and drug design algorithms play a crucial role in novel drug discovery. In the case of deep learning methods, target drugs and drug-related data are provided, with deep learning tasked with identifying drug design rules through the training of deep neural networks (see Part I Figure 2.1C). Although deep learning can train drug design rules for generating new drugs with similar properties, it still requires effective methods to generate 3D-structured ligands within 3D receptor pockets. Classical programming and deep learning each possess distinct merits in drug discovery. Here, MolAICal integrates the merits of de novo drug design and deep learning models for drug design. The molecular generative model, which is trained on fragments of FDA-approved drugs using GANs, can generate fragments for ligand growth within the receptor pocket. Additionally, MolAICal can perform classical de novo drug design based on given small fragments. At last, MolAICal also carries out the energy minimization to optimize the output results.

2.2. Parameters in the configuration file

These parameters on this part are contained in the configuration file, which is the input file for *de novo* drug design. The examples of input configuration files are shown in Appendices 1 and 2.

- **centerPoints** < center position of box >

Acceptable Values: decimal
Default Value: null
Description: The center coordinates of the appointed box.

- **boxLengthXYZ** < box length >

Acceptable Values: decimal
Default Value: null
Description: It is the box lengths that are sequentially along the X, Y, and Z axes.

- **receptorPDB** < path of receptor >

Acceptable Values: String
Default Value: null
Description: The path of the PDB format receptor in the operating system.

- **growMethod** < grow method >

Acceptable Values: fixFrag, randomFrag
Default Value: null
Description: The 'fixFrag' means the coordinates of the initial molecular fragment are fixed, and this molecular fragment may be extracted from the ligand in the crystal receptor-ligand complex. The "randomFrag" means the positions of the initial fragment are not fixed, and the initial fragment should be assigned in SMILES format. The initial position should also be appointed. If this part is on, it must set the "startAtomPosition" and "startSmiFrag".

- **startFragFile** < path of initial fragment >

Acceptable Values: String
Default Value: null
Description: The path of the initial fragment in the operating system. The initial fragment is in Sybyl mol2 format.

- **startSmiFrag** < Initial SMILES fragment >

Acceptable Values: String
Default Value: None
Description: If the growMethod is "randomFrag", this parameter should be set. If the value of **startSmiFrag** is the Canonical SMILES string, such as C, the Canonical SMILES string will be converted to the 3D seed file whose name is appointed by the parameter **startFragFile**. If the value of **startSmiFrag** is equal to "null" or "off", it means the user-defined fragment in mol2 format is employed as the initial seed. In this case, the users can set their own initial fragment file in the part of parameter **startFragFile.**

- **gSeedKValue** < control constant >

Acceptable Values: decimal
Default Value: 0.25
Description: If the growMethod is "randomFrag", this parameter can be selected to set. This is a

control constant. A small value of gSeedKValue can reduce the searching range around the assigned point, and vice versa.

- **gSeedSearchNum** < Searching number >

Acceptable Values: Integer
Default Value: 2000
Description: If the growMethod is "randomFrag", this parameter can be selected to set. This parameter can set the searching number around the assigned point.

- **startAtomPosition** < path of chosen atom file in receptor >

Acceptable Values: String
Default Value: null
Description: If the growMethod is "randomFrag", this parameter should be set. To set the position of the initial SMILES fragment, it should appoint the atom of the receptor as the coordinate reference. The appointed atom can be extracted from the receptor structure by graphical tools such as Pymol, VMD, UCSF Chimera, etc. The atom file must be in standard PDB format.

- **numCycleGen** < number of growth cycles>

Acceptable Values: Integer
Default Value: null
Description: It is the number of growth cycles. Each growth cycle is based on the previous cycle.

- **selTopMols** < number of parent>

Acceptable Values: Integer
Default Value: null
Description: This part is designed by the genetic algorithm. The "selTopMols" represents the number of parents.

- **selTopRootPercent** < percentage of parent selection >

Acceptable Values: decimal
Default Value: null
Description: It is the percentage of parent selection. Multiply "selTopMols" by "selTopRootPercent" to get the selected number.

- **selElitePercent** < percentage of elitist selection >

Acceptable Values: decimal
Default Value: null
Description: It is the percentage of elitist selection.

- **maximumPOP** < maximum of populations >

Acceptable Values: Integer
Default Value: null
Description: It is the maximum value of the population.

- **libStyle** < Selection way of fragments library >

Acceptable Values: mol2, SMILES, AIFrag
Default Value: null
Description: MolAICal supports three ways for ligand growth. The "mol2" means that MolAICal uses its own mol2 format fragments library. The "SMILES" means MolAICal uses its own SMILES sequence library. The "AIFrag" means that MolAICal calls the deep learning model named "AIGenMols" to generate new fragments.

- **genAIWay** < selected model for fragment generation >

Acceptable Values: Fdafrag
Default Value: null
Description: If you set "libStyle" to AIFrag. It should set the value of "genAIWay", which can be set to "Fdafrag".

- **genAINumber** < number of generated fragments >

Acceptable Values: Integer
Default Value: null
Description: If you set "libStyle" to AIFrag. It should set the value of "genAINumber". It can be an arbitrary integer value. For instance, if you set it to 81, it means the deep learning model named "AIGenMols" will generate 81 different fragments.

- **perturbeSearch** < whether to perform perturbation searching >

Acceptable Values: on or off
Default Value: null
Description: If "on", the procedure will grow the fragment by perturbation searching around bonds. If "off", the procedure will grow the fragment along the hydrogen atom direction.

- **genAlgorithm** < searching algorithm for fragment generation anchored on one point >

Acceptable Values: random, fibonacci
Default Value: null
Description: The "random" means the fragments choose random points for perturbation growth. The "fibonacci" means the fragments choose the spherical Fibonacci points for perturbation growth.

- **ranGenOptNum** < the number of fragment generations surrounding on anchoring point >

Acceptable Values: Integer
Default Value: null
Description: It represents the number of fragment generations surrounding on anchoring point. The big number will expend more computational resources, but the ligand growth will be more precise.

- **atomMutation** < mutation on one atom >

Acceptable Values: String
Default Value: null
Description: The value is "on" or "off". The "on" means atoms "C", "N", and "O" will be mutated during the process of ligand growth, and vice versa.

- **mutationPercent** < percentage of mutation ratio >

Acceptable Values: decimal
Default Value: null
Description: If the "atomMutation" is set, the "mutationPercent" should be assigned a value.

- **randomCrossMutation** < crossover and mutation >

Acceptable Values: String
Default Value: null
Description: This parameter is involved in crossover and mutation. It can be the value of "on" or "off". "On" means the crossover and mutation are carried out during the process of ligand growth, and vice versa. MolAICal regards the ligand as the chromosome. The fragments connected by rotated bonds are compared to "A", "T", "G", "C", etc.

- **randomCrossRatio** < ratio for cross operation >

Acceptable Values: decimal
Default Value: null
Description: If the "randomCrossMutation" is set, the "randomCrossRatio" should be appointed. It is the ratio of crossover operation in the generated population. This mutation unit is one fragment.

- **randomMutationRatio**< ratio of mutation >

Acceptable Values: decimal
Default Value: null
Description: If the "randomCrossMutation" is set, the "randomMutationRatio" should be appointed. It is the ratio of mutation. This mutation unit is one fragment.

- **randomCrossCompareNum** < number of selected molecules for crossover and mutation >

Acceptable Values: Integer
Default Value: null
Description: If the "randomCrossMutation" is set, the "randomCrossCompareNum" should be appointed. It represents the number of selected molecules from the left molecular population. For example, if the value is 2, it means the current ligand will crossover and mutate with 2 ligands of the left random remainder populations.

- **randomCrossRange** < range of crossover and mutation >

Acceptable Values: Discrete integer
Default Value: null
Description: If the "randomCrossMutation" is set, the "randomCrossRange" should be appointed. It corresponds to "numCycleGen". The discrete integer corresponds to the cycle of "numCycleGen", which represents that this cycle performs crossover and mutation.

- **writeResultParallel** < parallel writing mode >

Acceptable Values: String
Default Value: null

Description: If “on”, it means the temporal file is saved in the parallel mode, and vice versa.

- **unwantedFrag** < filtering unwanted fragments >

Acceptable Values: String
Default Value: null
Description: If “on”, it will filter the generated molecules that contain unwanted fragments, and vice versa.

- **PAINS** < filtering Pan-assay interference compounds >

Acceptable Values: String
Default Value: null
Description: If “on”, it will filter the generated molecules that contain Pan-assay interference compounds (PAINS) fragments, and vice versa.

- **growFilter** < filtering wrong position molecules >

Acceptable Values: String
Default Value: null
Description: If “on”, it will filter the generated molecules which are overlap and out-of-box fragments, and vice versa.

- **growFilterRatio** < cutoff of overlap molecule >

Acceptable Values: decimal
Default Value: null
Description: This ratio is used to filter overlapping molecules during the storage process of the temporal file.

- **resultsFilterRatio** < cutoff of overlap molecule >

Acceptable Values: decimal
Default Value: null
Description: This ratio is used to filter overlapping molecules during the storage process of final results ligands.

- **syntheticAccessibility** < synthetic accessibility >

Acceptable Values: String
Default Value: null
Description: If “on”, it will filter the unreachable criterion ligands according to the set value of synthetic accessibility, and vice versa.

- **synAccessibilityValue** < cutoff of synthetic accessibility >

Acceptable Values: decimal
Default Value: null
Description: If the “syntheticAccessibility” is set, the “synAccessibilityValue” should be appointed. Synthetic accessibility (SA) is a cutoff ranging from 0 to 100, in which value 100 is maximal synthetic accessibility; it means this molecule is most easily synthesizable.

- **MolsimilarPercent** < filtering the similar ligands >

Acceptable Values: decimal
Default Value: null
Description: This parameter is used to filter similar ligands and add new novel ligands into the parents' populations according to the cutoff.

- **RulesOfFive** < Lipinski's rule of five >

Acceptable Values: String
Default Value: null
Description: If "on", it will filter the unreachable criterion ligands according to Lipinski's rule of five, and vice versa. The Lipinski's rule of five is changed with the passage of time[8-9], so it can be set to a suitable value according to the research reports or the specific customization.

- **xlogPvalue** < XLogP value >

Acceptable Values: decimal
Default Value: null
Description: It is the XLogP value. If the "RulesOfFive" is set, the "xlogPvalue" should be appointed.

- **acceptors** < number of hydrogen bond acceptors >

Acceptable Values: Integer
Default Value: null
Description: It is the number of hydrogen bond acceptors. If the "RulesOfFive" is set, the "acceptors" should be appointed.

- **donors** < number of hydrogen bond donors >

Acceptable Values: Integer
Default Value: null
Description: It is the number of hydrogen bond donors. If the "RulesOfFive" is set, the "donors" should be appointed.

- **mwvalue** < molecular weight >

Acceptable Values: decimal
Default Value: null
Description: It is the molecular weight. If the "RulesOfFive" is set, the "mwvalue" should be appointed.

- **rotatablebonds** < number of rotatable bonds >

Acceptable Values: integer
Default Value: null
Description: It is the number of rotatable bonds. If the "RulesOfFive" is set, the "rotatablebonds" should be appointed.

- **pcClusterNumber** < number of clusters >

Acceptable Values: integer
Default Value: null
Description: Performing the appointed number of clusters on the final generated ligands by the K-means algorithm.

- **adjustOutSimPer** < fingerprint similarity >

Acceptable Values: decimal
Default Value: 0.999999
Description: This parameter filters similar ligands according to the values of fingerprint similarity when performing the cluster on the final generated ligands. The 0.999999 means it filters the same molecule approximately.

- **selMolMaxWeight** < maximum molecular weight >

Acceptable Values: decimal
Default Value: null
Description: This parameter filters the final generated ligands which is greater than the maximum molecular weight.

- **selMolMinWeight** < minimum molecular weight >

Acceptable Values: decimal
Default Value: null
Description: This parameter filters the final generated ligands that are smaller than the minimum molecular weight.

- **selBindingScore** < cutoff of binding score >

Acceptable Values: decimal
Default Value: null
Description: This parameter filters the final generated ligands that have higher binding scores than the set binding score.

- **obabelPath** < path of obabel >

Acceptable Values: *Deprecated*
Default Value: null
Description: Deprecated. This function is currently useless in this version. It can be omitted when the parameters are assigned.

- **pcClusterdFile** < path of cluster file>

Acceptable Values: String
Default Value: current directory
Description: It represents the path of the cluster file named “PC.dat”. The default value is the current directory, so you can omit it.

- **moloutputdir** < storing directory of final ligands >

Acceptable Values: String
Default Value: current directory
Description: It represents the storage directory of the final ligands. The default value is the current directory named “results”, so you can omit it.

- **coreNum** < number of cores for parallel computing >

Acceptable Values: Integer
Default Value: null
Description: It represents the number of CPU cores for parallel computing. The number of CPU cores used can be set to any value, but MolAICal has an upper limit on the number of CPU cores that can be utilized. For example, MolAICal can use maximally ~15 cores for loading 67 molecular fragments in one job.

- **input_receptor_min** < input receptor file for energy minimization >

Acceptable Values: String
Default Value: The default value is the same in the parameter “**receptorPDB**”.
Description: Sometimes, computing energy minimization for molecules and receptors can be computationally intensive. The purpose of this parameter is to allow inputting a selected portion of the receptor, thereby saving computation time. Its default value is identical to the "receptorPDB" parameter, meaning the full receptor is selected for energy minimization.

- **min_fix_receptor** < file containing fixed receptor atoms specification >

Acceptable Values: String
Default Value: The default value is the same in the parameter “**input_receptor_min**”.
Description: This file is used to assign the fixed receptor atoms or residues when energy minimization is performed. If it is null, empty, or a number, it will mean no atom to be fixed. For example, if it is set to 0, there is no receptor atom to be fixed.

- **min_fix_ligand** < file containing fixed ligand atoms specification >

Acceptable Values: String
Default Value: The default value is the same in the parameter “**startFragFile**”.
Description: This file is used to assign the fixed ligand atoms or residues when energy minimization is performed. If it is null, empty, or a number, it will mean no atom to be fixed. For example, if it is set to 0, there is no ligand atom to be fixed.

- **sel_min_way** < way of energy minimization >

Acceptable Values: String
Default Value: min12
Description: MolAICal provides two energy minimization methods: the default method and the UCSF Chimera-based method. When optimizing final results through energy minimization, this parameter accepts five valid values:

- "min1": Uses only the default energy minimization method
- "min2": Uses only the UCSF Chimera-based energy minimization method
- "min12": First attempts the default method; if optimization fails, switches to the UCSF

Chimera-based method

- "min21": First attempts the UCSF Chimera-based method; if optimization fails, switches to the default method
- "off": Quits the energy minimization process.

The default value is "min12".

➢ **chimera_path** < path of execution program chimera of UCSF Chimera >

Acceptable Values: String

Default Value: chimera

Description: The execution path of UCSF Chimera. This is used for the UCSF Chimera-based energy minimization method. It can have no setting value action. If the UCSF Chimera-based energy minimization method is chosen, it should be given a correct path.

➢ **chimera_charge_method** < charge method for energy minimization in UCSF Chimera >

Acceptable Values: String

Default Value: gas

Description: It is used to set the charge method for energy minimization in UCSF Chimera; the value can be "am1" or "gas". "am1" is am1-bcc, while "gas" is gasteiger. This is just for nonstandard residues containing the specified atoms. However, standard residue atoms (including water, standard amino acids, standard nucleic acids, along with common variants and capping groups) have their atomic charges and types assigned according to the Amber force field parameters.

➢ **chimera_mol_format_1** < saved format of first output molecule if UCSF Chimera is used >

Acceptable Values: String

Default Value: mol2

Description: It is used to set the molecular format of the first output molecule which corresponds to the output ligand file.

➢ **chimera_nsteps_value** < step number of steepest descent minimization >

Acceptable Values: Integer

Default Value: 10

Description: This parameter specifies the number of steepest descent minimization steps executed prior to conjugate gradient minimization. The default value is 10.

➢ **chimera_stepsize_value** < step size of steepest descent minimization >

Acceptable Values: decimal

Default Value: 0.02

Description: This parameter defines the step size for steepest descent minimization. The default value is 0.02.

➢ **chimera_cgsteps_value** < step number of conjugate gradient minimization >

Acceptable Values: Integer

Default Value: 2

Description: This parameter specifies the number of conjugate gradient minimization steps executed after completing steepest descent minimization. The default value is 2.

- **chimera_cgstepsize_value** < step size of conjugate gradient minimization >

Acceptable Values: decimal
Default Value: 0.02
Description: This parameter specifies the step size for conjugate gradient minimization. The default value is 0.02.

- **chimera_interval_value** < interval value for update >

Acceptable Values: Integer
Default Value: 1
Description: Display update rate (in minimization steps). The default value is 1.

- **chimera_mode** < energy minimization mode using UCSF Chimera >

Acceptable Values: Integer
Default Value: 4
Description: There are 5 ways for energy minimization mode using UCSF Chimera:

- Mode 0: Direct call to chimera_command (--input_f1 as commandFile, --input_f2 as outputLog);
- Mode 1: Single molecule minimization;
- Mode 2: Two-molecule minimization;
- Mode 3: Three-molecule minimization with one fixed;
- Mode 4: Four-molecule minimization with two fixed.

The default mode is 4.

- **chimera_sel_value** < selection specification of UCSF Chimera for energy minimization >

Acceptable Values: String
Default Value: "#0 z<0.01 | #1 za<0.01"
Description: The selection commands of atom, residue, and model are used to fix the atom for energy minimization. The default value is "#0 z<0.01 | #1 za<0.01". For more atom specification, see https://www.rbvi.ucsf.edu/chimera/current/docs/UsersGuide/midas/frameatom_spec.html.

- **chimera_output_sel** < selection specification of UCSF Chimera for output >

Acceptable Values: String
Default Value: "#4:UNK"
Description: The selection commands of atom, residue, and model are used to save the results. The default value is "#4:UNK".

- **whether_cal_mce18** < whether to calculate MCE-18 descriptor >

Acceptable Values: String
Default Value: on
Description: Whether to calculate the MCE-18 descriptor for the output results. The default value is on.

2.3. The commands for fragments grow

The command parameters of MolAICal for *de novo* drug design are as below:
-denovo: means to perform the job of *de novo* drug design. Value is “grow”.
-i: input path of the configuration file.
-g: whether to calculate grids, value is “on” or “off”, default value is “on”.
-l: whether to perform fragment grow, value is “on” or “off”, default value is “on”.
-o: whether output results, value is “on” or “off”, default value is “on”.

Here, the example commands for fragment grow are listed:
1) If you want to run the full process of fragment growth, you can use the command:
#> molaical.exe -denovo grow -i InputParFile.dat

2) This command omits the grid generation, only contains the fragments growth, and results outputs.
#> molaical.exe -denovo grow -g off -i InputParFile.dat

3) This command outputs results only.
#> molaical.exe -denovo grow -g off -l off -i InputParFile.dat

4) This command has no output.
#> molaical.exe -denovo grow -g off -l off -o off -i InputParFile.dat

5) This command generates grids only
#> molaical.exe -denovo grow -g on -l off -o off -i InputParFile.dat

6) Only run energy minimization for the output results
#> molaical.exe -denovo grow -g off -l off -o min -i InputParFileCP.dat

3. Molecular docking, virtual screening and deep learning model

3.1. Molecular docking

3.1.1. Molecular docking introduction in MolAICal

MolAICalD, which is derived from Autodock Vina[10] is packaged into the MolAICal soft package.

MolAICalD uses the same license as Autodock Vina (APACHE LICENSE, VERSION 2.0 https://www.apache.org/licenses/LICENSE-2.0). It is responsible for the molecular docking function in the MolAICal software package. MolAICalD is compiled based on boost_1_72_0. The 3130 complexes of proteins-ligands are chosen for assaying the '*docking*' and '*ranking*' power of MolAICalD from the refined-set of PDBbind database[11]. 20 docking poses of the ligand are generated on the receptor pocket in every extracted complex of the PDBbind database. Among the 20 docking poses, the generated ligand that shows the lowest RMSD relative to the original ligand is deemed to possess the best binding affinity. In addition, the Pearson and Spearman correlation coefficients are compared between the experimental binding affinity and predicted binding affinity, which is from the ligand with the lowest RMSD in every extracted complex. For Autodock Vina, the accuracy rate of docking is 54.665% on the 3130 complexes if the ligand with the lowest RMSD has the lowest binding affinity value among the 20 generated ligands. And the Pearson and Spearman correlation coefficients are 0.5259 and 0.5421 based on the experimental binding affinity. (The default weight values in Autodock Vina: "weight_gauss1" is: -0.035579; "weight_gauss2" is: -0.005156; "weight_repulsion" is: 0.84024500000000002; "weight_hydrophobic" is: -0.035069000000000003; "weight_hydrogen" is: -0.58743900000000004). For MolAICalD in MolAICal, the accuracy rate of docking is 54.888% on the 3130 complexes if the ligand with the lowest RMSD has the lowest binding affinity value among the 20 generated ligands. And the Pearson and Spearman correlation coefficients are 0.5335 and 0.5489 based on the experimental binding affinity. It indicates that MolAICal has better 'docking' and 'ranking' power than Autodock Vina. **MolAICalD has now been fully integrated into MolAICal,** allowing users to perform molecular docking directly through "**molaical.exe**" without the need to explicitly use "molaicald" binary executable.

3.1.2. Prepare files for MolAICal docking

The PDBQT file format is necessary for molecular docking by MolAICal. MolAICal supplies the following commands for file preparation via referring to MGLTools[12].

1) Prepare the receptor file. It will generate the PDBQT file with the same name as the receptor file if the suffix is not counted. The command is shown below:
#> molaical.exe -dock receptor -i protein.pdb

2) Prepare the ligand file. It will generate the PDBQT file with the same name as the ligand file if the suffix is not counted. The command is shown below:
#> molaical.exe -dock ligand -i ligand.pdb

3) Adding hydrogen to the molecular file. The input file can be in PDB, PDBQ, PDBQT, SYBYL mol2, or PQR format. The output file is in PDBQT format. The command is shown below:
#> molaical.exe -dock addh -i ligand.pdbqt

4) Transform PDBQT format into PBD format. The command is shown below:
#> molaical.exe -dock pdbqt2pdb -i ligand.pdbqt

5) Calculate root mean square deviation (RMSD) based on two PDBQT format files in the same atom order. The command is shown below:
#> molaical.exe -dock rmsd -f 1.pdbqt -s 2.pdbqt

Note: -f corresponds to the first file; -s corresponds to the second file.

3.1.3. Molecular docking by MolAICal

MolAICalD has now been fully integrated into MolAICal, allowing users to perform molecular docking directly through "**molaical.exe**" without the need to explicitly use "**molaicald**" binary executable.

usage: **sd** [-dock <arg>] [-h] [-r <arg>] [-i]
Run molecular docking:

-dock <arg>	Carry out molecular docking with the argument "**sd**". It should be placed before "**-i**".
-h	Display help information. It should be placed before "**-i**".
-r <arg>	It corresponds to the receptor name. And it will sequentially split output results into separate ligand files, then minimize all ligands. It should be placed before "**-i**".
-i	Input information for molecular docking. It should be placed before "**-i**".

Example:
#> molaical.exe -dock sd -r protein.pdb -i --config conf.txt --ligand ligand.pdbqt

The parameter following "-i" belongs to MolAICalD, specifically as follows:

Input:	
--receptor arg	rigid part of the receptor (PDBQT)
--flex arg	flexible side chains, if any (PDBQT)
--ligand arg	ligand (PDBQT)
Search space (required):	
--center_x arg	X coordinate of the center
--center_y arg	Y coordinate of the center
--center_z arg	Z coordinate of the center
--size_x arg	size in the X dimension (Angstroms)
--size_y arg	size in the Y dimension (Angstroms)
--size_z arg	size in the Z dimension (Angstroms)
Output (optional):	
--out arg	output models (PDBQT), the default is chosen based on the ligand file name

--log arg	optionally, write log file
Advanced options (see the manual):	
--score_only	score only - search space can be omitted
--local_only	do local search only
--randomize_only	randomize input, attempting to avoid clashes
--weight_gauss1 arg (=-0.049780097249188818)	gauss_1 weight
--weight_gauss2 arg (=-0.0056280282979584151)	gauss_2 weight
--weight_repulsion arg (=0.84004961455022986)	repulsion weight
--weight_hydrophobic arg (=-0.035854894848047061)	hydrophobic weight
--weight_hydrogen arg (=-0.62113230234751393)	Hydrogen bond weight
--weight_rot arg (=0.058459999999999998)	N_rot weight
Misc (optional):	
--cpu arg	the number of CPUs to use (the default is to try to detect the number of CPUs or, failing that, use 1)
--seed arg	explicit random seed
--exhaustiveness arg (=8)	exhaustiveness of the global search (roughly proportional to time): "1+". In the notation **"1+"**, the "1" represents the base value, while the "+" indicates scalability, allowing users to increase the value as needed to improve precision. This allows users to balance precision and computational resources.
--num_modes arg (=9)	maximum number of binding modes to generate
--energy_range arg (=3)	maximum energy difference between the best binding mode and the worst one displayed (kcal/mol)
Configuration file (optional):	
--config arg	the above options can be put here
Information (optional):	
--help	display usage summary
--help_advanced	display usage summary with advanced options
--version	display program version

It can put the above parameters into a file without the prefix "--". For example, the content of the configuration file **"conf.txt"** can be as below:

```
receptor = pro.pdbqt
out = all.pdbqt

cpu = 1
center_x = -10.733
center_y = 12.416
```

```
center_z = 68.829

size_x = 25
size_y = 30
size_z = 25

num_modes = 1
```

The example commands for molecular docking are as below:

1) Get the help information for molecule docking before "-i"

#> molaical.exe -dock sd -h

2) Get the help information for MolAICalD after "-i"

#> molaical.exe -dock sd -i --help

3) Get the advanced help information for MolAICalD after "-i"

#> molaical.exe -dock sd -i --help_advanced

4) Perform molecular docking, split the PDBQT result files, and conduct molecular energy minimization, where "-r" specifies the receptor name, thereby optimizing the ligand's energy minimization based on the receptor.

#> molaical.exe -dock sd -r protein.pdb -i --config conf.txt --ligand ligand.pdbqt

5) If there is no parameter "-r", only perform molecular docking

#> molaical.exe -dock sd -i --config conf.txt --ligand ligand.pdbqt

6) Show --out parameters for assigning the output file name, and perform molecular docking, split the PDBQT result files, and conduct molecular energy minimization, where "-r" specifies the receptor name, thereby optimizing the ligand's energy minimization based on the receptor.

#> molaical.exe -dock sd -r protein.pdb -i --config conf.txt --ligand ligand.pdbqt --out out_all.pdbqt

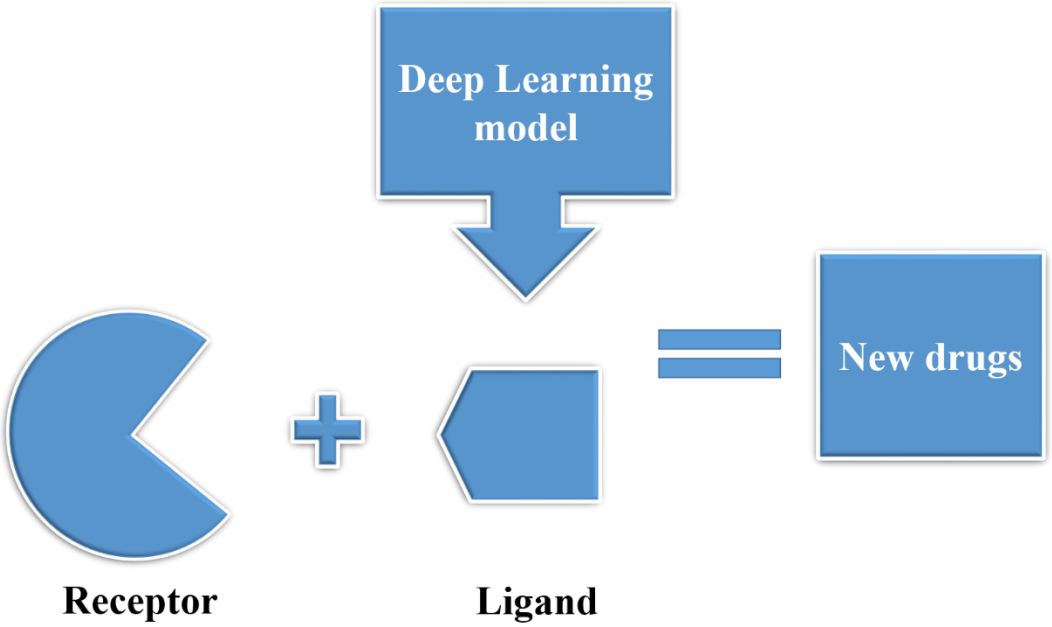
7) Show --out parameters for assigning the output file name, and only perform molecular docking

#> molaical.exe -dock sd -i --config conf.txt --ligand ligand.pdbqt --out out_all.pdbqt

Note: The parameters "-dock" and "-r" are built-in parameters of MolAICal, while the parameter following "-i" belongs to the molecular docking module MolAICalD. **Mandatory requirement: "-dock" and "-r" must be placed before "-i".**

3.2. Drug virtual screening by MolAICal and DL model

3.2.1. Theory



Part I Figure 3.1.1 Virtual screening based on molecular docking and deep learning models.

Virtual screening (VS) is a computational technique employed in drug discovery to identify small molecules with a high likelihood of binding to specific drug targets, such as protein receptors or DNA. Unlike de novo drug design, which generates novel molecular structures from scratch, VS focuses on evaluating existing or virtual compound libraries. VS encompasses two primary methodologies: ligand-based and structure-based virtual screening[13]. In particular, structure-based virtual screening utilizes molecular docking to identify optimal binding conformations of ligands within the protein target's binding site. This process is followed by the application of a scoring function to predict the binding affinity between the ligands and the receptor. Molecular docking is conceptually analogous to the Lock and Key Model, where the ligand (Key) fits into the receptor's binding site (Lock) (see Part I Figure 3.1.1). Autodock Vina[10] is a widely recognized open-source molecular docking software that has been extensively studied and applied in the field of drug discovery. MolAICalD, a derivative of Autodock Vina, enhances docking accuracy and ranking capabilities specifically for drug design applications. The scoring function of Autodock Vina has been thoroughly investigated and optimized to accurately estimate the binding affinity between ligands and receptors. The Vinardo score[14], a modified version of Autodock Vina's scoring function, offers improved predictions of binding affinity between ligands and receptors.

Deep learning, a subset of machine learning and artificial intelligence, utilizes multiple layers to extract progressively higher-level features from raw input data. Deep learning has been successfully used in drug discovery[2] and medicine[5]. It enables the training of molecular generative models, such as those based on generative adversarial networks (GANs), diffusion models, flow-based models,

variational autoencoder (VAE), autoregressive models, and energy-based models[6-7, 15], which can produce one-dimensional (1D) sequences, two-dimensional (2D) structural representations, or three-dimensional (3D) structures of molecules. Rational drug design often requires the analysis of three-dimensional ligand structures within the receptor's binding pocket. De novo drug design of MolAICal offers a methodology to generate three-dimensional drug candidates within the receptor's binding pocket, leveraging both deep learning models and classical algorithms, such as genetic algorithms. Additionally, molecular docking provides an alternative approach that integrates deep learning models with classical computational methods. In the context of virtual screening, MolAICal employs a deep learning model to generate a specified number of drug candidates. Subsequently, the molecular docking module, MolAICalD, is utilized to perform docking simulations, thereby assessing the binding affinity between the generated ligands and the target receptor (see Part I Figure 3.1.1). Furthermore, MolAICal can leverage predefined molecular databases for virtual screening by invoking its molecular docking program.

3.2.2. Parameter introduction of MolAICalD configuration file

This part introduces the parameters of drug virtual screening by MolAICal, the deep learning (DL) model (**Note:** only 64-bit MolAICal can perform the AI and molecular docking completely. 32-bit MolAICal is deprecated. Please run this process on a 64-bit operating system.). Before you run this drug virtual screening, please prepare the necessary MolAICalD configuration files named "conf.txt". The pdbqt format files of the protein must be together with "conf.txt". **The content "conf.txt" is as follows:**

```
receptor = protein.pdbqt

cpu = 1
center_x = -30.011
center_y = 1.665
center_z = -36.581

size_x = 30
size_y = 30
size_z = 30

num_modes = 3
```

Where the "receptor" represents the protein file, "cpu" means the number of CPU cores for running, the "center_x", "center_y" and "center_z" are the x, y, z coordinates of box center you set, the "size_x", "size_y" and "size_z" are lengths along x, y, z axis of box you set, the "num_modes" is

the number of generated conformations by MolAICalD.

MolAICalD has now been fully integrated into MolAICal, allowing users to perform molecular docking directly through "**molaical.exe**" without the need to explicitly use "**molaicald**" binary executable. For more detailed information, please see the part of Molecular docking by MolAICal.

3.2.3. Command introduction for AI virtual screening

The command parameters of MolAICal for AI drug virtual screening are as below:

-dock: value is AI, which means performing drug virtual screening by AI.
-s: selection of a deep learning model. Values are "ZINCMol", "FDAMol", "FDAFrag".
-n: number of generated molecules by the deep learning model.
-nf: number of generated molecules in one folder. It avoids very huge of molecules in one folder.
-p: the generated file name by the AI module.
-nc: number of CPU cores for running drug virtual screening.
-g: switch AI molecular generations. "on" means generation. "off" means no generation.
-b: switch molecular format conversion. "on" means conversion. "off" means no conversion.
-v: switch virtual screening (VS) based on docking. "on" means VS. "off" means no VS.
-m: the configuration file for molecular docking VS in the appointed directory. The default value is: "conf.txt".

Here, the example commands for AI drug virtual screenings are listed:

1) The simple command, the omitted parameters use the default values
#> molaical.exe -dock AI -s FDAMol -n 30 -nf 10 -nc 3

2) Run full drug virtual screening process by MolAICal, deep learning model, and MolAICalD
#> molaical.exe -dock AI -s FDAMol -n 30 -nf 10 -nc 3 -g on -b on -v on

3) Only run molecular artificial intelligence (AI) generation.
#> molaical.exe -dock AI -s FDAMol -n 30 -nf 10 -nc 3 -g on -b off -v off

4) Only run conversion of molecular formats only
#> molaical.exe -dock AI -s FDAMol -n 30 -nf 10 -nc 3 -g off -b on -v off

5) Only run MolAICalD docking. So you can customize your own AI SMILES file, but this file must be named "tmpGen.dat"
#> molaical.exe -dock AI -s FDAMol -n 30 -nf 10 -nc 3 -g off -b off -v on

6) Command for virtual screening with the assigned SMILES generated file "genvs.txt" by calling deep learning module
#> molaical.exe -dock AI -s FDAMol -n 30 -p genvs.txt -nf 10 -nc 3 -g on -b on -v on

7) Command contained full parameters with the appointed docking configuration file “conf2.dat”
#> molaical.exe -dock AI -s ZINCMol -n 30 -p genvs.txt -nf 10 -nc 3 -g on -b on -v on -m conf2.dat

8) This command has no drug virtual screening task
#> molaical.exe -dock AI -s FDAMol -n 30 -p genvs.txt -nf 10 -nc 3 -g on -b on -v off -m conf2.dat

3.2.4. Command introduction for virtual screening

This command is for virtual screening (VS) based on the known database, simply via MolAICalD. The command parameters of MolAICal for drug virtual screening are as below:

-dock: value is vs, means performing drug virtual screening.
-i: input directory which contains the molecules files. The format of molecule in the directory can be in PDB, PDBQ, PDBQT, SYBYL PQR or mol2 format.
-k: the files contain the keyword for VS. The default value is “.mol2”.
-nc: number of CPU cores for running drug virtual screening.
-b: switch molecular format conversion. “on” means conversion. “off” means no conversion.
-v: switch virtual screening (VS) based on docking. “on” means VS. “off” means no VS.
-m: the configuration file for molecular docking VS in the appointed directory. The default value is: “conf.txt”.
-p: can accept additional parameters for format conversion. The value following it must be enclosed in double quotes "". If the value contains multiple parameters, they must be separated by commas ",". The default value is an empty string "". The value "-Z" after the argument **'-p'** will inactivate all active torsions to realize the rigid docking.

Before running the VS process, files should be split into directories, for instance:
#> molaical.exe -tool mol2 -w split -n 1 -v true -m number -i "E:\all.mol2" -o "E:\split"
Note: You can use the second line string of the mol2 file as the file name by using the below command:
#> molaical.exe -tool split -w split -n 1 -v true -i "D:\aa.mol2" -o "D:\split"

Here, the example commands for drug virtual screenings are listed:

1) The simple command, the omitted parameters use the default values.
#> molaical.exe -dock vs -i D:\split

2) The simple command, the docked molecular format, and the running CPU cores are appointed.
#> molaical.exe -dock vs -i D:\split -k .mol2 -nc 3

3) Run the full drug virtual screening process.
#> molaical.exe -dock vs -i D:\split -k .mol2 -b on -v on -m conf2.dat -nc 3

4) Run the drug virtual screening process without molecular format conversion.
#> molaical.exe -dock vs -i D:\split -k .mol2 -b off -v on -m “conf2.dat” -nc 3

5) Run the drug virtual screening process by using the molecular PDBQT format directly.
#> molaical.exe -dock vs -i D:\split -k .pdbqt -b off -v on -nc 3

6) Rigid docking for the drug virtual screening by input argument "-Z" after '-p'
#> molaical.exe -dock vs -i linear -k pdb -nc 3 -p "-Z"

3.3. MM/GBSA calculation

MolAICal provides two methods for calculating MM/GBSA. One method includes detailed operational steps, which help readers learn the detailed process of MM/GBSA. The other is a fast calculation method that omits most operational steps, enabling quick MM/GBSA computation. Users can choose the calculation method based on their needs. Under the same parameter settings, the results computed by both methods are identical.

3.3.1. MM/GBSA calculation by MolAICal and NAMD

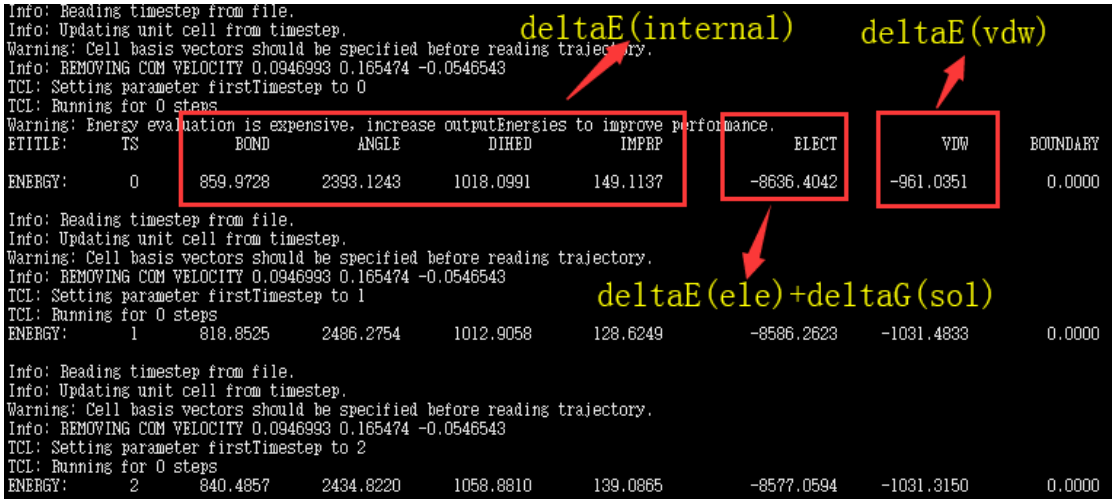
Molecular mechanics generalized Born surface area (MM/GBSA)[16-17] is a popular way to estimate the binding free energy between two molecules, such as ligand-protein, protein and protein, etc. Here, MM/GBSA is calculated by MolAICal on the basis of the MD log file of the NAMD software. The MM/GBSA is estimated by the following equations:

$$\Delta G_{bind} = \Delta H - T\Delta S \approx \Delta E_{MM} + \Delta G_{sol} - T\Delta S$$

$$\Delta E_{MM} = \Delta E_{internal} + \Delta E_{ele} + \Delta E_{vdw}$$

$$\Delta G_{sol} = \Delta G_{GB} + \Delta G_{SA}$$

Where ΔE_{MM}, ΔG_{sol}, and $T\Delta S$ represent the gas phase MM energy, solvation free energy (sum of polar contribution ΔG_{GB} and nonpolar contribution ΔG_{SA}), and conformational entropy, respectively. ΔE_{MM} contains van der Waals energy ΔE_{vdw}, electrostatic $\Delta E_{ele,}$ and $\Delta E_{internal}$ of bond, angle, and dihedral energies (Dihedral angles include two types: proper dihedral angles and improper dihedral angles). The unit of ΔG_{bind} is kcal/mol.




```
Info: Reading timestep from file.
Info: Updating unit cell from timestep.
Warning: Cell basis vectors should be specified before reading trajectory.
Info: REMOVING COM VELOCITY 0.0946993 0.165474 -0.0546543
TCL: Setting parameter firstTimestep to 0
TCL: Running for 0 steps
Warning: Energy evaluation is expensive, increase outputEnergies to improve performance.
ETITLE:      TS           BOND          ANGLE          DIHED          IMPRP           ELECT            VDW       BOUNDARY

ENERGY:       0       859.9728      2393.1243      1018.0991       149.1137       -8636.4042      -961.0351         0.0000

Info: Reading timestep from file.
Info: Updating unit cell from timestep.
Warning: Cell basis vectors should be specified before reading trajectory.
Info: REMOVING COM VELOCITY 0.0946993 0.165474 -0.0546543
TCL: Setting parameter firstTimestep to 1
TCL: Running for 0 steps
ENERGY:       1       818.8525      2486.2754      1012.9058       128.6249       -8586.2623      -1031.4833         0.0000

Info: Reading timestep from file.
Info: Updating unit cell from timestep.
Warning: Cell basis vectors should be specified before reading trajectory.
Info: REMOVING COM VELOCITY 0.0946993 0.165474 -0.0546543
TCL: Setting parameter firstTimestep to 2
TCL: Running for 0 steps
ENERGY:       2       840.4857      2434.8220      1058.8810       139.0865       -8577.0594      -1031.3150         0.0000
```

Part I Figure 3.3.1 The detailed information of the NAMD running log file

If there are no binding-induced structural changes in the process of molecular dynamics (MD) simulations, or very similar structural changes among different systems, the entropy calculation can be omitted. Currently, the MolAICal can evaluate the binding free energy without entropy calculation based on the MD results log of NAMD. The variables in the above equations correspond to Part I Figure 3.3.1.

The parameters for calculating binding free energy are as follows:

-mmgbsa: means MM/GBSA calculation.
-c: the MD log of receptor and ligand.
-r: the MD log of receptor.
-l: the MD log of the ligand.

Example command line as below:

#> molaical.exe -mmgbsa -c "D:\complex.log" -r "D:\protein.log" -l "D:\ligand.log"

3.3.2. MM/PBSA and fast MM/GBSA calculations

Here, MolAICal offers a fast and similar approach for MM/GBSA and MM/PBSA calculations. Below are the calculation parameters for MM/GBSA and MM/PBSA:

usage: mmpbgbsa	
-f <arg>	The path of the input file for MM/PB/GB/SA calculation.
-h	Print help information.
-i	The command arguments of the program mmpbgbsa. It includes all command-line arguments of external programs, but the '-i' parameter must be the last one specified, while all other identifiers (e.g., '-f', etc.) must come before '-i'.
-mmpbgbsa	MM/PB/GBSA calculation.
-n <arg>	The name of the executed program VMD, the default value is 'vmd'.

-o <arg>	Whether to print the output of the program.
-p <arg>	The path of the program VMD. The MolAICal environment setup command can be used to configure it once, eliminating the need for tedious path input later.

Usage: -mmgbpbsa -f xxx.vmd -i -system_topology top_file -system_coords coords_file -trajectory_files filename [-args...]

Mandatory arguments:

-system_topology or -s	<topology filename>
-trajectory_files or -t	<trajectory filename>
-system_coords or -c	<coordinates filename>

Optional arguments:

-namd_config_file	<namd custom configuration file>
-pb_gb_file or -pbe	<PB or GB custom configuration file>
-pb_gb_temperature	<temperature> -- default: 310K
-topology_type	<topology filetype> -- default: auto
-trj_type	<trajectory filetype> -- default: auto
-force_parameters or -ff	<force field parameters> -- default: auto-detect
-output_filename	<output filename> -- default: mmgbpbsa.log
-debug	<debug level> -- default: 0. It can be 0, 1, 2. If '2' is selected, it will save all generated temporary files. If '1' is selected, it will save some of the generated temporary files. If '0' is selected, it will only save the result files.
-first_frame	<first frame> -- default: 0
-last_frame	<last frame> -- default: -1
-frame_stride	<stride> -- default: 1
-complex_selection or -cs	<complex selection: having the same rules as "atomselect" command in VMD> -- default: ""
-receptor_selection or -rs	<receptor selection: having the same rules as "atomselect" command in VMD> -- default: ""
-ligand_selection or -ls	<ligand selection: having the same rules as "atomselect" command in VMD> -- default: ""
-molecular_mechanics	<do gas-phase calc> -- default: 0
-namd_executable or -rn	<path to NAMD> -- default: "namd2"
-namd_cutoff	<cutoff for MD simulations> -- default: 16.0
-namd_switching	<"on" means smooth switching function truncates van der Waals potential energy at the cutoff distance. "off" will close smooth switching function> -- default: on
-namd_switchdist	<distance at which to activate switching function> -- default: 15.0
-mm_dielectric	<dielectric constant> -- default: 1.0
-poisson_boltzmann or -pb	<do PB calculation> -- default: 2. It can be '0, 1, 2'. If '0' is selected, it will calculate MM/GBSA. If '1' is selected, it will calculate MM/PBSA with the program Delphi. If '2' is selected, it will calculate MM/PBSA with the program APBS.
-pb_executable or -pbe	<path to DelPhi/APBS> -- default: "apbs"
-pb_radii_type	<type of PB radii> -- default: bondi

-pb_boundary_flag	<boundary condition> -- default: sdh
-pb_charge_method	<charge method> -- default: spl0
-ion_charge_apbs_negtive	<total anionic charge> -- default: -1.0
-ion_charge_apbs_positive	<total cationic charge> -- default: 1.0
-ion_apbs_radius	<mobile ion species radius> -- default: spl0
-generalized_born or -gb	<do GB calculation> -- default: 0
-gb_exterior_dielectric	<external dielectric> -- default: 78.5
-pb_gb_ion_concentration	<ion concentration> -- default: 0.15
-gb_surface_area	<do LCPO calculation> -- default: 0
-gb_surface_gamma	<surface tension> -- default: 0.0072
-surface_accessibility	<do SA calculation> -- default: 1, it can be 1 or 2, "if '1', SASA calculation is performed by VMD; if '2', SASA calculation is performed by APBS.
-sa_input_apbs	<APBS input file for SA calculation> -- default: ""
-sa_radii_type	<type of SA radii> -- default: bondi
-sa_gamma	<surface tension> -- default: 0.0072
-sa_beta	<surface offset> -- default: 0.0
-sa_probe_radius	<radius of probe> -- default: 1.4
-sa_displacement_apbs	<displacement parameter for finite-difference-based force calculations (surface area derivatives), with unit: Å.> -- default: 0.2
-sa_sample_points	<number of samples> -- default: 500
-namd_seed	<namd seed for reproducibility> -- default: 12345
-num_calc_cores	<number of CPU cores for calculations> -- default: 1

3.3.2.1. MM/PBSA calculations

The MM/PBSA (Molecular Mechanics/Poisson-Boltzmann Surface Area) method computes binding free energies through the summation of gas-phase interaction energy (ΔE_{MM}) and solvation free energy (ΔG_{sol}). Here, ΔE_{MM} consists of molecular mechanical (MM) terms (internal energy, van der Waals and electrostatic interactions), while ΔG_{sol} is further decomposed into polar (Poisson-Boltzmann) and nonpolar (Surface Area) contributions. Here, MM/PBSA is calculated by MolAICal, [APBS (https://apbs.readthedocs.io)[18] or DelPhi (http://compbio.clemson.edu/lab/delphisw)[19-20]], VMD, and NAMD. The MM/PBSA is estimated by the following equations:

$$\Delta G_{bind} = \Delta H - T\Delta S \approx \Delta E_{MM} + \Delta G_{sol} - T\Delta S$$

$$\Delta E_{MM} = \Delta E_{internal} + \Delta E_{ele} + \Delta E_{vdw}$$

$$\Delta G_{sol} = \Delta G_{PB} + \Delta G_{SA}$$

$$\Delta G_{SA} = \gamma * SASA + \beta$$

Where ΔE_{MM}, ΔG_{sol}, and $T\Delta S$ represent the gas phase MM energy, solvation free energy (sum of polar (Poisson-Boltzmann) ΔG_{PB} and nonpolar (Surface Area) ΔG_{SA} contributions), and conformational entropy, respectively. ΔE_{MM} contains van der Waals energy ΔE_{vdw}, electrostatic ΔE_{ele}, and $\Delta E_{internal}$ of bond, angle, and dihedral energies (Dihedral angles include two types: proper

dihedral angles and improper dihedral angles). SASA is the solvent-accessible surface area. γ is the coefficient, and β is the offset. The unit of ΔG_{bind} is kcal/mol. If there are no binding-induced structural changes in the process of molecular dynamics (MD) simulations, or very similar structural changes among different systems, the entropy calculation can be omitted.

The PB energies calculated based on APBS and DelPhi may differ significantly due to variations in their parameter settings and inherent algorithmic differences. Therefore, for comparative studies or investigations of specific molecules or systems, it is advisable to use calculations exclusively from either APBS or DelPhi.

1) The contents below are the VMD script (e.g., mmpbsa_apbs.vmd) that can be used to calculate MM/PBSA based on APBS:

```
source mm_gbpb_sa.tcl
mmgbpbsa \
-system_topology            mpro.psf \
-system_coords              mpro.pdb \
-trajectory_files           mpro.dcd \
-debug          0 \
-output_filename              mmpbsa_apbs.log \
-force_parameters           par_all36_prot.prm \
-force_parameters           par_all36_cgenff.prm \
-force_parameters           ligand.str \
-force_parameters           toppar_water_ions.str \
-complex_selection            "segname A C" \
-receptor_selection           "segname A" \
-ligand_selection           "segname C" \
-first_frame          0 \
-last_frame           -1 \
-frame_stride         1 \
-namd_executable        E:/workdir/MolAICal/method/mmgbsa_v1/mmpbsa/namd2/namd2 \
-mm_dielectric              1      \
-poisson_boltzmann              2 \
-pb_radii_type        mparse \
-pb_boundary_flag      mdh \
-pb_charge_method      spl2 \
-surface_accessibility              2 \
-sa_radii_type        mparse \
-sa_gamma               0.0072 \
-sa_beta                0.00 \
-pb_executable E:/workdir/MolCalSys/ideamac/out/production/MolAICal/mtools/etools/apbs/bin/apbs
quit
```

Above parameter Specifications:

✧ For explanations of the content parameters in the above VMD script, consult the preceding documentation.

- ✧ Each parameter must occupy a distinct line and be **terminated** with a trailing backslash " \".
- ✧ Each line shall follow the schema: parameter_name followed by spaces, the corresponding parameter value, and the closing delimiter " \".
- ✧ The **final line** beginning with a **hyphen '-'** must **not include** the terminating delimiter " \".
- ✧ Conclude the script with either **'exit'** or **'quit'** to ensure program termination without remaining idle in memory.
- ✧ **Ignore** the first line of the script that starts with the **"source"** character; MolAICal will **automatically** locate the corresponding script path and **load it**.
- ✧ If '-poisson_boltzmann' is set to '2', it will calculate MM/PBSA with the program APBS. The default value of '-poisson_boltzmann' is '2'. So if '-poisson_boltzmann' is not set, it will use the default value '2'. If '-pb_executable' is not set, MolAICal will call APBS automatically.
- ✧ When running MMPB/GB/SA functionality, and setting environment variables using MolAICal (e.g., #> molaical.exe -call set -n VMD -p "path"), the variable names in MMPB/GB/SA corresponding to '-n' are fixed: 'vmd', 'namd', and 'delphi' (case insensitive). For example:

```
#> molaical.exe -call set -n VMD -p "C:\Program Files (x86)\University of Illinois\VMD\vmd.exe"
#> molaical.exe -call set -n namd -p "d:\namd2\namd2.exe"
#> molaical.exe -call set -n delphi -p "E:/delphi/delphicpp_v8.4.3_windows_x64.exe"
```

2) The contents below are the VMD script (e.g., mmpbsa_delphi.vmd) that can be used to calculate MM/PBSA based on DelPhi:

```
source mm_gbpb_sa.tcl
mmgbpbsa \
-system_topology            mpro.psf \
-system_coords              mpro.pdb \
-trajectory_files          mpro.dcd \
-debug         0 \
-output_filename           mmpbsa_delphi.log \
-force_parameters           par_all36_prot.prm \
-force_parameters           par_all36_cgenff.prm \
-force_parameters           ligand.str \
-force_parameters           toppar_water_ions.str \
-complex_selection            "segname A C" \
-receptor_selection           "segname A" \
-ligand_selection           "segname C" \
-first_frame         0 \
-last_frame          -1 \
-frame_stride        1 \
-namd_executable       E:/workdir/MolAICal/method/mmgbsa_v1/mmpbsa/namd2/namd2 \
-mm_dielectric              1.0    \
-poisson_boltzmann              1 \
-pb_radii_type       mparse \
-pb_boundary_flag     mdh \
-pb_charge_method     spl2 \
```

```
-surface_accessibility              1 \
-sa_radii_type       mparse \
-sa_gamma               0.0072 \
-sa_beta                0.00 \
-pb_executable E:/workdir/MolAICal/method/mmpbsa/8.5/delphicpp_v8.4.3_windows_x64.exe
quit
```

Above parameter Specifications:

- ✧ If '-poisson_boltzmann' is set to '1', it will calculate MM/PBSA with the program Delphi. If '-pb_executable' is not set, MolAICal will search the path environment for external programs in MolAICal and call Delphi automatically.

3) If users want to customize the input arguments of NAMD, APBS, SASA, or DelPhi, users can input the customized external file by adding the arguments: '-namd_config_file', '-pb_gb_file', and '-sa_input_apbs':

```
-namd_config_file          input.namd \
-pb_gb_file                Delphi_or_APBS.input \
-sa_input_apbs             input_sa.apbs \
```

Above parameter Specifications:

- ✧ For explanations of the content parameters such as '-namd_config_file', '-pb_gb_file', and '-sa_input_apbs', consult the preceding documentation.
- ✧ Users can customize the wanted parameters of NAMD in the file "input.namd" for molecular mechanics (MM) energy calculations, and customize the wanted parameters of APBS or DelPhi in the file "Delphi_or_APBS.input" for PB energy calculations, and customize the wanted parameters of APBS or SASA calculations.

Example command line as below:

1) Print help information for MM/PB/GB/SA
#> molaical.exe -mmpbgbsa -h

2) Calculate MM/PBSA with loading VMD automatically
#> molaical.exe -mmpbgbsa -n vmd -f mmpbsa_apbs.vmd

3) Calculate MM/PBSA by assigning the executed VMD path
#> molaical.exe -mmpbgbsa -n vmd -f mmpbsa_apbs.vmd -p "D:\Program Files (x86)\University of Illinois\VMD\vmd.exe"

4) Calculate MM/PBSA with loading VMD automatically. The following argument content of '-i' can be used to modify the arguments of the above VMD script.
#> molaical.exe -mmpbgbsa -n vmd -f mmpbsa_apbs.vmd -i -system_topology ../complex.psf

5) Calculate MM/PBSA with loading VMD automatically. The following argument content of '-i' can be used to modify the arguments of the above VMD script.

#> molaical.exe -mmpbgbsa -n vmd -f mmpbsa_apbs.vmd -i -system_topology ../complex.psf -system_coords ../complex.pdb

6) Calculate MM/PBSA using default parameters with loading VMD automatically. The following argument content of '-i' can be used to modify the arguments of the above VMD script.
#> molaical.exe -mmpbgbsa -f mmpbsa_apbs.vmd -i -system_topology ../complex.psf -system_coords ../complex.pdb

In this result file, when **'KINETIC'** is run multiple times, the **value differs each time** due to the **random motion of molecules at a given temperature, resulting in varying kinetic energy.** This can be tested: if the temperature is set to absolute zero, KINETIC will remain unchanged because molecular random motion ceases. The item of **'KINETIC'** can be omitted when MM/GBSA or MM/PBSA are calculated.

3.3.2.2. Fast MM/GBSA calculations

MolAICal offers a fast calculation method for MM/GBSA that does not require the complex steps. For specific mathematical principles and calculation formulas, please refer to the previous introduction.

The contents below are the VMD script (e.g., mmgbsa.vmd) that can be used to calculate MM/GBSA quickly.

```
source "E:/workdir/MolCalSys/ideamac/out/production/MolAICal1.40//scripts/mm_gbpb_sa.tcl"
mmgbpbsa \
-system_coords ../complex.pdb \
-system_topology ../complex.psf \
-trajectory_files ../complex.dcd \
-debug 0 \
-output_filename mmgbsa_results.log \
-force_parameters ../par_all36_prot.prm \
-force_parameters ../par_all36_cgenff.prm \
-force_parameters ../ligand.str \
-force_parameters ../toppar_water_ions.str \
-complex_selection "segname A C" \
-receptor_selection "segname A" \
-ligand_selection "segname C" \
-pb_gb_temperature 310.0 \
-first_frame 0 \
-last_frame -1 \
-frame_stride 1 \
-mm_dielectric 1 \
-poisson_boltzmann 0 \
```

```
-surface_accessibility              0 \
-gb_surface_gamma    0.0072 \
-generalized_born          1 \
-gb_surface_area         1 \
-namd_executable E:/workdir/MolAICal/method/mmgbsa_v1/mmpbsa/namd2/namd2.exe
quit
```

Above parameter Specifications:

- ✧ For explanations of the content parameters in the above VMD script, consult the preceding documentation.
- ✧ Each parameter must occupy a distinct line and be **terminated** with a trailing backslash " \";
- ✧ Each line shall follow the schema: parameter_name followed by spaces, the corresponding parameter value, and the closing delimiter " \".
- ✧ The **final line** beginning with a **hyphen '-'** must **not include** the terminating delimiter " \".
- ✧ Conclude the script with either **'exit'** or **'quit'** to ensure program termination without remaining idle in memory.
- ✧ If '-poisson_boltzmann' is set to '0', it will calculate MM/GBSA.
- ✧ **Ignore** the first line of the script that starts with the **"source"** character; MolAICal will **automatically** locate the corresponding script path and **load it**.
- ✧ **If adding the augment** "-namd_config_file input.namd \" **in the above VMD script**, it will automatically use the external template file "input.namd" that is prepared manually. MolAICal will modify the augment "coordinates, structure, and coorfile" for complex (1st molecule), protein (2nd molecule), and ligand (3rd molecule). This allows users to customize input parameters of NAMD in the file "input.namd".
- ✧ When running MMPB/GB/SA functionality, and setting environment variables using MolAICal (e.g., #> molaical.exe -call set -n VMD -p "path") the variable names in MMPB/GB/SA corresponding to '-n' are fixed: 'vmd', 'namd', and 'delphi' (case insensitive).

Example command line as below:

1) Print help information for mmpbgbsa
#> molaical.exe -mmpbgbsa -h

2) Calculate MM/GBSA with loading VMD automatically
#> molaical.exe -mmpbgbsa -n vmd -f mmgbsa.vmd

3) Calculate MM/GBSA by assigning the executed VMD path
#> molaical.exe -mmpbgbsa -n vmd -f mmgbsa.vmd -p "D:\Program Files (x86)\University of Illinois\VMD\vmd.exe"

4) Calculate MM/GBSA with loading VMD automatically. The following argument content of '-i' can be used to modify the arguments of the above VMD script.
#> molaical.exe -mmpbgbsa -n vmd -f mmgbsa.vmd -i -system_topology ../complex.psf

5) Calculate MM/GBSA with loading VMD automatically. The following argument content of '-i' can be used to modify the arguments of the above VMD script.
#> molaical.exe -mmpbgbsa -n vmd -f mmgbsa.vmd -i -system_topology ../complex.psf -system_coords ../complex.pdb

6) Calculate MM/GBSA using default parameters with loading VMD automatically. The following argument content of '-i' can be used to modify the arguments of the above VMD script.
#> molaical.exe -mmpbgbsa -f mmgbsa.vmd -i -system_topology ../complex.psf -system_coords ../complex.pdb

In this result file, when **'KINETIC'** is run multiple times, the **value differs each time** due to the **random motion of molecules at a given temperature, resulting in varying kinetic energy.** This can be tested: if the temperature is set to absolute zero, KINETIC will remain unchanged because molecular random motion ceases. The item of **'KINETIC'** can be omitted when MM/GBSA or MM/PBSA are calculated.

3.3.2.3. MM/PB/GBSA calculations via commands

MolAICal also supports MM/PBSA and MM/GBSA calculations via pure command-line interfaces. The meaning of specific parameters aligns with the descriptions in the "MM/PBSA and fast MM/GBSA calculations" section, while other parameters can be referenced in the "Call external programs by MolAICal" section. MolAICal enables MM/PBSA and MM/GBSA computations using NAMD-generated DCD, PDB, and PSF files. Alternatively, it can perform calculations with only a single PDB and PSF file. However, if these files (PDB and PSF files) lack energy minimization or prior molecular dynamics (MD) simulations, results may exhibit significant deviations. Hence, further energy minimization or MD refinement is strongly recommended. MolAICal provides an integrated interface for MD simulations; details are available in the "Call external programs by MolAICal" section.

To ensure the reproducibility of MM/PBSA and MM/GBSA results, the following conditions must be strictly met:

- ✧ Use the **CPU version of NAMD**, such as "Linux-x86_64-multicore";
- ✧ **NAMD** must employ **only 1 CPU core** for computation;
- ✧ If using a custom NAMD input file (e.g., "md_configure.conf"), a fixed random seed must be set, such as **"seed 12345"** in "md_configure.conf".

Example command line as below:

◆ Calculating MM/PB/GBSA based on a PDB and PSF file only

1) Print help information

#> molaical.exe -call run -c mmpbgbsa1 -i -dispdev text -args -help

2) Calculating MM/GBSA ("-pb 0" and "-gb 1")

#> molaical.exe -call run -c mmpbgbsa1 -i -dispdev text -args -s complex.psf -c complex.pdb -cs "segname,A,C" -rs "segname,A" -ls "segname,C" -pb 0 -gb 1 -ff toppar_water_ions.str -ff ligand.str

3) Calculating MM/PBSA by assigning the paths of the external program. The default value of "-pb" is 2, which calls the APBS program automatically.

#> molaical.exe -call run -c mmpbgbsa1 -i -dispdev text -args -s complex.psf -c complex.pdb -rn E:\workdir\MolAICal\method\mmgbsa_v1\mmpbsa\namd2\namd2.exe -pbe E:/workdir/MolCalSys/ideamac/out/production/MolAICal1.40/mtools/etools/apbs/bin/apbs -cs "segname,A,C" -rs "segname,A" -ls "segname,C" -ff par_all36_prot.prm -ff par_all36_cgenff.prm -ff toppar_water_ions.str -ff ligand.str

4) Calculating MM/PBSA more briefly.

#> molaical.exe -call run -c mmpbgbsa1 -i -dispdev text -args -s complex.psf -c complex.pdb -cs "segname A C" -rs "segname A" -ls "segname C" -ff toppar_water_ions.str -ff ligand.str

5) Calculating MM/PBSA by calling the DelPhi program if "-pb" is 1.

#> molaical.exe -call run -c mmpbgbsa1 -i -dispdev text -args -s complex.psf -c complex.pdb -cs "segname,A,C" -rs "segname,A" -ls "segname,C" -pb 1 -ff toppar_water_ions.str -ff ligand.str

Note: Here, MolAICal supports recognizing commas "," in strings like "segname,A,C" as equivalent to spaces " ", meaning "segname,A,C" is treated identically to "segname A C". This applies to similar characters by analogy. This input method helps prevent command-line parsing errors caused by spaces.

◆ Calculating MM/PB/GBSA based on the NAMD-generated DCD, PDB and PSF files

1) Print help information

#> molaical.exe -call run -c mmpbgbsa -i -dispdev text -args -help

2) Calculating MM/GBSA ("-pb 0" and "-gb 1")

#> molaical.exe -call run -c mmpbgbsa -i -dispdev text -args -s complex.psf -c complex.pdb -t complex.dcd -cs "segname,A,C" -rs "segname,A" -ls "segname,C" -pb 0 -gb 1 -ff toppar_water_ions.str -ff ligand.str

3) Calculating MM/PBSA by assigning the paths of the external program. The default value of "-pb" is 2, which calls the APBS program automatically.

#> molaical.exe -call run -c mmpbgbsa -i -dispdev text -args -s complex.psf -c complex.pdb -t complex.dcd -rn E:\workdir\MolAICal\method\mmgbsa_v1\mmpbsa\namd2\namd2.exe -pbe E:/workdir/MolCalSys/ideamac/out/production/MolAICal1.40/mtools/etools/apbs/bin/apbs -cs "segname,A,C" -rs "segname,A" -ls "segname,C" -ff par_all36_prot.prm -ff par_all36_cgenff.prm -ff toppar_water_ions.str -ff ligand.str

4) Calculating MM/PBSA more briefly.

#> molaical.exe -call run -c mmpbgbsa -i -dispdev text -args -s complex.psf -c complex.pdb -t complex.dcd -cs "segname,A,C" -rs "segname,A" -ls "segname,C" -ff toppar_water_ions.str -ff ligand.str

5) Calculating MM/PBSA by calling the DelPhi program if "-pb" is 1.

#> molaical.exe -call run -c mmpbgbsa -i -dispdev text -args -s complex.psf -c complex.pdb -t complex.dcd -cs "segname,A,C" -rs "segname,A" -ls "segname,C" -pb 1 -ff toppar_water_ions.str -ff ligand.str

Note: Here, MolAICal supports recognizing commas "," in strings like "segname,A,C" as equivalent to spaces " ", meaning "segname,A,C" is treated identically to "segname A C". This applies to similar characters by analogy. This input method helps prevent command-line parsing errors caused by spaces.

3.4. Entropy calculation

3.4.1. Fit molecules and entropy calculation by Carma and MolAICal

To calculate the entropy of protein-protein, protein-ligand, etc, the Carma[21] software is introduced to calculate the entropy contribution between two molecules based on the NAMD trajectories. Carma computes the entropy using both Schlitter's and Andricioaei's formulas[22] starting from DCD and PSF files. A more detailed introduction about Carma can be found on the official website of Carma: https://utopia.duth.gr/~glykos/Carma.html or https://github.com/glykos/carma. Users should download the new version of Carma (For example: “carma64”) for calculating the entropy from http://utopia.duth.gr/~glykos/progs or https://utopia.duth.gr/~glykos.

According to the LICENSE of Carma (https://github.com/glykos/carma/blob/master/LICENSE), which is without the limitation rights to use, copy, modify, merge, publish, distribute, etc. Hence, MolAICal can directly integrate and invoke Carma to fit the molecules and calculate the entropy of molecules.

The parameters are as follows:

-call:	when its value is "run", it will call the external program.
-c:	the calculation way, it will be used to calculate entropy by Carma if its value is "entropy".
-i:	after "-i", follow with Carma-related commands. The command-line argument "-i" **must** be placed **after** other arguments (e.g., after argument "-c").
-o <arg>	Whether to print the output of the program.

-h	Print help information.
-v:	verbose information
-fit:	Remove global rotations-translations from DCD frames
-force:	tells Carma to stop the calculation of entropies before they become undefined.
-atmid:	atom-name-based atom selection
-segid:	segment-identifier-based atom selection (dcd-psf only)

Example command line as below:

1) Print the help information of Carma
#> molaical.exe -call run -c entropy -i -h

2) Print the help information of entropy calculation in MolAICal
#> molaical.exe -call run -h

3) This command is used for removing rotations and translations.
#> molaical.exe -call run -c entropy -i -v -fit -force -atmid ALLID -segid A -segid C complex.dcd complex.psf

4) This command is used for calculating entropy.
#> molaical.exe -call run -c entropy -i -v -cov -eigen -mass -force -temp 310 -atmid ALLID -segid A -segid C -last 25 carma.fitted.dcd complex.psf

3.4.2. Calculate temperature multiplied by delta entropy via MolAICal

Here, MolAICal supplies a way to calculate the temperature multiplied by the delta entropy (T*ΔS) between two molecules based on the results of Carma.

The parameters for the calculation of temperature multiplied by delta entropy are as follows:

-entropy: means the delta entropy calculation.
-c: the value or file of complex entropy.
-r: the value or file of the receptor or first molecule entropy.
-l: the value or file of the ligand or second molecule entropy.
-t: the Kelvin temperature in MD simulation.
if: it will read entropy values from result files, if the input arguments contain "-if".

Example command line as below:

1) Calculating the delta entropy from the values (no "-if")

#> molaical.exe -entropy -c 15522.260938 -r 15211.880284 -l 1454.409253 -t 310.0

2) Calculating the delta entropy from the results files of Carma
#> molaical.exe -entropy -if -c com.log -r rec.log -l lig.log -t 310.0

4. Useful tools for drug design

4.1. Channel radii measurement

The Metropolis Monte Carlo simulated annealing method[23] is used to measure the channel radii of objects such as nanotubes, ion channel proteins, etc. The initial can be defined anywhere in the channel, and the exploration vector is approximately along the direction of the channel. The calculation of channel radii is realized in MolAICal. The examples of configuration files for radii calculation are shown in Appendices 3 and 4. The parameters for channel radii calculation are listed below:

- **pdbpath** < The path of object with pdb format >

Acceptable Values: String
Default Value: null
Description: It should point out the path of the object for radii measurement.

- **psfpath** < The path of object with PSF format >

Acceptable Values: String
Default Value: null
Description: The path of the object with PSF format is not necessary if you only perform the simple radii measurement in the channel of the object. If you want to measure the radii of an object by using the CHARMM force field, you should give the path of the PSF file that corresponds to the PDB file.

- **cpoint** < The initial point for radii measurement >

Acceptable Values: decimal
Default Value: null
Description: This is the initial point for radii measurement. The values following the cpoint are x, y, z coordinates.

- **Vector** < The measurement direction >

Acceptable Values: decimal
Default Value: null
Description: This parameter has three values that represent the x, y, and z directions. For example, “1.00 0.00 0.00” represents the measure channel radii along the x-axis. “0.00 1.00 0.00” represents

the measure channel radii along the y-axis. “0.00 0.00 1.00” represents the measure channel radii along the z-axis.

- **sample** < The interval of channel radii measurement along the appointed direction >

Acceptable Values: decimal
Default Value: null
Description: It is the interval of channel radii measurement along the appointed direction. It can be set according to the specific measurement requirement.

- **minprobe** < The minimum explored probe >

Acceptable Values: decimal
Default Value: Value calculated from the minimum radius
Description: It is the minimum explored detector. The MolAICal program will calculate radii of protein, nanotube, and so on by the minimum explored detector.

- **conpar** < control constant >

Acceptable Values: decimal
Default Value: 0.15
Description: It is a control constant. The detail parameter is shown in equation 4 in this paper[24]. Usually, it uses the default value of 0.15.

- **endrad** < cutoff for radii measurement termination >

Acceptable Values: decimal
Default Value: null
Description: It is the cutoff value for radii measurement termination. If the measured radii are larger than the setting cutoff, the program will end.

- **selatoms** < select atoms within the range of object >

Acceptable Values: decimal
Default Value: null
Description: For a large system, it needs to calculate a lot of unnecessary atoms. In this case, the selected atoms can save computing time. This value should be larger than the cutoff value of endrad parameter.

- **numpt** < number of points in the surface >

Acceptable Values: integer
Default Value: null
Description: The final output contains grids in a file, which can show the grid surface of the channel in the VMD software[25]. This parameter can determine the number of surface points that can be shown in the VMD software.

- **surColor** < surface color >

Acceptable Values: string
Default Value: null

Description: It is the surface color. Because the color is shown in the VMD software, the value of surColor can be chosen from the color values of the VMD software.

- **lineColor** < color of radii measurement routing >

Acceptable Values: string
Default Value: null
Description: The channel radii are measured along the appointed direction. The whole routing shows a curve in the channel. This parameter sets the color of the curve. The value of lineColor can be chosen from the color values of the VMD software.

- **lineWidth** < width of radii measurement routing >

Acceptable Values: integer
Default Value: null
Description: It represents the width of the curve. The "lineColor" is responsible for the color of measured routing, while the "lineWidth" is responsible for the width of measured routing.

- **searchNum** < measured number of radii in one plane >

Acceptable Values: integer
Default Value: null
Description: It is a trial number for the radii measurement of the channel in one plane.

- **material** < material property of sphere surface >

Acceptable Values: integer
Default Value: null
Description: It represents the material property of the sphere surface, which is shown in the VMD software. It can be chosen from the material values of the VMD software.

The command parameters of MolAICal for channel radii measurement are as below:

-channel: means the radii measurement function.
-cpp: the path of the input configuration file, which contains the parameters for radii calculations
-fc: means the calculated way. It contains two ways. One way is simple radii calculation, the other is radii measurement with the CHARMM force field, which needs the PSF file.

Here, the example commands for channel radii measurement are shown below:

1) Command for simple radii calculation.
#> molaical.exe -channel radii -cpp D:/GramicidinA/parameter.dat -fc simple

2) Command for radii calculation with the CHARMM force field.
#> molaical.exe -channel radii -cpp D:/GramicidinA/parameter.dat -fc charmm

3) Command with default parameters. The default calculated way is the simple radii calculation.

#> molaical.exe -channel radii -cpp D:/GramicidinA/parameter.dat

4.2. Energy Calculation

4.2.1. Potential of mean force

The potential of mean force (PMF) can be used to calculate the free energy changes as the function of a coordinate of the system. The PMF along the coordinate is computed from the average distribution function (see the equation below).

$\Delta G = -k_B T \ln \rho(x, y)$

Where T and k_B are the temperature and Boltzmann constant, respectively. The x and y represent two principal components. The unit of ΔG is **kcal/mol** in the MolAICal program. The PMF can profile the free energy landscape.

The command parameters of MolAICal for PMF calculation are as below:

-pmf: means PMF calculation function.
-i: the path of the input data file.
-g: number of grids for density calculation in a square.
-l: number of color levels.
-m: choose contours. Currently, it contains “conshd” and “contur”.
-b: is LABELS. It determines which label types will be plotted on an axis. It contains “**none**” and “float”. The default value is "**float**".
- ✧ "**none**" means no numeric labels are marked on the contour lines.
- ✧ "**float**" represents that numeric labels are marked on the contour lines.

-x: is x-axis label.
-y: is y-axis label.
-t: is temperature.
-f: is the output figure file; the default figure name is "pmfFig.png".

The example commands for PMF are shown below:

1) Do PMF calculation by using the default parameter
#> molaical.exe -pmf -i E:/pmf/rmsd-dis.dat

2) Do PMF calculation with “conshd” contour without numerical label
#> molaical.exe -pmf -i rmsd-dis.dat -g 20 -l 10 -m conshd -b none -x "RMSD (Å)" -y "Distance (Å)"

3) Do PMF calculation with “conshd” contour with numerical label
#> molaical.exe -pmf -i rmsd-dis.dat -g 20 -l 10 -m conshd -b float -x "RMSD (Å)" -y "Distance (Å)"

4) Do PMF calculation with “contur” contour without numerical label
#> molaical.exe -pmf -i rmsd-dis.dat -g 20 -l 10 -m contur -b none -x "RMSD (Å)" -y "Distance (Å)"

5) Do PMF calculation with “contur” contour with numerical label
#> molaical.exe -pmf -i rmsd-dis.dat -g 20 -l 10 -m contur -b float -x "RMSD (Å)" -y "Distance (Å)"

6) Do PMF calculation with “contur” contour with numerical label and assign output figure name
#> molaical.exe -pmf -i rmsd-dis.dat -g 20 -l 10 -m contur -b float -x "RMSD (Å)" -y "Distance (Å)"

Note: The output image will be saved in the **format specified by its file extension** (e.g., .tif, .png, etc.). If the "-f" option is not provided, the default output will be a file named "pmfFig.png".

4.2.2. Energy Minimization

Energy Minimization is a computational method in chemistry and molecular modeling. Its primary goal is to find the lowest energy conformation of a molecular system by reducing internal energies (such as bond stretching, angle bending, and non-bonded interactions). This process helps identify the most stable structure of a molecule, which is essential for further analyses like molecular docking, property prediction, and structure-activity relationship studies.

4.2.2.1. Energy Minimization by MolAICal

MMFF94, MMFF94s, and UFF are used here for molecular energy minimization based on BFGS (Broyden-Fletcher-Goldfarb-Shanno) algorithm (For the detailed BFGS information, please check: https://en.wikipedia.org/wiki/Broyden%E2%80%93Fletcher%E2%80%93Goldfarb%E2%80%93Shanno_algorithm). Other related introductions are as follows:

MMFF94, also known as **Merck Molecular Force Field 94**, is a computational method used to simulate the structures and energies of organic molecules, especially those pertinent to pharmaceutical applications. It enables molecular geometry optimization and energy calculations, which are valuable for drug development and associated scientific studies[26].

Parameterization and Scope: MMFF94 is best suited for organic molecules containing common elements like carbon, hydrogen, nitrogen, oxygen, sulfur, phosphorus, and halogens, as well as some ions. It's mainly used for preparing molecular structures, geometry optimization, energy calculations, generating low-energy shapes, and evaluating energy feasibility, with options to customize its application for specific needs. MMFF94 is suitable for modeling organic molecules with common elements, primarily in drug discovery and computational chemistry.

MMFF94s is a variant of the Merck Molecular Force Field (MMFF94), designed for accurate geometry optimization of organic and drug-like molecules. It's particularly good for systems needing planarity, like aromatic rings. According to research, MMFF94s is specifically parameterized for geometry optimization studies, differing from MMFF94 in its torsional and out-of-plane bending parameters to better planarize delocalized systems, such as trigonal nitrogen atoms in aromatic rings (https://en.wikipedia.org/wiki/Merck_molecular_force_field).

Parameterization and Scope: MMFF94s is optimized for **organic and drug-like molecules**, covering elements such as carbon (C), hydrogen (H), nitrogen (N), oxygen (O), fluorine (F), silicon (Si), phosphorus (P), sulfur (S), chlorine (Cl), bromine (Br), iodine (I), and common ions like Fe+2, Fe+3, F-, Cl-, Br-, Li+, Na+, K+, Zn+2, Ca+2, Cu+1, Cu+2, and Mg+2 [Open Babel Documentation, https://open-babel.readthedocs.io/en/latest/Forcefields/mmff94.html]. Its core parameters were derived from high-quality quantum calculations across approximately 500 test systems, ensuring accuracy for bond lengths, angles, and electrostatic interactions.

UFF, or Universal Force Field, is a general force field covering the entire periodic table, making it versatile for a wide range of chemical systems, including metals and inorganics, but it may trade some accuracy for speed and coverage.

Parameterization and Scope: UFF is capable of handling all elements, including transition metals (e.g., Zn, Cu, Ni, Co, Fe, Mn, Cr, V, Ti, Sc, Al) and is particularly noted for its application in inorganic and organometallic materials (see: https://avogadro.cc/docs/optimizing-geometry/molecular-mechanics). It trades accuracy for speed and wide chemistry coverage, making it suitable for systems where MMFF94s parameters are unavailable.

Generally, **use MMFF94s** for organic molecules (e.g., those with C, H, N, O, F, Si, P, S, Cl, Br, I, and some ions like Fe+2) when accuracy in geometry optimization is crucial. **Use UFF** for molecules with elements outside MMFF94s' scope, like metals, or as a fallback when MMFF94s parameters aren't available. It's also faster, which might be helpful for quick calculations. So **MMFF94s** and **UFF** are very **suitable** for energy minimization of **organic and drug-like molecules.**

MolAICal can minimize the single molecule and two molecules. **For minimization of the single molecule, the command parameters are as below:**

usage: -min1, minimization for the single molecule

-min1	Minimization for the single molecule
-f <arg>	Either fixed atoms (e.g., '1-3,5,7-9') or reference molecule file.
-h	Print help information.
-i <arg>	Input molecule file (.pdb, .mol2, .sdf, .mol).
-o <arg>	Output file for minimized molecule (.pdb, .mol2, .sdf, .mol).
-q <arg>	The iteration number of Quick optimization.

-r <arg> The iteration number of Refinement.

The example commands for minimization of the single molecule are shown below:

1) Print help information for energy minimization of the single molecule
#> molaical.exe -min1 -h

2) Print the information of input molecule without minimization
#> molaical.exe -min1 -i lig_10.mol2

3) Minimize a single molecule without the fixed atoms
#> molaical.exe -min1 -i lig_10.mol2 -o min_lig.mol2

4) Minimize a single molecule with the serial number of fixed atoms
#> molaical.exe -min1 -i lig_10.mol2 -o min_lig.mol2 -f 1-2

5) Minimize a single molecule with the serial number of fixed atoms, iteration numbers of quick optimization and refinement
#> molaical.exe -min1 -i lig_10.mol2 -o min_lig.mol2 -f 1-2 -q 10 -r 6

6) Minimize a single molecule with the serial number of fixed atoms, and iteration number of quick optimization
#> molaical.exe -min1 -i lig_10.mol2 -o min_lig.mol2 -f 1-2 -q 10

7) Minimize a single molecule with the serial number of fixed atoms, and iteration number of refinement
#> molaical.exe -min1 -i lig_10.mol2 -o min_lig.mol2 -f 1-2 -r 6

8) Minimize a single molecule with the reference molecule file, iteration numbers of quick optimization and refinement
#> molaical.exe -min1 -i lig_10.mol2 -o min_lig.mol2 -f lig_fix.mol2 -q 10 -r 6

9) Minimize a single molecule with the reference molecule file
#> molaical.exe -min1 -i lig_10.mol2 -o min_lig.mol2 -f lig_fix.mol2

For minimization of the two molecules, the command parameters are as below:

usage: -min2, minimization for the two molecules

-min2	Minimization for the two molecules
-f1 <arg>	Either fixed atoms (e.g., '1-3,5,7-9') or reference molecule file for first molecule.
-f2 <arg>	Either fixed atoms (e.g., '1-3,5,7-9') or reference molecule file for second molecule.
-h	Print help information.
-i1 <arg>	Input molecule file for first molecule (.pdb, .mol2, .sdf, .mol).
-i2 <arg>	Input molecule file for second molecule (.pdb, .mol2, .sdf, .mol).

-m <arg>	Operation mode for minimization (default: null), value can be **info, 1, or 2**: "**info**" means to print molecules information only; "**1**" means to minimize one molecule only; "**2**" means to minimize two molecules.
-o1 <arg>	Output file for minimized molecule for first molecule (.pdb, .mol2, .sdf, .mol).
-o2 <arg>	Output file for minimized molecule for second molecule (.pdb, .mol2, .sdf, .mol).
-q <arg>	The iteration number of Quick optimization.
-r <arg>	The iteration number of Refinement.
-c2	Convert the second input PDB file in mol2 format if "-c" in the command line, it is designed for molecular docking of MolAICal.

The example commands for the minimization of two molecules are shown below:

1) Print help information for energy minimization of two molecules
#> molaical.exe -min2 -h

Info-only mode (no minimization):
2) Print the information of two input molecules without minimization
#> molaical.exe -min2 -m info -i1 GCGRH.pdb -i2 lig_10.pdb

Output only the result of the second input molecule:
3) Minimize two input molecules
#> molaical.exe -min2 -m 1 -i1 GCGRH.pdb -i2 lig_10.pdb -o2 min_lig.pdb

4) Convert the second input PDB molecule in Mol2 and minimize two input molecules
#> molaical.exe -min2 -m 1 -i1 GCGRH.pdb -i2 lig_10.pdb -o2 min_lig.pdb -c2

5) Minimize two molecules with assigned iteration numbers of quick optimization and refinement
#> molaical.exe -min2 -m 1 -i1 GCGRH.pdb -i2 lig_10.pdb -o2 min_lig.pdb -q 10 -r 6

6) Minimize two molecules with the serial number of fixed atoms, and the assigned iteration numbers of quick optimization and refinement
#> molaical.exe -min2 -m 1 -i1 GCGRH.pdb -i2 lig_10.pdb -o2 min_lig.pdb -f1 1-6807,6843-6849,6853 -q 10 -r 6

7) Convert the second input PDB molecule into Mol2 and minimize two molecules with the serial number of fixed atoms, and the assigned iteration numbers of quick optimization and refinement
#> molaical.exe -min2 -m 1 -i1 GCGRH.pdb -i2 lig_10.pdb -o2 min_lig.pdb -f1 1-6807,6843-6849,6853 -q 10 -r 6 -c2

8) Minimize two molecules with the reference molecule files, and the assigned iteration numbers of quick optimization and refinement
#> molaical.exe -min2 -m 1 -i1 GCGRH.pdb -i2 lig_10.mol2 -o2 min_lig.mol2 -f1 GCGRH_fix.pdb -f2 lig_fix.mol2 -q 10 -r 6

9) Not convert the second input SDF molecule into Mol2 and minimize two molecules with the reference molecule files, and the assigned iteration numbers of quick optimization and refinement
#> -min2 -m 1 -i1 GCGRH.pdb -i2 lig_10.sdf -o2 min_lig.pdb -f1 GCGRH_fix.pdb -f2 lig_fix.sdf -q 10 -r 6 -c2

10) Convert the second input PDB molecule into Mol2 and minimize two molecules with the reference molecule files, and the assigned iteration numbers of quick optimization and refinement
#> molaical.exe -min2 -m 1 -i1 GCGRH.pdb -i2 lig_10.pdb -o2 min_lig.pdb -f1 GCGRH_fix.pdb -f2 lig_fix.mol2 -q 10 -r 6 -c2

11) Minimize two molecules with the reference molecule files, and the assigned iteration numbers of quick optimization and refinement
#> molaical.exe -min2 -m 1 -i1 GCGRH.pdb -i2 lig_10.pdb -o2 min_lig.pdb -f1 GCGRH_fix.pdb -f2 lig_fix.mol2 -q 10 -r 6

12) Minimize two molecules without fixing the second input molecule, and the assigned iteration numbers of quick optimization and refinement
#> molaical.exe -min2 -m 1 -i1 GCGRH.pdb -i2 lig_10.pdb -o2 min_lig.pdb -f1 GCGRH_fix.pdb -f2 0 -q 10 -r 6

13) Convert the second input PDB molecule in Mol2 and minimize two molecules without fixing the second input molecule, and the assigned iteration numbers of quick optimization and refinement
#> molaical.exe -min2 -m 1 -i1 GCGRH.pdb -i2 lig_10.pdb -o2 min_lig.pdb -f1 GCGRH_fix.pdb -f2 0 -q 10 -r 6 -c2

Output results for both input molecules:

14) Minimize two input molecules
#> molaical.exe -min2 -m 2 -i1 GCGRH.pdb -i2 lig_10.mol2 -o1 min_gcgr.pdb -o2 min_lig.mol2

15) Minimize two input molecules
#> molaical.exe -min2 -m 2 -i1 GCGRH.pdb -i2 lig_10.pdb -o1 min_gcgr.mol2 -o2 min_lig.mol2

16) Convert the second input PDB molecule in Mol2 and minimize two input molecules
#> molaical.exe -min2 -m 2 -i1 GCGRH.pdb -i2 lig_10.pdb -o1 min_gcgr.mol2 -o2 min_lig.mol2 -c2

17) Minimize two input molecules with the assigned iteration numbers of quick optimization and refinement
#> molaical.exe -min2 -m 2 -i1 GCGRH.pdb -i2 lig_10.mol2 -o1 min_gcgr.pdb -o2 min_lig.mol2 -q 3 -r 4

18) Convert the second input PDB molecule in Mol2 and minimize the two input molecules with the assigned iteration numbers of quick optimization and refinement

#> molaical.exe -min2 -m 2 -i1 GCGRH.pdb -i2 lig_10.pdb -o1 min_gcgr.pdb -o2 min_lig.mol2 -q 3 -r 4 -c2

19) Minimize two input molecules with the serial number of fixed atoms, and the assigned iteration numbers of quick optimization and refinement
#> molaical.exe -min2 -m 2 -i1 GCGRH.pdb -i2 lig_10.pdb -o1 min_gcgr.pdb -o2 min_lig.pdb -f1 1-6807,6843-6849,6853 -q 3 -r 4

20) Convert the second input PDB molecule in Mol2 and minimize the two input molecules with the serial number of fixed atoms, and the assigned iteration numbers of quick optimization and refinement
#> molaical.exe -min2 -m 2 -i1 GCGRH.pdb -i2 lig_10.pdb -o1 min_gcgr.pdb -o2 min_lig.pdb -f1 1-6807,6843-6849,6853 -q 3 -r 4 -c2

21) Minimize the two input molecules with the reference molecule files
#> molaical.exe -min2 -m 2 -i1 GCGRH.pdb -i2 lig_10.mol2 -o1 min_gcgr.pdb -o2 min_lig.mol2 -f1 GCGRH_fix.pdb -f2 lig_fix.mol2

22) Convert the second input PDB molecule in Mol2 and minimize the two input molecules with the reference molecule files
#> molaical.exe -min2 -m 2 -i1 GCGRH.pdb -i2 lig_10.pdb -o1 min_gcgr.pdb -o2 min_lig.mol2 -f1 GCGRH_fix.pdb -f2 lig_fix.mol2 -c2

23) Minimize the two input molecules with the reference molecule files, and the assigned iteration numbers of quick optimization and refinement
#> molaical.exe -min2 -m 2 -i1 GCGRH.pdb -i2 lig_10.mol2 -o1 min_gcgr.pdb -o2 min_lig.mol2 -f1 GCGRH_fix.pdb -f2 lig_fix.mol2 -q 3 -r 4

24) Convert the second input PDB molecule in Mol2 and minimize the two input molecules with the reference molecule files, and the assigned iteration numbers of quick optimization and refinement
#> molaical.exe -min2 -m 2 -i1 GCGRH.pdb -i2 lig_10.pdb -o1 min_gcgr.pdb -o2 min_lig.mol2 -f1 GCGRH_fix.pdb -f2 lig_fix.mol2 -q 3 -r 4 -c2

25) Minimize the two input molecules with the reference molecule files, 0 means no fixed atoms
#> molaical.exe -min2 -m 2 -i1 GCGRH.pdb -i2 lig_10.pdb -o1 min_gcgr.pdb -o2 min_lig.pdb -f1 GCGRH_fix.pdb -f2 0

26) Convert the second input PDB molecule in Mol2 and minimize the two input molecules with the reference molecule files, 0 means no fixed atoms
#> molaical.exe -min2 -m 2 -i1 GCGRH.pdb -i2 lig_10.pdb -o1 min_gcgr.pdb -o2 min_lig.pdb -f1 GCGRH_fix.pdb -f2 0 -c2

4.2.2.2. Energy Minimization by MolAICal and UCSF Chimera

Because UCSF Chimera (https://www.cgl.ucsf.edu/chimera) can directly invoke some very useful force fields for energy minimization of different kinds of molecules, MolAICal also provides an interface to call UCSF Chimera for energy minimization of molecules. UCSF Chimera primarily uses the Amber force field for energy minimization, with Amber ff99 for standard biological residues and GAFF for nonstandard residues, alongside Amber parameters for monatomic ions. Amber ff99 is suitable for proteins, DNA, and RNA, while GAFF handles small organic molecules like drugs, with some variation in older versions using Amber 1994 (ff94). Here's the detailed information about the force fields in UCSF Chimera and their roles:

- ✧ **Amber ff99:** This force field is used for standard residues, such as proteins (standard amino acids), DNA, RNA, water, and common variants. It's designed for biomolecular simulations, providing accurate descriptions of molecular interactions in biological systems.

- ✧ **GAFF** (General Amber Force Field): Applied to nonstandard residues, like small organic molecules or ligands, GAFF is used for drug discovery and modeling complexes involving biological macromolecules and small molecules. It's parameterized via Amber's Antechamber module.

- ✧ **Amber Parameters for Ions:** For monatomic ions (e.g., Na+, K+, Cl–, Mg2+), Chimera uses Amber parameters, including user-specified net charges and van der Waals parameters, to accurately represent ionic interactions.

The path of UCSF Chimera should be assigned when using energy minimization by MolAICal and UCSF Chimera. UCSF Chimera uses **steepest descent** and **conjugate gradient** methods for energy minimization, provided by the Molecular Modelling Toolkit (MMTK):

- ✧ **Steepest Descent:** This method iteratively moves atomic coordinates in the direction opposite to the energy gradient, helping to quickly reduce high-energy configurations like steric clashes. It's simple but can be slow for finding precise minima. Steepest Descent Update Rule:
  $r_{k+1}=r_k-\alpha_k\nabla E(r_k)$
  Here, r_k represents the atomic coordinates at iteration (k), $\nabla E(r_k)$ is the gradient (negative of the force), and α_k is the step size, which can be fixed (e.g., 0.02 Å) or adaptive based on line search.
- ✧ **Conjugate Gradient:** Following steepest descent, this method uses conjugate directions to improve convergence, making it more effective for reaching local energy minima after initial adjustments. Conjugate Gradient Update Rules:
  - ➢ Coordinate update: $r_{k+1}=r_k+\alpha_k d_k$
  - ➢ Search direction update: $d_{k+1}=-\nabla E(r_{k+1})+\beta_k d_k$,

  Here, β_k ensures conjugacy. α_k is found via line search.

usage: **-cmin** [-c <arg>] [-cm <arg>] [-cmin] [-cs <arg>] [-css <arg>]
[-f1 <arg>] [-f2 <arg>] [-fs <arg>] [-fv <arg>] [-h] [-i1 <arg>]

[-i2 <arg>] [-iv <arg>] [-m <arg>] [-mf1 <arg>] [-mf2 <arg>] [-ng <arg>] [-ns <arg>] [-o1 <arg>] [-o2 <arg>] [-os <arg>] [-p <arg>] [-pv <arg>] [-ss <arg>] [-sv <arg>] [-w <arg>]

Molecular Minimization with UCSF Chimera and MolAICal.

-c,--chimera_path <arg> Path to Chimera executable

> **Notice:** MolAICal also provides a method to set the Chimera path, for example:
>
> `#> molaical.exe -call set -n chimera -p "d:\Program Files\Chimera119\bin\chimera.exe"`
>
> The value of '-n' **must** be **'chimera'** (case-insensitive). Currently, **'VMD', 'NAMD'**, and **'chimera'** are fixed names for internal path specifications in **MolAICal**. Therefore, they must be written exactly as shown, though letter case does not matter. '-p' is the absolute path of the executable program of UCSF Chimera.
>
> **Once set**, this path will be saved and used automatically, **eliminating the need to input chimera_path every time**, thus reducing repetitive input. **Thus, it is not necessary to input '-c' or '--chimera_path' when performing minimization.**

-cm,--charge_method <arg>	Charge method (am1 or gas), default: gas
-cmin	minimization calculation from the interface for UCSF Chimera.
-cs,--cgsteps_value <arg>	Conjugate Gradient (CG) steps, default: 2
-css,--cgstepsize_value <arg>	Conjugate Gradient (CG) step size, default: 0.02
-f1,--fixed_f1 <arg>	Path to first fixed molecule file (for modes 3 and 4)
-f2,--fixed_f2 <arg>	Path to second fixed molecule file (for mode 4)
-fs,--freeze_spec <arg>	Freeze or spec method, default: freeze
-fv,--fragment_value <arg>	Whether to use fragment (true/false), default: false
-h,--help	Display help information
-i1,--input_f1 <arg>	Path to first input molecule file or command file (for mode 0)
-i2,--input_f2 <arg>	Path to second input molecule file or output log (for mode 0)
-iv,--interval_value <arg>	Interval, default: 1
-m,--mode <arg>	Mode of operation (0, 1, 2, 3, or 4): **Mode 0**: Direct call to chimera_command (--input_f1 as commandFile, --input_f2 as outputLog); **Mode 1:** Single molecule minimization; **Mode 2:** Two-molecule minimization; **Mode 3:** Three-molecule minimization with one fixed; **Mode 4:** Four-molecule minimization with two fixed.
-mf1,--mol_format_1 <arg>	First output format, default: mol2
-mf2,--mol_format_2 <arg>	Second output format, default: pdb
-ng,--nogui_value <arg>	Whether to use GUI (true/false), default: true
-ns,--nsteps_value <arg>	Steepest descent steps, default: 10
-o1,--output_f1 <arg>	First output file
-o2,--output_f2 <arg>	Second output file
-os,--output_sel <arg>	Output selection
-p,--print_output <arg>	Whether to print output (true/false), false means to output the log

	file, vice versa, default: true
-pv,--prep_value <arg>	Whether to use prep (true/false), default: true
-ss,--stepsize_value <arg>	Steepest descent step size, default: 0.02
-sv,--sel_value <arg>	Selection value for atoms
-w,--whether_output <arg>	Whether to output the second molecule (true/false), false means no second output, vice versa, default: true

Note: the values of "--sel_value" and "--output_sel" are the "atom specification" of UCSF Chimera (see https://www.rbvi.ucsf.edu/chimera/current/docs/UsersGuide/midas/frameatom_spec.html).
The aforementioned usage options, such as "--cgsteps_value, --cgstepsize_value, --interval_value, --fragment_value, --interval_value, --stepsize_value, --nsteps_value, --prep_value, --freeze_spec", derive from the "minimize" function of UCSF Chimera. If the initial letters of the options in MolAICal match those in the "minimize" function of UCSF Chimera, these options in MolAICal will bear the same interpretation and meaning as in the "minimize" function of UCSF Chimera (see: https://www.rbvi.ucsf.edu/chimera/current/docs/UsersGuide/midas/minimize.html). Please check the above URLs if needed to learn how to write atom specifications and options of the minimize function of UCSF Chimera.

The example commands for the minimization of molecules are shown below:

Mode 0: It can run the script file of UCSF Chimera directly
1) Run the command script of UCSF Chimera by MolAICal
#> molaical.exe -cmin -c "d:\Chimera\bin\chimera.exe" -m 0 -i1 run.com

2) Run the command script of UCSF Chimera by MolAICal with log file
#> molaical.exe -cmin -c "d:\Chimera\bin\chimera" -m 0 -i1 run.com -i2 run2.log -p false

3) Run the command script of UCSF Chimera by MolAICal with log file
#> molaical.exe -cmin -c "d:\Chimera\bin\chimera.exe" -m 0 -i1 run.com -i2 -p false

Mode 1: minimization for one molecule

> For Modes 1 and 2, the value following -sv can be: "none", "selected", "unselected", or atom specifications (atom-spec) using UCSF Chimera's atom selection syntax.

4) Minimize one molecule with the assigned atom serials
#> molaical.exe -cmin -c "d:\Chimera\bin\chimera.exe" -m 1 -i1 lig_10.mol2 -sv "#0:@/serialNumber>=20" -o1 out_1.mol2

5) Minimize one molecule with the assigned atom serials and with log file
#> molaical.exe -cmin -c "d:\Chimera\bin\chimera.exe" -m 1 -i1 lig_10.mol2 -sv "#0:@/serialNumber>=20" -o1 out_1.mol2 -p false

Note: "#0:@/serialNumber>=20" is the atom serial number in the input file, which is from the atom

specifications of UCSF Chimera. For more detailed information, please see the minimize function of UCSF Chimera as mentioned above. Here, "-sv" is used to assign the fixed atoms to perform minimization.

Mode 2: minimization for two molecules

For Modes 1 and 2, the value following -sv can be: "none", "selected", "unselected", or atom specifications (atom-spec) using UCSF Chimera's atom selection syntax.

6) Minimize two molecules with the assigned atom specifications and output two molecules
#> molaical.exe -cmin -c "d:\Chimera\bin\chimera.exe" -m 2 -i1 lig_10.mol2 -i2 GCGRH.pdb -sv "#2:<0>" -os "#2:<0>" -o1 out_1.mol2 -o2 out_2.pdb

Note:

- ✧ The option "-i1 lig_10.mol2" corresponds to '**#0**';
- ✧ The option "-i2 GCGRH.pdb" corresponds to '**#1**';
- ✧ The option -sv "#2:<0>" specifies selecting and fixing the residue named "<0>" in the merged complex (#2) for subsequent energy minimization. Here, #2 refers to the complex formed by merging GCGRH.pdb (#1) and lig_10.mol2 (#0).
- ✧ The option -os "#2:<0>" is used to select one molecule (<0>) from the complex (#2) for saving after energy minimization, while the other molecule is saved by excluding the selected one. This is because energy minimization is performed on the merged complex (#2) of the first molecule (#0) and the second molecule (#1).
- ✧ For Modes 1 and 2, the value following -sv can be: "none", "selected", "unselected", or an atom specification (atom-spec) using UCSF Chimera's atom selection syntax.

7) Minimize two molecules with the assigned atom specifications and output one molecule
#> molaical.exe -cmin -c "d:\Chimera\bin\chimera.exe" -m 2 -i1 lig_10.mol2 -i2 GCGRH.pdb -sv "#1" -os "#2:<0>" -o1 out_1.mol2 -o2 out_2.pdb -w false

8) Minimize two molecules with the assigned atom specifications, output one molecule and log file
#> molaical.exe -cmin -c "d:\Chimera\bin\chimera.exe" -m 2 -i1 lig_10.mol2 -i2 GCGRH.pdb -sv "#1" -os "#2:<0>" -o1 out_1.mol2 -o2 out_2.pdb -w false -p false

9) Minimize two molecules with the assigned atom specifications, output one molecule and log file
#> molaical.exe -cmin -c "d:\Chimera\bin\chimera.exe" -m 2 -i1 lig_10.mol2 -i2 GCGRH.pdb -sv "#1" -os "#2:<0>" -o1 out_1.mol2 -w false -p false

Mode 3: minimization for two molecules with one fixed molecule

- ✧ For Modes 3 and 4, the value following -sv must be an atom-spec (the atom selection syntax in UCSF Chimera) and **zone specifiers**.
- ✧ In Mode 3, the default value of -sv is "#0 za<0.01".
- ✧ **Zone specifiers** define atoms and residues located within or beyond a specified distance

from reference atom(s). The operators z< and zr< select all residues containing any atom within the given distance from reference atoms, while za< selects all individual atoms within that distance. The complementary operators z>, zr>, and za> return the inverse sets of their < counterparts.

For example:
#1:UTK za<12.5
Means to select all atoms within 12.5 Å of any atom in residue UTK from model #1.

10) Minimize two molecules with one fixed molecule and output two molecules
#> molaical.exe -cmin -c "d:\Chimera\bin\chimera.exe" -m 3 -i1 lig_10.mol2 -i2 GCGRH.pdb -f1 GCGRH_fix.pdb -os "#3:<0>" -o1 out_1.mol2 -o2 out_2.pdb

11) Minimize two molecules with one fixed molecule and output one molecule
#> molaical.exe -cmin -c "d:\Chimera\bin\chimera.exe" -m 3 -i1 lig_10.mol2 -i2 GCGRH.pdb -f1 GCGRH_fix.pdb -os "#3:<0>" -o1 out_1.mol2 -o2 out_2.pdb -w false

12) Minimize two molecules with one fixed molecule and output one molecule
#> molaical.exe -cmin -c "d:\Chimera\bin\chimera.exe" -m 3 -i1 lig_10.mol2 -i2 GCGRH.pdb -f1 GCGRH_fix.pdb -os "#3:<0>" -o1 out_1.mol2 -w false

13) Minimize two molecules with one fixed molecule, output one molecule and log file
#> molaical.exe -cmin -c "d:\Chimera\bin\chimera.exe" -m 3 -i1 lig_10.mol2 -i2 GCGRH.pdb -f1 GCGRH_fix.pdb -os "#3:<0>" -o1 out_1.mol2 -o2 out_2.pdb -w false -p false

Mode 4: minimization for two molecules with two fixed molecules

- ✧ For Modes 3 and 4, the value following -sv must be an atom-spec (the atom selection syntax in UCSF Chimera) and **zone specifiers**.
- ✧ In Mode 4, the default value of -sv is "#0,1 za<0.01".
- ✧ **Zone specifiers** define atoms and residues located within or beyond a specified distance from reference atom(s). The operators z< and zr< select all residues containing any atom within the given distance from reference atoms, while za< selects all individual atoms within that distance. The complementary operators z>, zr>, and za> return the inverse sets of their < counterparts.

  For example:
  #1:UTK za<12.5
  Means to select all atoms within 12.5 Å of any atom in residue UTK from model #1.

14) Minimize two molecules with two fixed molecules and output two molecules
#> molaical.exe -cmin -c "d:\Chimera\bin\chimera.exe" -m 4 -i1 lig_10.mol2 -i2 GCGRH.pdb -f1 GCGRH_fix.pdb -f2 lig_fix.mol2 -os "#4:<0>" -o1 out_1.mol2 -o2 out_2.pdb

15) Minimize two molecules with two fixed molecules and output two molecules with the detailed options of MolAICal

#> molaical.exe -cmin -c "d:\Chimera\bin\chimera.exe" -m 4 -i1 lig_10.mol2 -i2 GCGRH.pdb -f1 GCGRH_fix.pdb -f2 lig_fix.mol2 -os "#4:<0>" -cm gas -fs freeze -sv "#0,1 za<0.01" -fv false -pv false -ns 10 -ss 0.02 -cs 2 -css 0.02 -iv 1 -ng true -mf1 mol2 -mf2 pdb -o1 out_1.mol2 -o2 out_2.pdb

16) Minimize two molecules with two fixed molecules, output one molecule and log file based on the full options of MolAICal

#> molaical.exe -cmin -c "d:\Chimera\bin\chimera.exe" -m 4 -i1 lig_10.mol2 -i2 GCGRH.pdb -f1 GCGRH_fix.pdb -f2 lig_fix.mol2 -os "#4:<0>" -cm gas -fs freeze -sv "#0,1 za<0.01" -fv false -pv false -ns 10 -ss 0.02 -cs 2 -css 0.02 -iv 1 -ng true -mf1 mol2 -mf2 pdb -o1 out_1.mol2 -o2 out_2.pdb -w false -p false

Note: For extended requirements, the UCSF Chimera User's Guide should be consulted. The provided "run.com" script of UCSF Chimera can be modified to fulfill additional computational aims. Each line in the run.com script file corresponds to an executable command. To run these sequentially:

a) Navigate to Favorites → Command Line in the UCSF Chimera toolbar to invoke the command interface.

b) Input individual command strings from each line in the run.com script file.

c) Press "Enter" key after each entry to execute.

The file "run.com" is as follows:

```
open E:/minimization/GCGRH_fix.pdb
open E:/minimization/lig_fix.mol2
open E:/minimization/GCGRH.pdb
open E:/minimization/lig_10.mol2
combine #2,3 modelId 4
delete #2,3
select #0,1 za<0.01
delete #0,1
addh
addcharge all method gas
minimize freeze selected prep false nsteps 10 stepsize 0.02 cgsteps 2 cgstepsize 0.02 interval 1 nogui true
~select
select #4:<0>
write selected format mol2 relative 4 #4 E:/minimization/out_1.mol2
select invert selected
write selected format pdb relative 4 #4 E:/minimization/out_2.pdb
stop
```

Then run script file "run.com" in the command console in Windows DOS or Linux:

#> molaical.exe -cmin -c "d:\Chimera\bin\chimera" -m 0 -i1 run.com

Or
#> molaical.exe -cmin -c "/home/feng/Chimera/bin/chimera" -m 0 -i1 run.com

4.3. Molecular properties calculation

4.3.1. Quantitative structure-activity relationship (QSAR)

Quantitative structure-activity relationship (QSAR) is a method involved in regression or classification used in the chemical and biological sciences and engineering. Here, the QSAR is embedded into the MolAICal software with the genetic algorithm. The detailed parameters for running QSAR are listed below:

-qsar: means QSAR calculation function.
-i: is the path of the input data file for QSAR calculation.
-im: is mutation ratio.
-ic: number selection for fittest calculation.
-ie: whether to choose the elitism way.
-ip: population size.
-iw: choose the validation. Currently, it contains least squares R2 and cross validation Q2.
-in: number of set variables. If it is set to 3, it will be like: y = a*x1+b*x2+c*x3.
-iv: number of evolutions.
-is: interval steps for repeating population generation.
-io: number of cores for parallel computing.

The example commands for QSAR calculations are as below:

1) QSAR calculation with default parameters
#> molaical.exe -qsar GA -i D:\\QSAR\\all_QSAR_no.txt

2) QSAR calculation with full parameters
#> molaical.exe -qsar GA -i D:\\QSAR\\test1.txt -im 0.5 -ic 5 -ie true -ip 200 -iw Q2 -in 3 -iv 150000 -is 100 -io 3

3) QSAR calculation uses parallel running way acquiescently
#> molaical.exe -qsar GA -i D:\\QSAR\\test1.txt -im 0.5 -ic 5 -ie true -ip 200 -iw Q2 -in 3 -iv 150000 -is 100

The default output file is named “QSAROutFile.dat” whose content is interpreted as follows:

Q^2-LOO: Leave-one-out cross validation. This algorithm is applied once for each instance, choosing all other instances as a training set and using the selected instance as a single-item test set.

(see: https://doi.org/10.1007/978-0-387-30164-8_469)

R^2 fitting: Correlation coefficients between experimental and predicted values (see below equation PI-4.3.1.1)

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum_i (y_i - f_i)^2}{\sum_i (y_i - \overline{y})^2} \qquad (\text{PI} - 4.3.1.1)$$

Where y_i, f_i, and $\overline{y}$ are experimental, predicted, and average values, respectively. SS_{res} is the sum of squares of residuals, also called the residual sum of squares. SS_{tot} is the total sum of squares.

R^2 adjusted: is also called Adjusted R-Squared. The adjusted R^2 is defined as equation PI-4.3.1.2:

$$adjusted\ \ R^2 = 1 - \frac{PRESS/\text{df}_e}{SS_{tot}/\text{df}_t} \qquad (\text{PI} - 4.3.1.2)$$

Where df_e is the degrees of freedom n – p – 1 of the estimate of the underlying population error variance, and df_t is the degrees of freedom n – 1 of the estimate of the population variance of the dependent variable.

RSS: residual sum of squares (see equation PI-4.3.1.3)

$$RSS = \sum_i (y_i - f_i)^2 \qquad (\text{PI} - 4.3.1.3)$$

PRESS: predictive residual sum of squares that is a form of cross-validation used in regression analysis to provide a summary measure of the fit of a model to a sample of observations that were not themselves used to estimate the model (https://en.wikipedia.org/wiki/PRESS_statistic) (see equation PI-4.3.1.4).

$$PRSS = \sum_{i=1}^{n} (y_i - f_{i/i})^2 \qquad (\text{PI} - 4.3.1.4)$$

SDEC: standard deviation error in calculation (see equation PI-4.3.1.5).

$$SDEC = \sqrt{\frac{RSS}{n}} \qquad (\text{PI} - 4.3.1.5)$$

SDEP: standard deviation error in prediction (see equation PI-4.3.1.6).

$$SDEP = \sqrt{\frac{PRESS}{n}} \qquad (\text{PI} - 4.3.1.6)$$

For more detailed knowledge and equations, please read the related content in Wikipedia: https://en.wikipedia.org/wiki/Coefficient_of_determination

external Q2: the explained variance in prediction (see equation PI-4.3.1.7).

$$Q^2_{ext} = 1 - \frac{\sum_{i=1}^{n_{ext}} (y_i - f_i)^2}{\sum_{i=1}^{n_{ext}} (y_i - \overline{y})^2} \qquad (\text{PI} - 4.3.1.7)$$

Where y_i, and f_i, are experimental and predicted values in the test set, respectively. $\overline{y}$ is the average

value of the training set data. If there are outliers, external Q2 will become a negative value. So users can only refer to external Q2, ignoring some abnormal values, or select other suitable algorithms and models for specific purposes.

4.3.2. Lipinski's rule of five calculation

Lipinski's rule of five, also known as the rule of five (RO5), is a rule to estimate drug-like or determine if a chemical compound has pharmacological or biological activity that would be likely an orally active drug[3]. The RO5 cutoff values are multiples of five, which is the origin of the rule's name. It contains three or four rules. The MolAICal evaluates drug-like with five rules:

1) No more than 5 hydrogen bond donors
2) No more than 10 hydrogen bond acceptors
3) A molecular mass less than 500 daltons
4) An octanol-water partition coefficient (log P) that does not exceed 5
5) No more than 10 rotatable bonds

In the past years, some rules of Lipinski's rule of five have changed according to the statistical research on the properties of FDA-approved drugs (see *J Med Chem. 2019 Feb 28;62(4):1701-1714*)[8]. To estimate the drug-like ligands, the author can set suitable parameters according to the studied needs. MolAICal supplies an easy way to set rule parameters.

Notice: please distinguish the molecular weight and molecular mass. Please see the detailed information about them: https://en.wikipedia.org/wiki/Molecular_mass. The MolAICal outputs the molecular mass of ligands.

-tool: appoint the tool function. Here, it is “ruleoffive”.
-f: molecular format.
-n: molecule file.
-i: the file that contains the molecules list.
-o: output results file.
-x: logP.
-a: number of hydrogen bond acceptors.
-d: number of hydrogen bond donors.
-m: molecular mass.
-r: number of rotatable bonds.

The example commands for RO5 calculations are as below:

1) Calculate RO5 with default original rules (see above five rules)
#> molaical.exe -tool ruleoffive -f mol2 -n "D:\zinc_1879871.mol2"

2) Calculate RO5 of the ligands set. Note: “mol2List.dat” must be in the directory of the molecular

set.
#> molaical.exe -tool ruleoffive -f mol2list -i "D:\mol2List.dat" -o "D:\result.dat"

3) Calculate RO5 with your given values of rules
#> molaical.exe -tool ruleoffive -f mol2list -i "D:\mol2List.dat" -o "D:\result.dat" -x 5.0 -a 10 -d 5 -m 500.0 -r 10

4.3.3. PAINS calculation

Pan-assay interference compounds (PAINS) are the compounds that often show the false positive results in the biological assay[4]. PAINS tend to react with numerous biological receptors nonspecifically rather than binding the desired target specifically[27]. Some disruptive functional groups are shared by many PAINS. The Common PAINS contains "toxoflavin, isothiazolones, curcumin, hydroxyphenyl hydrazones, phenol-sulfonamides, enones, rhodanines, catechols, and quianones"[27]. The MolAICal software provides an easy way to calculate PAINS:

-tool: appoint the tool function. Here, it is "pains".
-f: molecular format. It contains "SMILES" and "mol2" formats. We recommend "SMILES" format. You can use Open Babel[28] to change it to "SMILES" format.
-n: input molecular file or SMILES sequence.
-i: input file which contains molecules list or sequences.
-o: output results.

The example commands for PAINS calculations are as below:
1) Calculate PAINS with SMILES format
#> molaical.exe -tool pains -f smiles -n "c1ccccc1N=Nc1ccccc1"

2) Calculate PAINS with abbreviation SMILES command parameter
#> molaical.exe -tool pains -f smi -n "c1ccccc1N=Nc1ccccc1"

3) Calculate PAINS with mol2 format
#> molaical.exe -tool pains -f mol2 -n "D:\ZINC00154323.mol2"

4) Calculate PAINS from the file with contains the list of molecular SMILES sequences
#> molaical.exe -tool pains -f smilist -i "D:\painsTest.txt" -o "D:\smiResult.txt"

5) Calculate PAINS from the file with contains mol2 list
#> molaical.exe -tool pains -f mol2list -i "D:\list.dat" -o "D:\mol2Result.txt"

4.3.4. Synthetic Accessibility calculation

Synthetic accessibility (SA)[29-30] is an important aspect of drug design. The prediction of SA can guide the *de novo* drug design and give chemists useful information for compound synthesis. The calculated parameters of SA by MolAICal are shown below:

-tool: appoint the tool function. Here, it is “sa”.
-i: input molecular SMILES sequence or file containing the list of molecular SMILES sequences.

The example commands for SA calculations are as below:
1) Calculate SA with molecular SMILES sequence
#> molaical.exe -tool sa -i "CC(=O)OC1=CC=CC=C1C(=O)O"

2) Calculate SA from the file that contains the list of molecular SMILES sequences
#> molaical.exe -tool sa -i "D:\\SA\\SmilTest.smi"

Note: the higher value of SA indicates the compound is easier to synthesize.

4.3.5. Medicinal Chemistry Filters (MCFs) calculation

Medicinal chemistry filters (MCFs) function to eliminate compounds that contain so-called "**structural alerts**". Molecules with such moieties either have **unstable or reactive groups** or they undergo biotransformations that result in the production of toxic metabolites or intermediates.

MCFs were employed for the rational pre-selection of compounds more suitable for modern drug design and development. They include certain electrophilic alkylating groups, such as Michael acceptors (MCF1-3), alkyl halides (MCF4), epoxides (MCF5), isocyanates (MCF6), aldehydes (MCF7), imines (Schiff base, MCF8), and aziridines (MCF9), all of which are highly prone to nucleophilic attack. In many cases, this leads to non-selective damage to proteins and/or DNA. Hydrazines (MCF10) are metabolized to produce diazene intermediates (MCF11), which also function as alkylating warheads. Monosubstituted furans (MCF12) and thiophenes (MCF13) are converted into reactive intermediates through epoxidation. Their active metabolites bind irreversibly to nucleophilic groups and modify proteins. Electrophilic aromatics (e.g., halopyridines, MCF14), oxidized anilines (MCF15), and disulfides (MCF16) are also highly reactive. In vivo, alkylating agents are trapped and inactivated by the thiol group of glutathione, a key natural antioxidant. Azides (MCF17) are highly toxic; compounds containing this functional group particularly induce genotoxicity. Aminals (MCF18) and acetals (MCF19) are often unstable and inappropriate in generated structures. Additionally, virtual structures containing a large number of halogens (MCF20-22) should be excluded due to increased molecular weight and lipophilicity (insufficient solubility for oral administration), issues with metabolic stability, and toxicity. The detailed toxicity mechanisms associated with the aforementioned structural alerts have been comprehensively described in the previous researches[31-33].

PAINS (pan-assay interfering compounds) filters consist of a set of substructure filters designed to reduce false positives, assay artifacts, and non-specific bioactive molecules in screening libraries. They are based on the premise that certain fragments within a structure can give rise to undesirable properties—such as reactivity, chelation, colloidal aggregate formation, and dye-related effects—that interfere with assay results.

Since both MCFs and PAINS involve reactivity, here, **MCFs** function in MolAICal **encompasses structures covered by both MCFs and PAINS.**

In addition, this calculation also outputs some properties of the molecule as follows:

- ✧ **SMILES**: the SMILES sequence of the input molecule.
- ✧ **Valid**: determines whether a molecule is valid; its value is True or False.
- ✧ **Passes filters**: determines whether it passes the MCFs evaluation, its value is True or False. If the value is True, it indicates that the molecule passes the MCFs and PAINS filters
- ✧ **LogP**: octanol-water partition coefficient, a proportion of a substance's concentration in the octanol phase relative to its concentration in the aqueous phase of a biphasic octanol/water system.
- ✧ **SA Score**: also known as **Synthetic Accessibility Score**, a rule-of-thumb assessment of the **difficulty (rated 10)** or **ease (rated 1)** of synthesizing a specific molecule. The SA score is derived from a blend of contributions from the molecule's fragments[29].
- ✧ **NP Score**: also known as **Natural Product-likeness Score**, a Bayesian metric that enables the assessment of how similar molecules are to the structural space encompassed by natural products[34].
- ✧ **QED**: also known as **Quantitative Estimation of Drug-likeness**, a [0, 1] value assessing the likelihood of a molecule being a feasible drug candidate. Like SA, the descriptor thresholds in QED have shifted over the past decade, and current thresholds may not include the most recent drugs[8].
- ✧ **Molecular weight (MW)**: the summation of atomic weights in a compound.

The calculated parameters of MCFs in MolAICal are shown below:

usage: MCFs

-tool <arg>	appoint the tool function. Here, it is "mcf".
-f <arg>	Output format (default: text), values contain: 'text', 'json', 'csv'.
-h	Print help message.
-i <arg>	File containing SMILES strings (one per line).
-o <arg>	Output file path (default: stdout).
-p <arg>	values: true and false. "true" means: output molecular properties, and vice versa.
-s <arg>	SMILES string to process.

The example commands for MCFs calculations are as below:

1) Calculate MCFs for a single SMILES string

#> molaical.exe -tool mcf -s "CC(=O)OC1=CC=CC=C1C(=O)O"

2) Calculate MCFs for a single SMILES string without properties
#> molaical.exe -tool mcf -s "CC(=O)OC1=CC=CC=C1C(=O)O" -p false

3) Calculate MCFs with output file and use format (e.g., 'text', 'json', or 'csv')
#> molaical.exe -tool mcf -s "CC(=O)OC1=CC=CC=C1C(=O)O" -o result.csv -f csv

4) Calculate MCFs with the output file and no properties output
#> molaical.exe -tool mcf -s "CC(=O)OC1=CC=CC=C1C(=O)O" -o result.csv -f csv -p false

5) Calculate MCFs from the file, which can contain multiple lines of SMILES, with one molecule's SMILES represented per line
#> molaical.exe -tool mcf -i molecules.smi -o result.csv -f csv

6) Calculate MCFs from the file without properties output, which can contain multiple lines of SMILES, with one molecule's SMILES represented per line.
#> molaical.exe -tool mcf -i molecules.smi -o result.csv -f csv -p false

Note: Only a few examples are listed here; MCFs calculations can be run with various parameters or combinations thereof.

4.3.6. MCE-18 Descriptor calculation

MCE-18, standing for "medicinal chemistry evolution, 2018", is an original molecular descriptor proposed to effectively score the novelty and lead potential of pharmacologically relevant molecules[35]. It reflects the quality and complexity of the overall molecular framework, particularly the cumulative **sp^3** complexity, and helps trace the evolution of medicinal chemistry. It can be applied to assess molecule novelty, profile high-throughput screening (HTS) libraries, prioritize molecules, and identify compounds with reliable intellectual property (IP) positions.

Formula of MCE-18:

$$\text{MCE-18}=\left(\text{AR}+\text{NAR}+\text{CHIRAL}+\text{SPIRO}+\frac{\overbrace{\text{sp}^3+\text{Cyc-Acyc}}^{\text{NCSPTR}}}{1+\text{sp}^3}\right)\times Q^1$$

Explanation of Parameters

- ✧ **AR**: Presence of an aromatic or heteroaromatic ring (0 = absent, 1 = present).
- ✧ **NAR**: Presence of an aliphatic or heteroaliphatic ring (0 = absent, 1 = present).
- ✧ **CHIRAL**: Presence of a chiral center (0 = absent, 1 = present).
- ✧ **SPIRO**: Presence of a *spiro* point (0 = absent, 1 = present).
- ✧ **sp^3**: Proportion of sp^3-hybridized carbon atoms (ranging from 0 to 1).
- ✧ **Cyc**: Proportion of cyclic carbons that are sp^3-hybridized (ranging from 0 to 1).
- ✧ **Acyc**: Proportion of acyclic carbons that are sp^3-hybridized (ranging from 0 to 1).
- ✧ **NCSPTR**: A composite term calculated as (sp^3 + Cyc - Acyc).

✧ **Q^1**: Normalized quadratic index, encoding the degree of branching of the structural framework.

✧ **Is a higher MCE-18 value always better, or is there an optimal range?**

The value of MCE-18 is positively correlated with the novelty and 3D complexity of a molecule, but "a higher value is not always better". Its suitability depends on the stage of drug development and practical application scenarios; **all the contents below are summarized from the MCE-18 paper[35]:**

➢ **From the perspective of medicinal chemistry evolution, an increase in MCE-18 is an overall trend**

Data in the paper show that between 1950 and 2018, the average MCE-18 value of compounds in patents of major pharmaceutical companies increased from 43.2 to 75.9, indicating that the 3D complexity and novelty of modern drug molecules have significantly improved[35]. This trend is consistent with the needs of complex targets such as protein-protein interactions (PPIs) — such targets require higher structural complexity to match their large and deep binding sites (800-2000 $Å^3$). Thus, **in early research and development (e.g., patent compounds, preclinical candidates), a higher MCE-18 value generally indicates stronger innovation and potential adaptability to complex targets**.

➢ **From the full drug development process, excessively high MCE-18 may pose challenges**

The paper found that MCE-18 values gradually decrease from preclinical research to marketed drugs[35]: the average value of preclinical candidates is 57.3, while that of marketed drugs is 45.98. This is because excessively high structural complexity may increase synthetic difficulty and deteriorate pharmacokinetic properties (e.g., solubility, bioavailability), leading to elimination in later development. Therefore, **in the later stages of drug development, a higher MCE-18 value is not always better; a balance between complexity and developability is needed**.

➢ **From practical application scenarios, 45-78 is a relatively optimal range**

Analysis of supplier compound libraries (e.g., ChemDiv, Enamine) shows[35]:

◆ MCE-18 < 45: Molecules have simple structures, low novelty, mostly "old scaffolds", and limited attractiveness;

◆ MCE-18: 45-78: Sufficient novelty, in line with current trends in medicinal chemistry (45-63), and high structural similarity to patent compounds of pharmaceutical companies during the past decade (63−78);

◆ MCE-18 > 78: Need manual evaluation of target adaptability and drug-likeness, as excessive complexity may lead to development difficulties.

MCE-18 is an effective indicator to measure molecular novelty and 3D complexity. Its increasing value reflects the evolution of modern medicinal chemistry towards more complex structures, especially suitable for early research and development stages (e.g., patent compounds, preclinical candidates). However, **a higher value is not always better**: excessively high values may increase synthetic and development difficulties, and the range of 45-78 is more in line with practical application needs (balancing novelty and developability). The relatively low MCE-18 values of marketed drugs further indicate that a balance between complexity and druggability must be struck.

The calculated parameters of MCE-18 in MolAICal are shown below:

usage: mce18

-tool <arg>	appoint the tool function. Here, it is "mce18".
-h	Print help information.
-i <arg>	Input file containing SMILES strings (one per line).
-o <arg>	Output CSV file path.
-q <arg>	values: true and false. “true” means: suppress detailed output, and vice versa.
-s <arg>	SMILES string of the molecule.
-sm <arg>	values: true and false. “true” means: only output SMILES and MCE-18 value, and vice versa.
-v <arg>	values: true and false. “true” means: print detailed results, and vice versa.

The example commands for MCE-18 calculations are as below:

1) Printi help information

#> molaical.exe -tool mce18 -h

2) Calculate MCE-18 for a single SMILES string with printing detailed results

#> molaical.exe -tool mce18 -s "CC(=O)OC1=CC=CC=C1C(=O)O" -v true

3) Calculate MCE-18 for a single SMILES

#> molaical.exe -tool mce18 -s "CC(=O)OC1=CC=CC=C1C(=O)O"

4) Calculate MCE-18 for a single SMILES without detailed output

#> molaical.exe -tool mce18 -s "CC(=O)OC1=CC=CC=C1C(=O)O" -q true

5) Only output SMILES and MCE-18 value

#> molaical.exe -tool mce18 -s "CC(=O)OC1=CC=CC=C1C(=O)O" -sm true

6) Only output SMILES and MCE-18 value without detailed output

#> molaical.exe -tool mce18 -s "CC(=O)OC1=CC=CC=C1C(=O)O" -sm true -q true -v true

7) Input SMILES and output CSV

#> molaical.exe -tool mce18 -s "CC(=O)OC1=CC=CC=C1C(=O)O" -o output.csv

8) Input SMILES and output CSV without detailed output

#> molaical.exe -tool mce18 -s "CC(=O)OC1=CC=CC=C1C(=O)O" -o output.csv -q true

9) Input SMILES and output CSV only containing SMILES and MCE-18 value

#> molaical.exe -tool mce18 -s "CC(=O)OC1=CC=CC=C1C(=O)O" -o output.csv -sm true

10) Input SMILES and output CSV only containing SMILES and MCE-18 value without printing results

#> molaical.exe -tool mce18 -s "CC(=O)OC1=CC=CC=C1C(=O)O" -o output.csv -sm true -q true -v true

11) Input file containing SMILES strings (one per line) and output CSV file
#> molaical.exe -tool mce18 -i example_smiles.txt -o results.csv

12) Input file containing SMILES strings (one per line) and output CSV file with printing detailed results for calling process
#> molaical.exe -tool mce18 -i example_smiles.txt -o results.csv -v true

13) Input file containing SMILES strings (one per line) and output CSV without printing results
#> molaical.exe -tool mce18 -i example_smiles.txt -o results.csv -q true

14) Input file containing SMILES strings (one per line) and output CSV only containing SMILES and MCE-18
#> molaical.exe -tool mce18 -i example_smiles.txt -o results.csv -sm true

15) Input file containing SMILES strings (one per line) and output CSV only containing SMILES and MCE-18 without printing results
#> molaical.exe -tool mce18 -i example_smiles.txt -o results.csv -sm true -q true

16) Input file containing SMILES strings (one per line) and output CSV only containing SMILES and MCE-18 without detailed output
#> molaical.exe -tool mce18 -i example_smiles.txt -o results.csv -sm true -q true -v true

4.4. Vinardo score calculation

Vinardo score, which is based on the Vina score function[14], is trained on the basis of the PDBbind database[11, 36-37]. The score function of Vina is shown below. The Vinardo score can be trained by the following equations:

$$Gauss_1(d) = e^{-((d-o_1)/s_1)^2} \quad (1)$$

$$Gauss_2(d) = e^{-((d-o_2)/s_2)^2} \quad (2)$$

$$Repulsion(d) = \begin{cases} d^2, & \text{if } d < 0\text{Å} \\ 0, & \text{if } d \geq 0\text{Å}, \end{cases} \quad (3)$$

$$Hydrophobic(d) = \begin{cases} 1, & \text{if } d < p_1 \\ p_2 - d, & \text{if } p_1 \leq d \leq p_2 \\ 0, & \text{if } d > p_2 \end{cases} \quad (4)$$

$$Hbond(d) = \begin{cases} 1, & \text{if } d < h_1 \\ 1 - \dfrac{d - h_1}{-h_1}, & \text{if } h_1 \leq d \leq 0\text{Å} \\ 0, & \text{if } d > 0\text{Å} \end{cases} \quad (5)$$

4.4.1. Batch of Vinardo score calculation

MolAICal retrains the Vinardo score according to the above five equations. And the command parameters are as below:

-score: score function for evaluation of binding affinity. Here, it is “vinardo”.
-i: input molecular file or file containing molecules’ list.
-o: output result.
-g: the referred grid file for binding affinity calculation.
-p: pick up the computing way for Vinardo score calculations.
-n: number of cores for parallel computing.

The example commands for Vinardo score calculations are as below:

1) Calculate Vinardo score without loading all molecules
#> molaical.exe -score vinardo -i D:\workdir\ligandList.dat -o D:\workdir\output.dat -g D:\workdir\MAICGrid.dat

2) Calculate Vinardo score by loading all molecules.
#> molaical.exe -score vinardo -i D:\workdir\ligandList.dat -o D:\workdir\output.dat -g D:\workdir\MAICGrid.dat -p memoryWay -n 2

4.4.2. Vinardo score Decomposition

The MolAICal can calculate the $Gauss_1$ score, $Gauss_2$ score, Repulsion score, Hydrophobic score, Hbond score, and Vinardo score based on one ligand, respectively (see equations in 3.7). The calculated parameters are below:

-tool: appoint the tool function. Here, it is “vinardo”.
-i: input molecular file.
-g: the referred grid file.
-t: type of score which is gaussScore1, gaussScore2, repulsionScore, hydrophobicScore, hbondScore or vinardoScore.
-n: number of cores for parallel computing.

1) Calculate Vinardo score from one molecule.
#> molaical.exe -tool vinardo -i D:\workdir\1a1e_ligand.mol2 -g D:\workdir\vinardoScore.dat -t vinardoScore

2) Calculate $Gauss_1$ score with 1 core of computer

#> molaical.exe -tool vinardo -i D:\workdir\1a1e_ligand.mol2 -g D:\workdir\gaussScore1.dat -t gaussScore1 -n 1

3) Calculate $Gauss_2$ score
#> molaical.exe -tool vinardo -i D:\workdir\1a1e_ligand.mol2 -g D:\workdir\gaussScore2.dat -t gaussScore2

4) Calculate Repulsion score
#> molaical.exe -tool vinardo -i D:\workdir\1a1e_ligand.mol2 -g D:\workdir\repulsionScore.dat -t repulsionScore

5) Calculate Hydrophobic score
#> molaical.exe -tool vinardo -i D:\workdir\1a1e_ligand.mol2 -g D:\workdir\hydrophobicScore.dat -t hydrophobicScore

6) Calculate Hbond score
#> molaical.exe -tool vinardo -i D:\workdir\1a1e_ligand.mol2 -g D:\workdir\hbondScore.dat -t hbondScore

4.4.3. Grid file generation for score calculation

The $Gauss_1$ score, $Gauss_2$ score, Repulsion score, Hydrophobic score, Hbond score, and Vinardo score need the grid files for binding score calculation. MolAICal supplies a module to calculate the relative grid files for binding score calculation. The parameters are below:

-tool: appoint the tool function. Here, it is “grid”.
-i: the path of the box file, which is measured based on the receptor.
-p: the path of the receptor file.
-n: term of Vinardo score which can be gaussScore1, gaussScore2, repulsionScore, hydrophobicScore, hbondScore or vinardoScore.
The example commands are as below:

1) Grid file generation of $Gauss_1$ score
#> molaical.exe -tool grid -i "D:\1a1e\boxPar.dat" -p "D:\1a1e\protein2.pdb" -n gaussScore1

2) Grid file generation of $Gauss_2$ score
#> molaical.exe -tool grid -i "D:\1a1e\boxPar.dat" -p "D:\1a1e\protein2.pdb" -n gaussScore2

3) Grid file generation of Repulsion score
#> molaical.exe -tool grid -i "D:\1a1e\boxPar.dat" -p "D:\1a1e\protein2.pdb" -n repulsionscore

4) Grid file generation of Hydrophobic score

#> molaical.exe -tool grid -i "D:\1a1e\boxPar.dat" -p "D:\1a1e\protein2.pdb" -n hydrophobicscore

5) Grid file generation of Hbond score
#> molaical.exe -tool grid -i "D:\1a1e\boxPar.dat" -p "D:\1a1e\protein2.pdb" -n hbondscore

6) Grid file generation of Vinardo score
#> molaical.exe -tool grid -i "D:\1a1e\boxPar.dat" -p "D:\1a1e\protein2.pdb" -n vinardoScore

4.4.4. Box generation based on receptor

Grid file generation needs the box parameters, which are extracted on the basis of the receptor. MolAICal supplies a function to generate box parameters that can be displayed in UCSF Chimera[38]. The parameters are below:

-tool: appoint the tool function. Here, it is "box".
-i: box center.
-l: box length.
-t: transparency shown in UCSF Chimera.
-c: color shown in UCSF Chimera.
-g: grid space. The default value is 1.0 Å.

The example commands are as below:
1) Calculate the box parameter by using the default value. (Note: **the double quotes are necessary for X, Y, Z coordinates. The interval distance between X, Y, Z coordinates should be one space**.)
#> molaical.exe -tool box -i "52.646 7.634 53.411" -l "30.0 30.0 30.0" -o "D:\chimerabox\box.bild"

2) Calculate box parameter with appointed value
#> molaical.exe -tool box -i "52.646 7.634 53.411" -l "40.0 30.0 30.0" -t 0.5 -c red -g 1.0 -o "D:\box.bild"

3) This command emphasizes the importance of double quotes. The minus sign must be enclosed by double quotes.
#> molaical.exe -tool box -i "-30.011 1.665 -36.581" -l "30.0 30.0 30.0" -o "D:\box.bild"

4.4.5. Binding free energy from pKd or pKi

To train the Vinardo score, it needs to calculate the binding free energy according to the Ki, Kd, pKd, or pKi values from the PDBBind database[36, 39-40]. Here, the MolAICal provides an easy way to calculate binding free energy if the value of Ki, Kd, pKd, or pKi is given.

Here, the International System of Units (SI) is introduced (see Table 4.4.5.1). Most laboratory and literatures uses mol/dm^3, which is the same as mol/L (also named "M"). For example:

$mol/m^3 = 10^{-3}$ $mol/dm^3 = 10^{-3}$ mol/L = 10^{-3} M = 1 mmol/L = 1 mM.

Table 4.4.5.1 Molar concentration units

Name	Abbreviation	Concentration	Concentration (SI unit)
millimolar	mM	10^{-3} mol/L	10^{0} mol/m^3
micromolar	μM	10^{-6} mol/L	10^{-3} mol/m^3
nanomolar	nM	10^{-9} mol/L	10^{-6} mol/m^3
picomolar	pM	10^{-12} mol/L	10^{-9} mol/m^3
femtomolar	fM	10^{-15} mol/L	10^{-12} mol/m^3
attomolar	aM	10^{-18} mol/L	10^{-15} mol/m^3
zeptomolar	zM	10^{-21} mol/L	10^{-18} mol/m^3
yoctomolar	yM	10^{-24} mol/L (6 particles per 10 L)	10^{-21} mol/m^3

The "millimolar" and "micromolar" refer to mM and μM (10^{-3} mol/L and 10^{-6} mol/L), respectively. About the detailed relative information of molar concentration units, please see the website: https://en.wikipedia.org/wiki/Molar_concentration

The binding free energy is calculated as the equation below:

Binding free energy = $RT * \log_e(10^{-pKx})$

where T and R are the temperature and gas constant, respectively.

The parameters for binding free energy calculation in MolAICal are shown as below:

-tool: appoint the tool function. Here, it is “pkdpki”.
-i: input value.
-t: type of calculating way. It can choose “pkx” (meaning pKd or pKi) or “molar” (meaning Ki or Kd).
-u: unit of molar. It can choose values from the second column values of Table 4.4.5.1.
-k: temperature. The default temperature is 298.15 K.

1) Calculate binding free energy from pkd (pkd = -logKd or pki = -logKi), use default temperature: 298.15 K
#> molaical.exe -tool pkdpki -i 11.92 -t pkx

2) Calculate binding free energy from pkd (pkd = -logKd or pki = -logKi), use appointed temperature.
#> molaical.exe -tool pkdpki -i 11.92 -t pkx -k 300

3) Calculate from Kd or Ki with concentration. The default is M (mol/L)
#> molaical.exe -tool pkdpki -i 1.2 -t molar

4) Calculate from Kd or Ki with pm unit
#> molaical.exe -tool pkdpki -i 1.2 -t molar -u pm

5) Calculate from Kd or Ki with pm unit under 300 K
#> molaical.exe -tool pkdpki -i 1.2 -t molar -u pm -k 300

4.5. Molecular descriptor

DRAGON is used with a commercial license. To let the researcher study freely and handily. The PaDEL-Descriptor[41] and Mordred[42] are introduced to calculate descriptors. PaDEL-Descriptor, which contains 12 types of fingerprints (total 16092 bits) and 1875 descriptors (1444 1D, 2D descriptors, and 431 3D descriptors), is free for all (e.g., personal, academic, non-profit, non-commercial, government, commercial, etc) to use. Mordred, which contains 1826 descriptors (Copyright (c) 2015-2017, Hirotomo Moriwaki), uses the BSD 3-Clause "New" or "Revised" License (see: https://github.com/mordred-descriptor/mordred/blob/develop/LICENSE). Based on the functions of PaDEL-Descriptor and Mordred, MolAICal supplies molecules in SDF molecular format for descriptor calculation. The Mordred is used to calculate 3D molecular descriptors via converting the SMILES string to the 3D conformational structure based on the MMFF94 force field[43] in MolAICal. The commands and parameters for descriptor calculation are below:

-tool: choose the descriptor tool, it can be “mordred” or “padel”.
-f: file format.
-i: input file.

The example commands are as below:
1) Calculating the molecular descriptors based on Mordred. It will output the 2D descriptor file named “without3D-descriptors.csv” and the 3D descriptor file named “with3D-descriptors.csv”.
#> molaical.exe -tool mordred -i E:\006-QSAR\mordred\example.smi

2) Calculating the molecular descriptors based on SDF format via PaDEL-Descriptor. Please input the directory that only contains files in SDF format. It will output the 2D descriptor file named “2DDescriptor_mdl.csv” and the 3D descriptor file named “3DDescriptor_mdl.csv”.
#> molaical.exe -tool padel -f sdf -i E:\006-QSAR\PaDEL\sdf

3) Calculating the molecular descriptors based on MDL format via PaDEL-Descriptor. Please input the directory that only contains files in MDL format. It will output the 2D descriptor file named “2DDescriptor_mdl.csv” and the 3D descriptor file named “3DDescriptor_mdl.csv”.
#> molaical.exe -tool padel -f mdl -i E:\006-QSAR\PaDEL\mdl

4) Calculating the molecular descriptors based on SMILES format via PaDEL-Descriptor. Please input the directory that only contains files in SMILES format. It will output the 2D descriptor file named “2DDescriptor_2Dstrcuture.csv” based on SMILES strings and the 2D descriptor file named

“2DDescriptor_3Dstrcuture.csv” based on the converted 3D structures from SMILES strings.
#> molaical.exe -tool padel -f smi -i E:\006-QSAR\PaDEL\smi

In certain scenarios, it is necessary to obtain and analyze molecular descriptors from RDKit. To facilitate this process, MolAICal provides an interface specifically designed to access descriptors generated by RDKit. The following outlines the available commands and parameters:

usage: rdkit descriptor

-tool:	appoint the tool function. Here, it is “rdkit”.
-h	Print help information.
-i <arg>	Calculate descriptors for molecules in an input file.
-n <arg>	values: true and false. “false” means: do not include 3D descriptors (faster). “true” means: include 3D descriptors. Default value: “false”.
-o <arg>	Write results to a CSV file.
-s <arg>	Calculate descriptors for a SMILES string.

The example commands are as below:
1) Input SMILES string and output RDKit descriptors without 3D descriptors in console
#> molaical.exe -tool rdkit -s "CCO"

2) Input SMILES string and output RDKit descriptors including 3D descriptors in console
#> molaical.exe -tool rdkit -s "CCO" -n true

3) Input SMILES string and output RDKit descriptors in CSV file without 3D descriptors
#> molaical.exe -tool rdkit -s "CCO" -o ethanol.csv

4) Input SMILES string and output RDKit descriptors in CSV file including 3D descriptors
#> molaical.exe -tool rdkit -s "CCO" -o ethanol.csv -n true

5) Input SMILES file and output RDKit descriptors in CSV file without 3D descriptors
#> molaical.exe -tool rdkit -i molecules.smi -o descriptors.csv

6) Input SMILES file and output RDKit descriptors in CSV file including 3D descriptors
#> molaical.exe -tool rdkit -i molecules.smi -o descriptors.csv -n true

5. Analysis for drug design

5.1. k-means clustering

The k-means clustering is a popular way of clustering analysis in data mining. The results of drug

virtual screening show very similar structures and affinity scores. Even though the screened drugs have good binding scores, they may have very similar structures and functions. It is difficult to pick the representative molecules for further drug assay. To solve this problem, the k-means algorithm is used to cluster the results of drug design for further activity assay. The parameters for k-means in MolAICal are:

-tool: appoint the tool function. Here, it is “kmeans”.
-n: clustering number.
-i: the file of the principal component.
-o: output file of clustering results.

The example commands are as below:
1) k-means command for clustering
#> molaical.exe -tool kmeans -n 3 -i "D:\PC.dat" -o "D:\result.dat"

5.2. Similarity search

5.2.1. Fingerprint for similarity search

Molecular fingerprints are a way to encode the molecular structure with a series of binary digits (bits) that can be used to compare the similarity between two molecules. This function employs Tanimoto coefficient[44] to measure the similarity of two sets’ elements (see below equation PI-5.2.1.1):

$$\mathrm{T}(a,b) = \frac{N_c}{N_a + N_b - N_c} \qquad (\mathrm{PI} - 5.2.1.1)$$

Where N is the number of attributes in sets *a* and *b*. C is the intersection set.

The parameters in MolAICal are shown below:

-tool: appoint the tool function. Here, it is “finger”.
-i: input file of molecules.
-o: calculated output results.

The example commands are as below:
1) Fingerprint calculation command
#> molaical.exe -tool finger -i "D:\workdir\listfiles.dat" -o "D:\workdir\result.dat"

5.2.2. 3D structures similarity comparison

It can compare 3D molecular similarity between two ligands on the basis of four specific points, which contain the centroid of the ligands, two atoms that are closest to and farthest from the centroid, and one atom that is farthest from the previous atom. The detailed algorithm is introduced by Ballester PJ and Richards WG[45]. The parameters in MolAICal are shown below:

-tool: appoint the tool function. Here, it is "3Dcompare".
-i: the first input file, it can be a mol2 format file or a list file containing molecular names of mol2 format files.
-s: the second file, it can be a mol2 file or an output file.
-f: file format, it can be "mol2" or "mol2list". The default value is "mol2".

The example commands are as below:
1) Calculate two molecular similarity with the default value.
#> molaical.exe -tool 3Dcompare -i D:\workdir\MolAICal\example\3Dcompare\ligand.mol2 -s D:\workdir\MolAICal\example\3Dcompare\1_10023_out.mol2

2) Calculate two molecular similarity with the mol2 format.
#> molaical.exe -tool 3Dcompare -i D:\workdir\MolAICal\example\3Dcompare\ligand.mol2 -s D:\workdir\MolAICal\example\3Dcompare\1_10023_out.mol2 -f mol2

3) Calculate molecular similarity from a list file. **Note:** the first line in the file is compared to the remaining lines in the file. So please put your compared molecular name in the first place of the file.
#> molaical.exe -tool 3Dcompare -i D:\workdir\MolAICal\example\3Dcompare\list.txt -s D:\workdir\MolAICal\example\3Dcompare\result.dat -f mol2list

5.3. Merge and split files

5.3.1. Extract data from one column of file

This function can extract one column data from a file. If the users want to extract some appointed column data from a file, the users can use this function to extract the column data one by one, and then, employ the function of MolAICal to merge these extracted files by column. The parameters in MolAICal are shown below:

-tool: appoint the tool function. Here, it is "colth".
-i: path of input file.
-o: path of output file.
-n: column order. It begins with 0. The first column is 0, and so on.

The example commands are as below:
1) Extract data from the second column of file
#> molaical.exe -tool colth -i "D:\mergecol\5ee7-final_result.txt" -o "D:\mergecol\ouputfile.dat" -n 1

5.3.2. Merge column of two files

MolAICal supplies the function to merge two files by column. For example, one file contains affinity scores, and the other contains fingerprint values. This function can merge these two files into one file that can be named as “PC.dat” for k-means clustering. You can merge more files one by one based on the previously merged file. The parameters in MolAICal are shown below:

-tool: appoint the tool function. Here, it is “col”.
-f: the path of the first input file.
-l: the path of the second input file.
-s: separator for merging columns of two files.
-o: path of **the** output merged file.

The example commands are as below:
1) command example for merging two files
#> molaical.exe -tool col -f "D:\name.dat" -l "D:\Fingerprint.dat" -s " " -o "D:\all.dat"

5.3.3. Merge files

ZINC database[46] supplies many small molecular sets in mol2 format. Merging and splitting molecular files with the mol2 format are necessary for handling drug design. Here, MolAICal supplies the merge function for dealing with molecular files in mol2 format. The parameters in MolAICal are shown below:
-tool: appoint the tool function. Here, it is “merge”.
-m: appointed keyword. MolAICal will merge the file whose name contains this keyword.
-i: path of input directory.
-o: path of output file.

The example commands are as below:
1) Mol2 format files merging command example
#> molaical.exe -tool merge -m mol2 -i "D:\merge" -o "D:\merge\all.mol2"
2) PDBQT format files merging command example
#> molaical.exe -tool merge -m PDBQT -i "D:\merge" -o "D:\merge\all.pdbqt"

5.3.4. Split and number statistics of mol2 file

In some cases, it needs to split the molecular file into multiple files for drug design. For instance, MolAICalD performs drug virtual screening one by one. It needs to divide the molecular set into many molecular files. The MolAICal software supplies a function for splitting the molecular set into multiple molecular files with the mol2 format. The parameters in MolAICal are shown below:

-tool: appoint the tool function. Here, it is “mol2”.
-w: choose functions. It can be “split”, “chunk”, or “num”. “split” means file split, “chunk” means to split the mol2 file into small chunks, and “num” means to count the molecular number in the entire mol2 file.
-n: when choosing the parameter “chunk”, it means the appointed number of molecules in one split chunk file. When choosing the parameter “num”, it means the interval number for the mol2 file splitting. The default value is 1.
-m: It can be “number” or “name”. If the value is “number”, it will split the molecular set into multiple molecular files named with ordered numbers, e.g., 1.mol2, 2.mol2, and so on. If the value is “name”, it will split the molecular set into multiple molecular files named with the string that is under the line “@<TRIPOS>MOLECULE” in the mol2 format file. The default value is “name”.
-v: whether to overlap the existing molecular file. The default value is true for overlap.
-i: path of input molecular mol2 file.
-o: path of output directory.

The example commands are as below:
1) Split molecular set to multiple single molecules named by the string that under the line “@<TRIPOS>MOLECULE” in mol2 file:

```
#> molaical.exe -tool mol2 -w split -n 1 -v true -i "D:\split\all.mol2" -o "D:\split\1"
```

2) Census the number of molecular set:

```
#> molaical.exe -tool mol2 -w num -i "D:\split\all.mol2"
```

3) Splits mol2 file to the small chunk which contains 20 molecules:

```
#> molaical.exe -tool mol2 -w chunk -n 20 -i "D:\split\all.mol2" -o "D:\split\2"
```

4) Split molecular set to multiple single molecules named by the ordered numbers:

```
#> molaical.exe -tool mol2 -w split -n 1 -v true -m number -i "D:\split\all.mol2" -o "D:\split\1"
```

5.3.5. Molecular format conversion

5.3.5.1. Molecule to SMILES

5.3.5.1.1. Change Mol2 to SMILES

This function serves as a utility to convert molecular structures from the Mol2 format to the SMILES format, specifically tailored for de novo drug design applications. The resulting SMILES strings are formatted to be fully compatible with the filtering capabilities of the MolAICal software, ensuring seamless integration into the drug design workflow. For instance, it can generate SMILES strings from mol2 format for filtering unwanted molecules (see folder: MolAICal-xxx/filterMols). This function will continue to be developed. The parameters in MolAICal are shown below:

-tool: appoint the tool function. Here, it is "mol2tosmi".
-i: path of molecular input file with mol2 format.

The example commands are as below:
1) command example of mol2 to SMILES
#> molaical.exe -tool mol2tosmi -i "D:/mol2tosmi/zinc_98180786.mol2"

This is just a format conversion way for one molecule. If users want to do molecular format conversion in batch, users can modify the below Linux bash shell code according to the specific requirements (For example, the files contain the suffix .mol2):

```
#!/bin/bash
for i in `ls -l | grep ".mol2" | awk '{print $9}'`
do
        # Above ".mol2" is a keyword. The below command is to remove the suffix.
        tmp=$(echo $i | awk -F'.' '{print $1}')
        echo $tmp
        molaical.exe -tool mol2tosmi -i  $i > $tmp'.smi'
        # Remove the first two lines
        sed -i '1,2d' $tmp'.smi'
        # Remove the last line
        sed -i -e '$d' $tmp'.smi'
done
```

5.3.5.1.2. Convert various formats to canonical SMILES

For the purposes of calculating MCE-18, SA, and MCFs, chemical file formats such as mol2, sdf, mol, etc., often need to be converted into SMILES notation. In this context, MolAICal offers an

RDKIT-based method for converting molecular structures from formats including mol2, sdf, mol, pdb, inchi, and SMILES into their canonical SMILES representations. It means that this feature enables the conversion of molecular formats such as .mol2, .sdf, .mol, .pdb, InChI, SMILES to canonical SMILES. The parameters are as follows:

-tool <arg> appoint the tool function. Here, it is “tosmi”.
-i <arg> Input file or string.
-q <arg> values: true and false, “true” means to suppress conversion failure messages.
-r <arg> values: true and false, “true” means to use raw mode with no sanitization.
-t <arg> Input type (mol2, sdf, mol, pdb, inchi, smiles).
-u <arg> values: true and false, “true” means to use unsafe mode for problematic SMILES.

The example commands are as below:

1) Print help information
#> molaical.exe -tool tosmi -h

2) Mol2 to canonical SMILES
#> molaical.exe -tool tosmi -i F:/data/new/mm/rdkit_in_out/lig.mol2 -t mol2

3) Mol2 to canonical SMILES without conversion failure messages
#> molaical.exe -tool tosmi -i F:/data/new/mm/rdkit_in_out/lig.mol2 -t mol2 -q true

4) Mol2 to canonical SMILES with unsafe mode for problematic SMILES and without conversion failure messages, sanitization
#> molaical.exe -tool tosmi -i F:/data/new/mm/rdkit_in_out/lig.mol2 -t mol2 -q true -u true -r true

5) SMILES to canonical SMILES
#> molaical.exe -tool tosmi -i "CCO" -t smiles

6) InChI to canonical SMILES
#> molaical.exe -tool tosmi -i "InChI=1S/C2H6O/c1-2-3/h3H,2H2,1H3" -t inchi

7) PDB to canonical SMILES
#> molaical.exe -tool tosmi -i F:/data/new/mm/rdkit_in_out/lig.pdb -t pdb

8) mol to canonical SMILES
#> molaical.exe -tool tosmi -i F:/data/new/mm/rdkit_in_out/lig.mol -t mol

9) SDF to canonical SMILES
#> molaical.exe -tool tosmi -i F:/data/new/mm/rdkit_in_out/lig.sdf -t sdf

Note: “-q, -u, or -r” can also enable the conversion of molecular formats such as .mol2, .sdf, .mol, .pdb, InChI, SMILES to canonical SMILES

5.3.5.2. Conversion of different molecular formats

This function, which is derived from Open Babel[28] can be used to convert different molecular formats such as pdb-mol2, sdf-pdb, sdf-mol2, etc. If you want to convert pdb to mol2 format, you must have the correct suffix. For example, if the input file is a PDB format file, it must have the suffix .pdb. If the output file is an SDF format file, it must have the suffix .sdf. The parameters of molecular format conversion are as follows:

-tool: appoint the tool function. Here, it is "format".
-i: input file with correct suffix.
-o: output file with correct suffix.
-c: select parameters for dealing with molecules. Currently, it can be "none", "gen3d", "delh", and "addh". The "none" is the default value, which means molecular format conversion without 3D coordinates generation. The "gen3d" means molecular format conversion with 3D coordinates generation. The "delh" means molecular format conversion with the deletion of hydrogen. The "addh" means molecular format conversion with adding hydrogen.

The example commands are as below:

1) PDB to Mol2. It must have a suffix .pdb for input and a suffix .mol2 for output.
#> molaical.exe -tool format -i E:/ligand.pdb -o E:/ligand.mol2

2) SDF to pdb. It must have a suffix .sdf for input and a suffix .pdb for output.
#> molaical.exe -tool format -i "E:/ligand.sdf" -o "E:/ligand.pdb"
Note: This is just an example; users can try more molecular format conversions.

3) Convert 2D molecule in 3D coordinates
#> molaical.exe -tool format -i E:/2D.pdb -o E:/3D.mol2 -c gen3d

4) Add hydrogen on molecule
#> molaical.exe -tool format -i E:/4.mol2 -o E:/4h.mol2 -c addh

5) Delete hydrogen on molecule
#> molaical.exe -tool format -i E:/ligand.pdb -o E:/delH.pdb -c delh

5.3.5.3. Batch format conversion of molecules and VS run-flat

In this part, MolAICal supplies a script for converting the files of receptors and ligands in batches. The parameters in MolAICal for batch format conversion of receptor and ligand files are as follows:

-ir: The file of the receptors list.
-or: Output names of receptors.
-irf: The format for inputted receptor. The format name is the suffix of the molecule, for example, “test.mol”, “test.pdb”, or “test.sdf” should use the letters “mol”, “pdb” or “sdf” after the parameter “**-irf**”.
-orf: The format for outputted receptor. The format name is the suffix of the molecule, for example, “test.mol”, “test.pdb”, or “test.sdf” should use the letters “mol”, “pdb” or “sdf” after the parameter “**-orf**”.
-il: The file of ligand list.
-ol: In the section of continuing virtual screening when using “-cvs”, it is a folder for storing. Otherwise, output the names of ligands.
-ilf: The format of the input ligand.
-olf: The format of the output ligand.
-rc: Whether to convert receptor format, if input “-rc” only, it means True to convert receptor format.
-lc: Whether to convert ligand format, if input “-lc” only, it means True to convert ligand format.
-lc3: Whether convert 3D ligand format, if input “-lc3” only, it means True to convert 3D ligand format.
-ds: Whether search files with appointed characters, if input “-ds” only, it means True to search files with appointed characters.
-sd: The directory for searching.
-ik: The keyword is for searching.
-nk: The keyword is excluded from the search.
-fn: The file contains different keywords for searching.
-dp: In the section of continuing virtual screening when using “-cvs”, it is the stored number of the screened molecules in every folder. The default value is “None,” which means to store all screened molecules in one folder. It can be 1, 2, or 3… which represents the number of molecules stored in every folder.
Otherwise, it represents the depth of searching directories; the default value is “None,” which represents searching all the directories. It can be 0, 1, or 2… that represents the current, 1, or 2…level directories.
-mp: The absolute path of the MolAICal executable file.
-cp: The absolute path of the UCSF Chimera executable file.
-m: Whether to run the MolAICal command for molecular format conversion, if input “-m” only, it means True to run the MolAICal command for molecular format conversion.
-dr: Whether to run MolAICAl for dealing with the receptor files. For example, delete ions, and water, and modify hydrogen. If input “-dr” only, it means True to run MolAICAl for dealing with the receptor files.
-rn: The writing receptor name.
-ah: Whether to delete hydrogen. The value is “none”, “add”, “dah”, or “del”:
“none” is no action;
“add” means to add hydrogen;
“dah” means to add hydrogen after deleting hydrogen;
“del” means to delete hydrogen; it is the default value.
-cvs: Whether to prepare for continuing to run virtual screening. If input “-cvs” only, it means True

to prepare for continuing to run virtual screening jobs.

This part needs the virtual screening console; please see “Part III. Virtual Environment Interface” for entering the virtual screening console. Once going into the virtual screening console, it can follow the below steps for converting in batch:

1) Getting ligands’ list
#> python D:/MolAICal-xxx/scripts/molaical_batch.py -sd "batch_conv" -ds -ik "_ligand.sdf" > lig_list.dat

2) Getting receptors’ list
#> python D:/MolAICal-xxx/scripts/molaical_batch.py -sd "batch_conv" -ds -ik "_protein.pdb" -nk "_protein.pdbqt" > rec_list.dat

3) Getting the receptors’ list without the appointed characters
#> python D:/MolAICal-xxx/scripts/molaical_batch.py -sd "batch_conv" -ds -ik "_protein.pdb" -nk "_protein.pdbqt" > rec_list.dat

4) Getting the receptors’ list without the appointed characters and with different keywords in a file that can be merged with the previous keyword after "-ik".
#> python D:/MolAICal-xxx/scripts/molaical_batch.py -sd "batch_conv" -ds -ik "_protein.pdb" -nk "_protein.pdbqt" -fn "list_name"

Sometimes, receptors contain unnecessary ions and water or the wrong hydrogen. MolAICal can be used to deal with receptor files in batches as follows:

5) Handle receptor files without hydrogen
#> python D:/MolAICal-xxx/scripts/molaical_batch.py -m -dr -cp "D:/Chimera 1.15rc/bin/chimera.exe" -ir "rec_list.dat"

6) Handle receptor files with adding hydrogen
#> python D:/MolAICal-xxx/scripts/molaical_batch.py -m -ah add -dr -cp "D:/Chimera 1.15rc/bin/chimera.exe" -ir "rec_list.dat"

7) Handle receptor files without hydrogen and assign the suffix name of receptor files.
#> python D:/MolAICal-xxx/scripts/molaical_batch.py -m -dr -rn "_noh.pdb" -cp "D:/Chimera 1.15rc/bin/chimera.exe" -ir "rec_list.dat"

8) Handle receptor files with adding hydrogen and assign the suffix name of receptor files.
#> python D:/MolAICal-xxx/scripts/molaical_batch.py -m -ah add -dr -rn "_noh.pdb" -cp "D:/Chimera 1.15rc/bin/chimera.exe" -ir "rec_list.dat"

9) Converting in the batch of receptor files with the prefixes of receptor names
#> python D:/MolAICal-xxx/scripts/molaical_batch.py -rc -ir "rec_list.dat" -irf "pdb" -orf "pdbqt"

10) Converting in the batch of receptor files with the assigned names
#> python D:/MolAICal-xxx/scripts/molaical_batch.py -rc -ir "rec_list.dat" -irf "pdb" -orf "pdbqt" -or "rec_t.pdbqt"

11) Converting in the batch of ligand files with the prefixes of ligand names
#> python D:/MolAICal-xxx/scripts/molaical_batch.py -lc -il "lig_list.dat" -ilf "sdf" -olf "pdbqt"

12) Converting in the batch of ligand files with the assigned names
#> python D:/MolAICal-xxx/scripts/molaical_batch.py -lc -il "lig_list.dat" -ilf "sdf" -olf "pdbqt" -ol "lig_t.pdbqt"

Besides, the way of **batch format conversion of files of receptors and ligands by MolAICal command** is introduced as follows:

13) Converting in the batch of receptor files with the MolAICal command
#> python D:/MolAICal-xxx/scripts/molaical_batch.py -m -rc -ir "rec_list.dat" -mp "D:/MolAICal-xxx/molaical.exe"

14) Converting the batch of ligand files with the MolAICal command
The MolAICal command does not support the SDF format, but it supports the mol2 or PDB format for conversion. Here, the mol2 format is selected as an example:

#> python D:/MolAICal-xxx/scripts/molaical_batch.py -sd "batch_conv" -ds -ik "_ligand.mol2" > lig_list_mol2.dat

Then, converting in batch of ligand files:

#> python D:/MolAICal-xxx/scripts/molaical_batch.py -m -lc -il "lig_list_mol2.dat" -mp "D:/MolAICal-xxx/molaical.exe"

Note: Please replace “MolAICal-xxx” or “Chimera 1.15rc” with your real path of MolAICal or UCSF Chimera. “lig_list.dat”, “lig_list_mol2.dat”, and “list_name” should be replaced with the real path in the users' OS system. It should set your real path for "batch_conv" after “-sd”.

15) Convert 3D ligands without any operation for hydrogens
#> python D:/MolAICal-xxx/scripts/molaical_batch.py -m -lc3 -ah none -il "lig_list_mol2.dat" -olf "pdb" -mp -mp "D:/MolAICal-xxx/molaical.exe"

16) Convert 3D ligands with deleting hydrogens
#> python D:/MolAICal-xxx/scripts/molaical_batch.py -m -lc3 -ah del -il "lig_list_mol2.dat" -olf pdb -mp "D:/MolAICal-xxx/molaical.exe"

17) Convert 3D ligands with adding hydrogens

#> python D:/MolAICal-xxx/scripts/molaical_batch.py -m -lc3 -ah add -il "lig_list_mol2.dat" -olf pdb -mp "D:/MolAICal-xxx/molaical.exe"

18) Convert 3D ligands with adding hydrogen after deleting hydrogen
#> python D:/MolAICal-xxx/scripts/molaical_batch.py -m -lc3 -ah dah -il "lig_list_mol2.dat" -olf pdb -mp "D:/MolAICal-xxx/molaical.exe"

This script also supplies the function for the continuing run of virtual screening (VS), the main idea for VS run-flat is to move the screened molecules into new folders after obtaining the list of ligand paths "out_list.dat**" by searching the keywords of the screened molecules (see above commands):**
19) Move results into one folder
#> python D:/MolAICal-xxx/scripts/molaical_batch.py -cvs -il F:\out_list.dat -ol F:\target

20) Move results into the folders every appointed number
#> python D:/MolAICal-xxx/scripts/molaical_batch.py -cvs -dp 1 -il F:\out_list.dat -ol F:\target

If the screened molecules are moved into the new place, the remaining molecules can continue to run in the VS way of MolAICal.

5.3.6. Split and extract PDBQT file

The docking result file of MolAICalD may contain multiple docking poses. The MolAICal software supplies a function for calculating the total number of poses in the docking result file, splitting the molecular set into multiple molecular files with PDBQT format, and extracting the appointed docking pose in the docking result file. The parameters in MolAICal are shown below:

-tool: appoint the tool function. Here, it is "pdbqt".
-t: appointed string for splitting the docking result file. The default value is "MODEL".
-n: it has three integer values: 0, -1, >0. "0" means 0 means split docking result file one by one; "-1" means calculate the total number of poses in the docking result file; ">0" means extracting an appointed molecule from the docking result file in order.
-v: whether to overlap the existing molecular file. The default value is true for overlap.
-i: path of input molecular PDBQT file.
-o: path of output directory.

The example commands are as follows:

1) Split molecular set in PDBQT format one by one:
#> molaical.exe -tool pdbqt -i "E:\workdir\all.pdbqt" -o "E:\workdir\1"

2) Full command for splitting molecular set in PDBQT format one by one:
#> molaical.exe -tool pdbqt -t "MODEL" -v true -n 0 -i "E:\workdir\all.pdbqt" -o "E:\workdir\1"

3) Census the number of molecular set
#> molaical.exe -tool pdbqt -n -1 -i "E:\workdir\all.pdbqt"

4) Extract an appointed molecule from the docking result file in order, e.g., obtain the second molecule:
#> molaical.exe -tool pdbqt -n 2 -i "E:\workdir\all.pdbqt" -o "E:\workdir\1"

5.4. Split molecules into fragments

It will get useful information on drug design by splitting the appointed molecules into fragments. For example, the fragments that are split from FDA-approved drugs can give effective small chips for *de novo* drug design based on fragment growth. In addition, it can find the important rules for drug discovery via analysis of fragment features of the appointed molecule set, such as natural compounds, marine drugs, and so on. Here, MolAICal supplied one way to split one molecule into fragments by the rotation bond. The parameters in MolAICal are shown below:

-tool: appoint the tool function. Here, it is “fragSplit”.
-i: path of molecular list file.
-o: path of output fragments.
-oa: path of output fragments with adding hydrogen.
-w: method for dealing with hydrogen. It contains “add”, “del”, and “off”. The “add” means the split fragments will add hydrogen and copy it into the “-oa” appointed directory. The “del” means the split fragments will delete hydrogen and copy it into the “-oa” appointed directory. The “off” means the split fragments will not perform any action and copy them into the “-oa” appointed directory. The default value is “off”.

The example commands are as follows:

1) Split molecules into fragments and do not perform any hydrogen operation
#> molaical.exe -tool fragSplit -i "D:\\fragment\\list.dat" -o "D:\\fragment\\1" -oa "D:\\fragment\\3"

2) Split molecules into fragments and do not perform any hydrogen operation
#> molaical.exe -tool fragSplit -i "D:\\fragment\\list.dat" -o "D:\\fragment\\1" -oa "D:\\fragment\\3" -w off

3) Split molecules into fragments with adding hydrogen
#> molaical.exe -tool fragSplit -i "D:\\fragment\\list.dat" -o "D:\\fragment\\1" -oa "D:\\fragment\\3" -w add

4) Split molecules into fragments with deleting hydrogen
#> molaical.exe -tool fragSplit -i "D:\\fragment\\list.dat" -o "D:\\fragment\\1" -oa "D:\\fragment\\3" -w del

6. Call external programs

6.1. Set path environment for external programs

For certain research tasks, MolAICal may require calling external programs. Manually entering the program path for each command invocation would be cumbersome. To streamline this process, MolAICal provides a **persistent configuration feature**: **users can define the program name and path once, which remains effective until MolAICal is uninstalled, undergoes major updates, or the configuration is explicitly removed**. Incorrect external program paths/names can be corrected anytime. Previous settings will be overwritten by new inputs by default.

usage: set

-h	Print help information of the set environment function.
-n <arg>	The name of the program.
-o <arg>	Print the environment variable and path information. The value is 'all' and the assigned characters for retrieval. 'all' means to print all set environment variables and paths in MolAICal; the assigned characters are for retrieval. For example, if the assigned characters are 'vmd', MolAICal will search the environment variable and the path of 'vmd'.
-p <arg>	The path of the program.

The example commands are as follows:

1) Print help information
#> molaical.exe -call set -h

2) Set the name and path of the program (e.g. VMD)
#> molaical.exe -call set -n VMD -p "C:\Program Files (x86)\University of Illinois\VMD\vmd.exe"

3) Print the environment variable and the path of "vmd"
#> molaical.exe -call set -o vmd

4) Print all set environment variables and paths in MolAICal
#> molaical.exe -call set -o all

5) Clean environment variables and paths in MolAICal
#> molaical.exe -call set -o clean

6.2. Call external programs by MolAICal

In certain scenarios, **repeatedly entering external program paths and switching between MolAICal and other external applications can make operations cumbersome.** To address this issue, MolAICal provides a method to configure the names and paths of external programs, allowing users to set them once for persistent use. For example:

```
#> molaical.exe -call set -n VMD -p "C:\Program Files (x86)\University of Illinois\VMD\vmd.exe"
```

This enables MolAICal to execute command-line operations with a single call. Currently, MolAICal does not support multiple interactive inputs from a single command. When an external program enters a suspended interactive state, the current process can be terminated by pressing 'Ctrl + C' on the keyboard. This applies both when program execution is no longer required or when initiating new command operations. If multiple command executions are required, users can write a script and then invoke the external program via MolAICal to run it. For instance, the below command can be used to call VMD to execute the "pbcwrap.vmd" script. The parameters in MolAICal are shown below:

usage: run

-h	Print help information.
-call	Interact with external programs or commands. Its value can be **'set'** or **'run'**. **'set'** means to set the environment of external programs, including name and path. **'run'** means to call the external programs, which can search the path of the program from the set environment file.
-i	The command arguments of the program. It includes **all command-line arguments** of **external programs**, but the '-i' parameter must be the last one specified, while all other identifiers (e.g., '-p', etc.) of MolAICal must come **before '-i'.**
-n <arg>	It can be: 1) the name of the executed program; 2) the path of the input script file that can be carried out by MolAICal.
-o <arg>	Its function is: 1) whether to print the output of the program; 2) whether to replace the executed path of MolAICal in the script file. The value can be (true/false), default: true.
-p <arg>	The path of the program. The MolAICal environment setup command can be used to configure it once, eliminating the need for tedious path input later.
-w <arg>	Whether to carry out the task (true/false), default: true.
-c<arg>	the calculation way: ◆ It will be used to deal with the DCD file if its value is "**stripDCD**"; ◆ It will perform entropy calculation if its value is "**entropy**"; ◆ It will run the VMD command once in the Tk Console if its value is "**vmdargs**"; ◆ It will run the scripts (VMD, Python, bash, etc) of MolAICal if its value is "**sfile**"; ◆ It will merge the PDB and PSF files if its value is "**merge**"; ◆ It will handle the DCD file, including printing the number of frames and extracting the DCD file with the assigned frames if its value is "**catdcd**"; ◆ It will run NAMD if its value is "**runnamd**";

- MolAICal will run the script (commands' list running in Windows DOS or Linux command console) if its value is "**script**";
- Autopsf of VMD now has a bug that can be checked and fixed if its value is "**fixpsf**". The default force field files in the MolAICal package contain: "par_all36_cgenff.prm", "par_all36_lipid.prm", "par_all36_na.prm", "par_all36_prot.prm", "toppar_water_ions.str", "top_all36_cgenff.rtf", "top_all36_lipid.rtf", "top_all36_na.rtf", and "top_all36_prot.rtf".
- It will run pocket detection by calling "p2rank" if its value is "**pocket**".
- It will run pocket detection by calling "fpocket" if its value is "**fpocket**".
- It will run molecular docking (protein-protein, protein-peptide, protein-DNA or protein-RNA) by calling "LightDock" if its value is "**ldock**".
- It will run the DOS command in Windows or run the shell bash command in Linux if its value is "**ecommand**".
- It will generate the peptides if its value is "**pepgen**".

The example commands are as follows:

(1) Print help information

#> molaical.exe -call run -h

(2) If users have set the environment variable, it can be run as follows (Here, VMD is selected as an example):

```
#> molaical.exe -call set -n VMD -p "C:\Program Files (x86)\University of Illinois\VMD\vmd.exe"
```

Then, run the arguments of the external program:

```
#> molaical.exe -call run -n vmd -i -dispdev text -e pbcwrap.vmd
```

Note: In MolAICal's environment variable configuration, the parameter name following '-n' (e.g., '-n VMD') must match the corresponding runtime parameter (e.g., '-n vmd') in terms of spelling, though case sensitivity is not enforced.

"pbcwrap.vmd" is shown as an example below:

```
package require pbctools
mol new mpro.psf
mol addfile mpro.dcd waitfor all
pbc wrap -centersel protein -center com -compound chain -all
set all [atomselect top all]
animate write dcd mpro.dcd sel $all beg 0 end 24 waitfor all
quit
```

(3) If users have not set the environment variable, it should assign the path of the external program as follows (Here, VMD is selected as an example):

```
#> molaical.exe -call run -n vmd -p "c:\Program Files (x86)\University of Illinois\VMD\vmd.exe" -i -dispdev
```

```
text -e pbcwrap.vmd
```

Notice: The '-call run' interface command provided by MolAICal is not limited to VMD; it is generally applicable to any external program that can be executed via the command-line console.

(4) If the value of "-c" is "stripDCD", it will deal with the NAMD DCD file by calling the inner script of MolAICal.

```
#> molaical.exe -call run -n vmd -c stripDCD -i -dispdev text -psf "mpro.psf" -args protein,or,resname,LIG "mpro.dcd" "complex" mpro.psf mpro.pdb
```

Note: because **'-args'** is after '-i', the **'-args'** belongs to the VMD argument, which is the usage like the command “atomselect” in the Tk Console of VMD software, such as "atomselect top protein or resname LIG". Here, the comma "," represents a blank space " ". The string "complex" is the prefix of the output files, which can be modified to the desired name. Here, it will generate like "complex.psf", "complex.pdb", and "complex.dcd". The mpro.psf and mpro.pdb are referred to generate the files that have the prefix "complex". The default value of **'-n'** will use "vmd" if **'-n'** is not set.

(5) Autopsf of VMD now has a bug that can be checked and fixed if the value of "-c" is "fixpsf". Fix the "autopsf" function in VMD.

```
#> molaical.exe -call run -c fixpsf
```

(6) If the value of "-c" is "vmdargs", MolAICal will run the VMD command once in the Tk console.

```
#> molaical.exe -call run -c vmdargs -i -pdb ligand.pdb -dispdev text -args autopsf autopsf -mol 0 -top ligand.str
```

```
#> molaical.exe -call run -c vmdargs -i -pdb ligand.pdb -args autopsf autopsf -mol 0 -top ligand.str
```

```
#> molaical.exe -call run -c vmdargs -i -pdb P_peptide.pdb -dispdev text -args none set sel [atomselect top all], \\\$sel set chain B, \\\$sel writepdb P_peptide_B.pdb
```

```
#> molaical.exe -call run -c vmdargs -i -pdb P_peptide.pdb -args none set sel [atomselect top all], \\\$sel set chain B, \\\$sel writepdb P_peptide_B.pdb
```

Note: If running VMD "**autopsf**", it should fix the topology bug of VMD by MolAICal. Please see the above related introduction. The arguments after '-i' are the VMD parameters. The detailed information of the arguments is as follows:

'-pdb': means to load the molecule in PDB format.

'-dispdev text': means to load VMD in text mode. It is the default parameter and does not need to be entered.

'-args': pass subsequent arguments to the text interpreter to run the VMD command once in the Tk console:

- The first parameter after **'-args'** can be a package name of VMD or **'none'**. If the first parameter is **'none'** after **'-args'**, it will omit the package for loading. For example, it will run the command "package require **autopsf**" in the Tk console of VMD if the first parameter is **'autopsf'** after **'-args'**. It will not run the command "package" if the first

parameter is **'none'** after **'-args'**.

- The string following **'-args'** contains one command that can be executed in VMD's Tk console before each comma **','**; in other words, the comma character replaces the process of entering each command in VMD's Tk console and pressing the Enter key. This allows multiple lines of commands to be executed with just a single command.
- Some special characters, such as the character **"$"**, require the use of the escape character "\". For special characters, MolAICal uses three escape characters "\" (i.e., "\\\").
- The second parameter after **'-args'** is a command after loading the corresponding package. And the following strings are the arguments of this command. For example, "**autopsf** autopsf -mol 0 -top ligand.str"
- Autopsf defaults to load the topology files containing "top_all36_prot.rtf", "top_all36_lipid.rtf", "top_all36_na.rtf", "top_all36_carb.rtf", "top_all36_cgenff.rtf", "toppar_all36_carb_glycopeptide.str", "toppar_water_ions_namd.str". Therefore, users can refer to these topology files. If there's no need to introduce special topology files, they can avoid the hassle of inputting "-top".

✧ Users can check the help information for VMD commands as follows:

```
#> molaical.exe -call run -c vmdargs -i -dispdev text --help
```

```
#> molaical.exe -call run -c vmdargs -i --help
```

It will show:

```
Info)    -dispdev <win | cave | text | none> Specify display device
Info)    -dist <d>                 Distance from origin to screen
Info)    -e <filename>             Execute commands in <filename>
Info)    -python                   Use Python for -e file and subsequent text input
Info)    -eofexit                  Exit when end-of-file occurs on input
Info)    -h | --help               Display this command-line summary
Info)    -pos <X> <Y>              Lower-left corner position of display
Info)    -nt                       No title display at start
Info)    -size <X> <Y>             Size of display
Info)    -startup <filename> Specify startup script file
Info)    -m                        Load subsequent files as separate molecules
Info)    -f                        Load subsequent files into the same molecule
Info)    <filename>                Load file using best-guess file type
Info)    -<type> <filename>    Load file using specified file type
Info)    -args                     Pass subsequent arguments to text interpreter
```

✧ Users can check the help information of "**autopsf**" of VMD as follows:

```
#> molaical.exe -call run -c vmdargs -i -dispdev text -args autopsf autopsf
```

It will show:

```
autopsf
Usage: autopsf -mol <molnumber> <option1> <option2>...
```

```
Options:
  Switches:
    -protein (only generate psf for protein)
    -nucleic (only generate psf for nucleic acid segment)
    -solvate (run solvate on structure after psf generation)
    -ionize (run autoionize on structure after psf generation)
    -noguess (don't use guesscoord during psf building)
    -none (it will add 'first none' and 'last none' in the read pdb part. It is a new fixed function in MolAICal.)
    -noterm (don't automatically add terminii to proteins)
    -rotinc <increment> (degree increment for rotation)
    -gui (force graphical interface mode)
    -regen (regenerate angles/dihedrals)
    -splitonly (split chains, but don't build structure)
  Single option arguments:
    -mol <molecule> (REQUIRED: Run on molid <molecule>)
    -prefix <prefix> (Use <prefix> as the prefix to output files)
    -top <top> (Use topology file <top>)
    -include <selection> (Include fragment specified by <selection> in psf generation
    -patch <patch> (Apply a patch -- see docs for syntax)
```

Users can use the MolAICal command to build PDB and PSF files for their research purposes.

(7) If the value of "-c" is "sfile", the script file of MolAICal will be chosen to be invoked according to the specific name of the script file.

The arguments after '-args' belong to the VMD. For the sake of concise expression, the terms 1st, 2nd, 3rd, and 4th below represent the first argument, the second argument, the third argument, and the fourth argument, respectively, and so on.

'-pdb': means to load the molecule in PDB format.

'-args': pass subsequent arguments to the text interpreter to run the VMD script file of MolAICal.

- "get_res.tcl": This file is designed to get the relevant restraint residues for LightDock.
  - **1st:** The chain name of the selected molecule;
  - **2nd:** The chain name of reference molecule used for selection;
  - **3rd:** The residues of the selected molecule within the specified distance from the reference molecule will be selected;
  - **4th:** The saved filename;
  - **5th:** Specifies the molecular type, **'R'** stands for Receptor and **'L'** stands for Ligand.

  The example is as follows:

```
#> molaical.exe -call run -c sfile -i get_res.tcl -pdb 7LLL.pdb -args P R 3.5 lig.dat L
```

- "align_axis.tcl": This file is designed to align molecules along a specific axis. Its primary application is to orient the transmembrane regions of membrane proteins along the z-axis, which facilitates research. If the membrane protein is already aligned along the z-axis, this script is not necessary.
  - **1st:** Atom selection (same as the command "atomselect" in the Tk console of VMD). If there is one more than a letter (e.g., protein and chain B), it will use the comma "," to

represent the blank space " " (e.g., protein,and,chain,B);

- **2nd:** If the protein aligns horizontally (lying flat), change 0 to 2;
- **3rd:** The output file name;
- **4th:** The selected axis for "transaxis" of VMD, which can be x, y, z, or "". "" can be equal to no character, including space for inputting.

The examples are as follows:

```
#> molaical.exe -call run -c sfile -i align_axis.tcl -pdb R_protein.pdb -args protein 2 new.pdb
#> molaical.exe -call run -c sfile -i align_axis.tcl -pdb R_protein.pdb -args protein 2 new.pdb x
```

- "gen_mem.tcl": This script generates hypothetical topmost and bottommost boundary membranes for LightDock molecular docking. It prevents glowworm generation within the membrane-defined region, where glowworms determine docking positions, translations, rotations, etc. This mechanism primarily avoids placing glowworms in unsuitable locations (e.g., within the membrane interior), which would result in invalid docking. The vertical positions of these two topmost and bottommost boundary membrane layers (parameters **2nd** and **3rd**) can be numerically adjusted based on user expertise and experimental requirements.
  - **1st:** Atom selection (same as the command "atomselect" in the Tk console of VMD). If there is one more than a letter (e.g., protein and chain B), it will use the comma "," to represent the blank space " " (e.g., protein,and,chain,B);
  - **2nd:** The minimum z-axis for membrane generation;
  - **3rd:** The maximum z-axis for membrane generation;
  - **4th:** The saved output file name;
  - **5th:** The delta value is used to [($max - $delta), $max], which represents the range of values used to determine the maximum and minimum x,y values along the z-axis region. This ensures that membrane particles in the cross-section of the membrane are not located inside the membrane protein. Default value: 3.0;
  - **6th:** Safety buffer for the bounding box (set 0 to strictly use protein limits). Default value: -2.0;
  - **7th:** Membrane Padding (Angstroms). Default value: 15.0;
  - **8th:** The minimum spacing between particles. Default value: 3.5;
  - **9th:** The maximum spacing between particles. Default value: 6.0;
  - **10th:** The distance to keep away from protein atoms (Angstroms). Default value: 4.0.

The examples are as follows:

```
#> molaical.exe -call run -c sfile -i gen_mem.tcl -pdb new.pdb -args protein -29 10
#> molaical.exe -call run -c sfile -i gen_mem.tcl -pdb new.pdb -args protein -29 10 mpro.pdb 3.0 -2.0 20.0
3.5 6.0 4.0
```

- "printscore.py": Print the scores of docked results in PDBQT format by MolAICal.
  - **1st:** the docked molecules in the folder, it can use a method similar to regular expressions, such as "splitdir/*out.pdbqt", which indicates traversing all molecules in the splitdir folder that contain the string "out.pdbqt".
  - **2nd:** if it is "d", it will print the ascending order results. Default sorting: ascending.
  - **3rd:** if it is "f", it will not print the failure results. Default: print the failure results.

```
#> molaical.exe -call run -c sfile -i printScore.py "splitdir/*out.pdbqt"
```

```
#> molaical.exe -call run -c sfile -i printScore.py "splitdir/*out.pdbqt" d f
```

- "get_top_results.py": Get two top-ranked molecules and move into the assigned folder.
  - **1st:** The docked molecules in the folder. It can use a method similar to regular expressions, such as "splitdir/*out.pdbqt", which indicates traversing all molecules in the splitdir folder that contain the string "out.pdbqt";
  - **2nd:** The saved number of molecules for docked results;
  - **3rd:** The folder for the ranked and saved molecules.

```
#> molaical.exe -call run -c sfile -i molaicaldTopResults.py "splitdir/*out.pdbqt" 2 results
```

- "addhsmol2.pml": A script for open source PyMol (https://github.com/schrodinger/pymol-open-source or https://github.com/urban233/pymol-open-source-whl). It can load molecules, add hydrogens to the loaded molecules, and then save the loaded molecules in Mol2 files.
  - **1st:** The string "--" can avoid loading the parameters automatically. E.g., load the file "list.txt";
  - **2nd:** The list file that contains all the molecular names in a folder.

```
#> molaical.exe -call run -c sfile -i addhsmol2.pml -- list.txt
```

Note: other PyMol scripts can be run in the above similar way.

- "lrank.sh": It is used for conformation generation, clustering, and ranking after LightDock's setup and simulation. For more detailed usages, please check the biomolecular docking part in the MolAICal manual.
- "mmgbpbsa_batch.sh": The calculations of MM/GBSA or MM/PBSA. For more detailed usages, please check the biomolecular docking part in the MolAICal manual.
- "mrank.py": The Python script for ranking the results of LightDock, MM/GBSA, or MM/PBSA. For more detailed usages, please check the biomolecular docking part in the MolAICal manual.
- "fix_pdb.tcl": The VMD Tcl script for repairing the resname issue of nucleic acids (DNA and RNA) in the PDB format for "reduce" (https://github.com/rlabduke/reduce). This issue may be from the chain setting and save operations of VMD. For more detailed usages, please check the biomolecular docking part in the MolAICal manual.
- In the Linux version of MolAICal, for the Python script, if wanting to call the "py" environment rather than the "ldock" environment, it must have a prefix "molaical_" or "m_" such as "molaical_batch.py".
- admet_plot.py: ADMET results visualizer.

```
usage: m_admetp.py [-h] [-i INPUT] [-n NUM_ITEMS] [--merge]

optional arguments:
  -h, --help                show this help message and exit
  -i, --input INPUT         Input results.dat file
  -n, --num_items NUM_ITEMS Number of ADMET items per figure (default: auto)
  --merge                   Generate merged plots by ADMET item (one plot per item
                            across molecules)
```

➢ lmfix_rec.py: Fix PDB atom names to match DFIRE compatibility standards.

```
usage: lmfix_rec.py [-h] [-i INPUT] [-o [OUTPUT]] [-vo] [--verbose]

optional arguments:
  -h, --help                    show this help message and exit
  -i, --input INPUT             Input PDB file path
  -o, --output [OUTPUT]         Output PDB file path (optional)
  -vo, --validate-only          Only validate DFIRE compatibility without fixing
  --verbose, -v                 Show detailed correction report

Examples:
    python lmfix_rec.py -i protein.pdb
    python lmfix_rec.py -i protein.pdb -o protein_fixed.pdb
    python lmfix_rec.py -i protein.pdb --validate-only
```

Notice:

✧ If users want to run their own standard Python, PyMol, DOS, and Bash scripts directly through MolAICal, these files can be placed in the "MolAICal-xxx/scripts" folder and executed using the above similar example command: "molaical.exe -call run -c sfile -i addhsmol2.pml --list.txt". MolAICal identifies script types by file extensions:

- ◆ ".py" → Python script
- ◆ ".pml" → PyMol script
- ◆ ".sh" → Linux Bash script
- ◆ ".tcl" → TCL script in VMD
- ◆ ".bat" → Windows batch script

✧ Currently, MolAICal includes many script files. This section only lists the functions of some scripts, while the others have already been mentioned in the MolAICal tutorials or documents. Users can refer to the relevant MolAICal tutorials when using them.

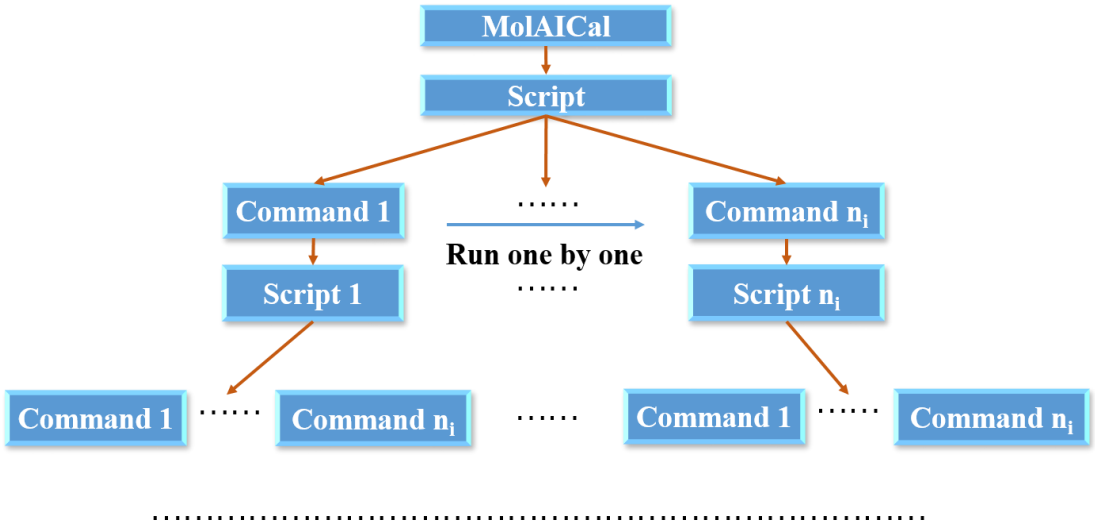
(8) MolAICal will run the script (commands' list in Windows DOS or Linux command console) if the value of "-c" is "script".

a) Principle of MolAICal Script

Part I Figure 6.2.8 provides a structured view of the MolAICal script execution process. Here's a detailed, professional description:

- ◆ MolAICal serves as the primary module, initiating the execution of the main Script.
- ◆ The Script contains sequential commands, such as Command 1, which triggers Script 1.
  - ■ Command 1 executes first, followed by Script 1.
  - ■ After Script 1 is executed, it may loop back to execute further commands, including Command 1 again or other commands like Command n_i.
- ◆ Command n_i can be a different operation within the script. It also leads to the execution of Script n_i.
- ◆ This setup demonstrates a recursive, modular process, where commands and scripts interact and trigger subsequent operations, supporting a dynamic execution flow.

The blue arrows indicate direct connections between commands and their respective scripts, and the horizontal blue arrow illustrates the flow between command executions.



Part I Figure 6.2.8 The framework of MolAICal script

b) Arguments of MoAICal Script

The strings after '-i' are the parameter list that will be split by space. The split strings will replace the strings $molargs1, $molargs2, $molargs3, etc. in the script file. The number of the string $molargs1, $molargs2, $molargs3, etc., is the same as the length of the split strings after '-i'. Some special characters, such as the comment "#" and a string space " ", are shown as below:

- If a $molargs[x]$_{x=1,2...}$ is the string "^*^", it represents comments (e.g., #).
- If a $molargs[x]$_{x=1,2...}$ is the string "^^^", it represents a string space " ".

The examples are as follows:

```
1. No modification run in the file 'runscript'
#> molaical.exe -call run -c script -n runscript

2. Modify $molargs1 in the file 'runscript' to complex.dcd
#> molaical.exe -call run -c script -n runscript -i complex.dcd

3. Modify $molargs1 and $molargs2 in the file 'runscript' to complex.dcd and complex.psf
#> molaical.exe -call run -c script -n runscript -i complex.dcd complex.psf

4. Modify $molargs1 and $molargs2 in the file 'runscript' to complex.dcd and complex.psf, and will not modify the executed path of MolAICal automatically.
#> molaical.exe -call run -c script -n runscript -o false -i complex.dcd complex.psf
```

The "runscript" can be (just an example):

```
# 1. fix autopsf
molaical.exe -call run -c fixpsf
```

```
# 2. generate psf
molaical.exe -call run -c vmdargs -i -pdb $molargs2 -dispdev text -args autopsf autopsf -mol 0 -top $molargs3 -prefix ligand

# 3. merge file
molaical.exe -call run -c merge -i -dispdev text -args -first ligand_formatted_autopsf.psf ligand_formatted_autopsf.pdb -second protein_formatted_autopsf.psf protein_formatted_autopsf.pdb -output complex_final

# 4. default to run 20 fs minimization and 40 fs MD simulation
molaical.exe -call run -c runnamd -i -dispdev text -args -s complex_final.psf -c complex_final.pdb -nf md_configure.conf -output_prefix com_md

# 5. Cut last 10 steps for MM/PBSA calculation
molaical.exe -call run -c catdcd -i -dispdev text -args -out result.dcd -first 51 -last 60 -stride 1 com_md.dcd

# 6. Calculate MM/PBSA
molaical.exe -call run -c mmpbgbsa -i -dispdev text -args -s complex_final.psf -c complex_final.pdb -t result.dcd -cs "segname,AP1,CO1" -rs "segname,AP1" -ls "segname,CO1" -pb 2 -ff $molargs3

# 7. delete file with command (in windows: del xxx, in linux: rm xxx)
rm *md.* *run.* *formatted* result.dcd *final.* *_md* *_wb* FFTW_*
```

(9) MolAICal will merge the PDB and PSF files if the value of "-c" is "merge".

```
#> molaical.exe -call run -c merge -i -dispdev text -args -first ligand.psf ligand.pdb -second protein.psf protein.pdb -output test
```

The default value of **'-n'** will use "vmd" if **'-n'** is not set.

(10) MolAICal will handle the DCD file, including printing the number of frames and extracting the DCD file with the assigned frames if the value of "-c" is "catdcd".

```
1. Extract every 2nd frame from the 1st to 25th frame of the DCD file, with the output filename as www.dcd and the input DCD file being complex.dcd. Note: Here, the "1st frame" refers to the first frame (i.e., frame 0 in some programs), where counting starts from 1 instead of 0.
#> molaical.exe -call run -c catdcd -i -dispdev text -args -out www.dcd -first 1 -last 25 -stride 2 complex.dcd

2. It has the same function as above
#> molaical.exe -call run -c catdcd -i -dispdev text -args -out www.dcd -first 1 -last 25 -stride 2 -psf complex.psf complex.dcd

3. Only output the number of frames of DCD file
#> molaical.exe -call run -c catdcd -i -dispdev text -args -num complex.dcd
```

(11) MolAICal will run NAMD if the value of "-c" is "runnamd".

MolAICal defaults to providing molecular dynamics simulations with the Generalized Born Implicit Solvent model, where parameters can be directly modified via command-line arguments as following parameter introduction such as "-system_topology" and "-system_coords". Additionally, MolAICal enables custom NAMD input files through the "-namd_config_file" parameter to accommodate specialized simulation requirements. The parameter correspondence is as follows:

Usage: generate_run_namd -system_topology top_file [-args...]

Mandatory arguments:

-system_topology or -s <topology filename>	-- corresponds to 'structure' in run.namd

Optional arguments (all correspond to run.namd variables):

-system_coords or -c <coordinates filename>	-- corresponds to 'coordinates'
-temperature <temperature in K>	-- corresponds to 'set temperature' (default: 310)
-seed <random number seed>	-- corresponds to 'seed for producing the same results' (default: 12345)
-outputname <output name>	-- corresponds to 'set outputname' (default: hth-eq)
-stepspercycle <timesteps per cycle>	-- corresponds to 'stepspercycle' (default: 10)
-firsttimestep <first timestep>	-- corresponds to 'firsttimestep' (default: 0)
-paraTypeCharmm <on/off>	-- corresponds to 'paraTypeCharmm' (default: on)
-force_parameters or -ff <parameter files>	-- corresponds to 'parameters' lines (can be used multiple times)
-mergeCrossterms <yes/no>	-- corresponds to 'mergeCrossterms' (default: yes)
-exclude <exclusion type>	-- corresponds to 'exclude' (default: scaled1-4)
-scaling_1_4 <scaling factor>	-- corresponds to '1-4scaling' (default: 1.0)
-GBIS <on/off>	-- corresponds to 'GBIS' (default: on)
-ionConcentration <concentration>	-- corresponds to 'ionConcentration' (default: 0.15)
-sasa <on/off>	-- corresponds to 'sasa' (default: on)
-surfaceTension <tension>	-- corresponds to 'surfaceTension' (default: 0.0072)
-switching <on/off>	-- corresponds to 'switching' (default: on)
-switchdist <distance>	-- corresponds to 'switchdist' (default: 15.0)
-cutoff <distance>	-- corresponds to 'cutoff' (default: 16.0)
-timestep <timestep>	-- corresponds to 'timestep' (default: 2)
-rigidBonds <all/none>	-- corresponds to 'rigidBonds' (default: all)
-nonbondedFreq <frequency>	-- corresponds to 'nonbondedFreq' (default: 2)
-fullElectFrequency <frequency>	-- corresponds to 'fullElectFrequency' (default: 4)
-langevin <on/off>	-- corresponds to 'langevin' (default: on)
-langevinDamping <damping>	-- corresponds to 'langevinDamping' (default: 5.0)
-langevinTemp <temperature>	-- corresponds to 'langevinTemp' (default: $temperature)
-langevinHydrogen <on/off>	-- corresponds to 'langevinHydrogen' (default: off)
-restartfreq <frequency>	-- corresponds to 'restartfreq' (default: 20)
-dcdfreq <frequency>	-- corresponds to 'dcdfreq' (default: 20)
-outputEnergies <frequency>	-- corresponds to 'outputEnergies' (default: 20)
-outputPressure <frequency>	-- corresponds to 'outputPressure' (default: 20)
-minimize <steps>	-- corresponds to 'minimize' (default: 30)
-run_steps <steps>	-- corresponds to 'run' command (default: 40)

-if_run <0/1>	-- corresponds to 'set if_run' (default: 0. 1 means: not run '-run_steps'; 0 means: run '-run_steps')
-namd_config_file or -nf <config filename>	-- use existing NAMD config file instead which can be customized.
-namd_executable or -rn <path to NAMD>	-- NAMD executable (default: namd2)
-namd_cores <number of cores>	-- number of cores (default: 1)
-box_center <Whether to set box>	-- Whether to set the box size and center (default: "")
-help <help information>	-- print this help message.

Notice:

- ✧ The default force field files in MolAICal contain: "par_all36_cgenff.prm", "par_all36_lipid.prm", "par_all36_na.prm", "par_all36_prot.prm", "toppar_water_ions.str", "top_all36_cgenff.rtf", "top_all36_lipid.rtf", "top_all36_na.rtf", and "top_all36_prot.rtf".
- ✧ When calling to **run NAMD functionality** (the value of "-c" is "runnamd"), and setting environment variables using MolAICal (e.g., #> molaical.exe -call set -n namd -p "path"), the variable names corresponding to '-n' are fixed: 'namd' (case insensitive). Otherwise, MolAICal will not be able to find the default NAMD path if the NAMD path is not manually specified either. For example:

```
#> molaical.exe -call set -n namd -p "E:\workdir\namd2\namd2.exe"
```

The example commands are as follows:

1) Print help information

#> molaical.exe -call run -c runnamd -i -dispdev text -args -help

2) Giving the path of NAMD and the default force field files for NAMD running

#> molaical.exe -call run -c runnamd -i -dispdev text -args -system_topology complex.psf -system_coords complex.pdb -namd_executable E:/soft/namd2.exe -ff par_all36_prot.prm -ff par_all36_cgenff.prm -ff toppar_water_ions.str -ff ligand.str

3) Using the default path of "extra_path" for NAMD in MolAICal and the default force field files for NAMD running

#> molaical.exe -call run -c runnamd -i -dispdev text -args -system_topology complex.psf -system_coords complex.pdb -ff par_all36_prot.prm -ff par_all36_cgenff.prm -ff toppar_water_ions.str -ff ligand.str

4) Using the default path of "extra_path" for NAMD in MolAICal and loading force field files for NAMD running manually when **"-w"** is false. Here, **"**-if_run**"** is set to 0, it will run molecular dynamics (MD) simulations, default running time is 60 ns.

#> molaical.exe -call run -n vmd -w false -c runnamd -i -dispdev text -args -s complex.psf -c complex.pdb -if_run 0 -ff par_all36_prot.prm -ff par_all36_cgenff.prm -ff toppar_water_ions.str -ff ligand.str

5) Using the default path of "extra_path" for NAMD in MolAICal and the default force field files for NAMD running

#> molaical.exe -call run -c runnamd -i -dispdev text -args -system_topology complex.psf -system_coords complex.pdb -ff toppar_water_ions.str -ff ligand.str

6) Using the default path of "extra_path" for NAMD in MolAICal and the default force field files for NAMD running. "-s" equals "-system_topology"; "-c" equals "-system_coords".
#> molaical.exe -call run -c runnamd -i -dispdev text -args -s complex.psf -c complex.pdb -ff toppar_water_ions.str -ff ligand.str

7) Giving the path of NAMD and the default force field files for NAMD running. "-s" equals "-system_topology"; "-c" equals "-system_coords".
#> molaical.exe -call run -c runnamd -i -dispdev text -args -s complex.psf -c complex.pdb -rn E:/soft/namd2.exe -nf test.namd -ff toppar_water_ions.str -ff ligand.str

8) Customizing the configuration file for NAMD input and the default force field files for NAMD running. "-s" equals "-system_topology"; "-c" equals "-system_coords".
#> molaical.exe -call run -c runnamd -i -dispdev text -args -s complex.psf -c complex.pdb -nf test.namd -ff toppar_water_ions.str -ff ligand.str

9) Customizing the configuration file for NAMD input without any modification.
#> molaical.exe -call run -n vmd -c runnamd -i -dispdev text -args -nf test.namd

(12) MolAICal will run the DOS command in Windows or run the shell bash command in Linux if its value is "ecommand".
In Windows, MolAICal can call the DOS command (e.g., the list command "dir") as follows:

```
#> molaical.exe -call run -c ecommand -i dir
```

In Linux, MolAICal can call the bash command (e.g., the list command "ls") as follows:

```
#> molaical.exe -call run -c ecommand -i ls
```

Note: the DOS or bash commands are inputted after the argument **"-i".**

6.2.1 Detect potential pocket of receptor

Fpocket (MIT license)[47] is an open-source computational framework designed for the rapid detection of potential ligand-binding pockets in protein structures. The algorithm employs a **Voronoi** tessellation approach to partition three-dimensional atomic space and identify cavities that may accommodate small molecules. Its modular architecture allows users to analyze both static structures and dynamic ensembles, extract geometric and physicochemical pocket descriptors, and develop customized scoring functions for druggability assessment. fpocket's strengths lie in its computational efficiency, scalability, and adaptability, making it particularly suitable for large-scale structural screenings and virtual screening pipelines. Despite its geometric nature, which may occasionally overpredict superficial cavities, it remains one of the most widely adopted tools for

protein pocket identification due to its speed, transparency, and integrability into structural bioinformatics workflows.

P2Rank (MIT license)[48] is a machine learning–based tool that predicts ligand-binding sites directly from protein structures. Rather than relying on geometric partitioning, it evaluates solvent-accessible surface points using a set of local structural and physicochemical features to estimate their likelihood of ligand interaction. These points are subsequently clustered into putative binding pockets ranked by predicted ligandability. Implemented as a standalone command-line program, P2Rank operates without the need for external databases or precomputed templates and supports multiple structure formats, including PDB, mmCIF, and AlphaFold-derived models. The software integrates rescoring functionality—allowing refinement of predictions from other methods such as fpocket—and provides outputs suitable for visualization in PyMOL or ChimeraX. Its learning-based paradigm typically yields fewer but more biologically relevant pockets, enhancing prioritization in drug discovery and structural analysis tasks.

- **Comparison of the Two Tools**
  - **Algorithmic paradigm:** fpocket is primarily based on geometric and topological principles (specifically, the Voronoi tessellation method), whereas P2Rank relies on machine learning, integrating local surface features and clustering techniques as its core methodology.
  - **Speed and computational resources:** fpocket launches quickly and requires minimal computational resources; in contrast, P2Rank has a slightly slower startup and demands somewhat greater memory and processing capacity.
  - **Number and quality of predicted pockets:** fpocket tends to generate a larger number of potential pockets, which often necessitate further post-processing and filtering. P2Rank, on the other hand, typically predicts fewer pockets but ranks them with higher biological relevance and confidence.
  - **Extended functionality:** fpocket supports analysis of conformational ensembles and molecular dynamics trajectories through its mdpocket module, whereas P2Rank offers rescoring capabilities that allow refinement of pocket predictions generated by other tools.
  - **Recommended application scenarios:** fpocket is well-suited for rapid and large-scale screening of protein structure databases. In contrast, P2Rank is more appropriate when higher prediction accuracy, machine learning–based refinement, and prioritization of top candidate binding sites are desired.
  - **Operating platforms:** at present, fpocket is not compatible with the Windows operating system and can only be executed on Linux. In contrast, P2Rank is cross-platform and can be run on both Windows and Linux systems.

The usages of fpocket are as follows:

Mandatory parameters :

fpocket -f --file pdbFile

[fpocket -F --fileList fileList]

Optional output parameters

-x --calculate_interaction_grids : Specify this flag if you want fpocket to calculate VdW and

	Coulomb grids for each pocket
-d --pocket_descr_stdout :	Put this flag if you want to write fpocket descriptors to the standard output
Optional input parameters	
-l --model_number (int) :	Number of Model to analyze.
-y --topology_file (string) :	File name of a topology file (Amber prmtop).
Optional pocket detection parameters (default parameters)	
-m --min_alpha_size (float) :	Minimum radius of an alpha-sphere. (6.2)
-M --max_alpha_size (float) :	Maximum radius of an alpha-sphere. (3.4)
-D --clustering_distance (float) :	Distance threshold for clustering algorithm (2.4)
-C --clustering_method (char):	Specify the clustering method wanted for grouping voronoi vertices together (s) s : single linkage clustering m : complete linkage clustering a : average linkage clustering c : centroid linkage clustering
-e --clustering_measure (char) :	Specify the distance measure for clustering (e) e : euclidean distance b : Manhattan distance
-i --min_spheres_per_pocket (int) :	Minimum number of a-sphere per pocket. (15)
-p --ratio_apol_spheres_pocket (float) :	Minimum proportion of apolar sphere in a pocket (remove otherwise) (0.0)
-A --number_apol_asph_pocket (int) :	Minimum number of apolar neighbor for an a-sphere to be considered as apolar. (3)
-v --iterations_volume_mc (integer) :	Number of Monte-Carlo iteration for the calculation of each pocket volume.(300)

The usages of p2rank are as follows:

usage:

prank <command> <dataset.ds> [options]

commands:

predict	... predict pockets (P2RANK)
eval-predict	... evaluate model on a dataset with known ligands
rescore	... rescore previously detected pockets (PRANK)
eval-rescore	... evaluate rescoring model on a dataset with known ligands

datasets:

Dataset files for prediction should contain list of pdb files.
Dataset files for rescoring should contain list of protein files
that are outputs of one of the supported pocket prediction methods
(fpocket, ConCavity). In datasets for evaluation and training they
must be paired with liganated-proteins (correct solutions).
See example datasets in test_data/ directory.

options:

-f <path>	run on single pdb file instead of a dataset

-c <path>	use configuration file that overrides default configuration
	in config/default.groovy, path relative to config/ directory
-m <path>	use previously trained classifier file relative to models/ directory
-o <path>	specify output directory (relative to working dir)
	default: test_output/<comamnd>_<dataset>
other parameters:	
-threads <int>	number of execution threads, dafault: num. of processors + 1
-visualizations <0/1>	produce PyMOL visualizations, default: true
-<param> <value>	for full list of parameters see config/default.groovy

The example commands are as follows:
1) Print help information of detecting potential pocket of receptor in MolAICal
#> molaical.exe -call run -help

2) Print help information of "fpocket"
#> molaical.exe -call run -c fpocket

3) Print help information of "p2rank"
#> molaical.exe -call run -c pocket -i help

4) Find pockets by calling "p2rank" in MolAICal with PDB molecular format, the output files will be in the folder "results"
#> molaical.exe -call run -c pocket -i predict -f 1UYD.pdb -o results

5) Find pockets by calling "p2rank" in MolAICal with Crystallographic Information File (CIF) molecular format, the output files will be in the folder "results"
#> molaical.exe -call run -c pocket -i predict -f 1UYD.cif -o results

6) Find pockets by calling "fpocket" in MolAICal with PDB molecular format
#> molaical.exe -call run -c fpocket -i -f 1UYD.pdb

7) Find pockets by calling "fpocket" in MolAICal with Crystallographic Information File (CIF) molecular format
#> molaical.exe -call run -c fpocket -i -f 1UYD.cif

6.2.2 Docking of protein-protein, protein-petide and protein-nucleic acid

MolAICal integrates the open-source LightDock[49-51] software (https://lightdock.org) and further enhances the docking accuracy of biomacromolecules through MM/GBSA or MM/PBSA methods. LightDock is an open-source (GPL-3.0 license), **docking framework** for **protein-protein**, **protein-**

peptide, **protein-RNA,** and **protein-DNA** complexes. It handles flexible and challenging cases (e.g., transient interactions, low-affinity complexes, or systems requiring backbone/side-chain flexibility) **particularly well and serves as an extensible platform for testing new scoring functions, restraints**, or **optimization strategies.** LightDock adapts the Glowworm Swarm Optimization (GSO) algorithm[52], a bio-inspired swarm-intelligence method originally developed for multimodal function optimization.

The general steps of the GSO algorithm for molecular docking are as follows:

a) **Swarm initialization** — Hundreds of "swarm centers" are evenly distributed on the receptor surface. Around each center, "glowworms" (candidate ligand poses, e.g., 200–400 glowworms) are randomly placed.

b) **Pose representation** — Each glowworm is defined by 7 Degrees of Freedom (DoF) (3 translation + 4 quaternion rotation) + optional ANM normal-mode extents (typically first 10 non-trivial modes per partner).

c) **Luciferin** — Indicates pose quality. The luciferin level (l_i) reflects the quality of the scoring function value (a better score results in higher/brightness of luciferin).

d) **Iteration cycle** (user-defined number of steps, typically 100–300):
   - ✧ Update the luciferin of every glowworm according to its current score.
   - ✧ For each glowworm, identify neighbors within a dynamic radial sensor range.
   - ✧ Probabilistically move toward the brightest neighbor (higher luciferin):
     - ◆ Translation: fixed step along the vector to the neighbor.
     - ◆ Rotation: spherical linear interpolation (SLERP) of quaternions.
     - ◆ ANM modes (if enabled): interpolation of normal-mode extents.
   - ✧ Optional local Powell minimization on the best glowworm of each swarm every few steps.

e) **Convergence** — Swarms independently converge toward low-energy regions; the dynamic sensor range and local decision-making help the population escape local minima and sample multiple binding modes simultaneously.

f) **Post-processing** — All final glowworm poses are clustered (typically ligand RMSD < 4 Å cutoff), and clusters are ranked by score.

g) **Further evaluation** — MolAICal can be employed to dynamically assess the binding affinity of 'glowworms' (candidate ligand poses) using MM/GBSA or MM/PBSA, enabling further screening for more accurate binding conformations.

This swarm-based strategy makes LightDock particularly robust for flexible and ab initio cases and distinguishes it from single-trajectory methods like traditional Monte Carlo or genetic algorithms.

Overall, LightDock consists of two main steps: setup and simulation. The command-line parameters involved in these steps are described below (also refer to: https://lightdock.org):

✧ **Setup**

```
usage: lightdock3_setup.py [-h] [-s SWARMS] [-g GLOWWORMS]
                           [--seed_points STARTING_POINTS_SEED] [--noxt] [--noh]
                           [--now] [--verbose_parser] [-anm]
                           [--seed_anm ANM_SEED] [--anm_rec_rmsd ANM_REC_RMSD]
                           [--anm_lig_rmsd ANM_LIG_RMSD] [-ar ANM_REC]
```

```
                        [-al ANM_LIG] [-r restraints] [-membrane]
                        [-transmembrane] [-sp] [-sd SURFACE_DENSITY]
                        [-sr SWARM_RADIUS] [-flip] [-fd FIXED_DISTANCE]
                        [-spr SWARMS_PER_RESTRAINT] [--ds] [-V]
                        receptor_pdb_file ligand_pdb_file

positional arguments:
  receptor_pdb_file     Receptor structure PDB file
  ligand_pdb_file       Ligand structure PDB file
optional arguments:
  -h, --help            show this help message and exit
  -s SWARMS, --s SWARMS, --swarms SWARMS
                        Fixed number of swarms of the simulation
  -g GLOWWORMS, --g GLOWWORMS, --glowworms GLOWWORMS
                        Number of glowworms per swarm
  --seed_points STARTING_POINTS_SEED
                        Random seed used in starting positions calculation
  --noxt, -noxt         Remove OXT atoms
  --noh, -noh           Remove Hydrogen atoms
  --now, -now           Remove H2O
  --verbose_parser, -verbose_parser
                        PDB parsing verbose mode
  -anm, --anm           Activates the use of ANM backbone flexibility. This parameter may cause
                        structural distortion in docking results by activating backbone flexibility modeled
                        via ANM (using ProDy). Avoid using it unless implementing better
                        conformational sampling (e.g., energy minimization). Instead, pre-generate
                        multiple conformations to bypass backbone flexibility during docking.
  --seed_anm ANM_SEED, -seed_anm ANM_SEED
                        Random seed used in ANM intial extent
  --anm_rec_rmsd ANM_REC_RMSD, -anm_rec_rmsd ANM_REC_RMSD
                        RMSD interval to generate random ANM conformations
  --anm_lig_rmsd ANM_LIG_RMSD, -anm_lig_rmsd ANM_LIG_RMSD
                        RMSD interval to generate random ANM conformations
  -ar ANM_REC, --ar ANM_REC, -anm_rec ANM_REC, --anm_rec ANM_REC
                        Number of ANM modes for receptor
  -al ANM_LIG, --al ANM_LIG, -anm_lig ANM_LIG, --anm_lig ANM_LIG
                        Number of ANM modes for ligand
  -r restraints, --r restraints, -rst restraints, --rst restraints
                        Restraints file. If restraints_file is provided, residue restraints will be considered
                        during the setup and the simulation. If restraints_file is not provided, the setup and
                        the simulation will focus on the entire receptor.
  -membrane, --membrane
                        Enables the extra filter for membrane restraints. When enabled, this flag considers
                        the receptor partner is aligned to the Z-axis and filters out swarms that would not be
```

compatible with a transmembrane domain. To do so, it makes use of the residues provided as receptor restraints.

-transmembrane, --transmembrane
Enables the extra filter for transmembrane restraints. When enabled, this flag considers the receptor partner to be aligned to the Z-axis and filters out swarms that would not be inside the membrane.

-sp, --sp, -starting_positions, --starting_positions
Enables the generation of support files in the "init" directory

-sd SURFACE_DENSITY, --sd SURFACE_DENSITY, -surface_density SURFACE_DENSITY, --surface_density SURFACE_DENSITY
Surface density used for calculating the number of swarms

-sr SWARM_RADIUS, --sr SWARM_RADIUS, -swarm_radius SWARM_RADIUS, --swarm_radius SWARM_RADIUS
Swarm radius at generating glowworm poses (translation)

-flip, --flip Activates the 180° flip of 50% of starting poses when using restraints

-fd FIXED_DISTANCE, --fd FIXED_DISTANCE, -fixed_distance FIXED_DISTANCE, --fixed_distance FIXED_DISTANCE
Use a fixed distance (Å) instead of using ligand to calculate distance of swarms to receptor surface

-spr SWARMS_PER_RESTRAINT, --spr SWARMS_PER_RESTRAINT, -swarms_per_restraint SWARMS_PER_RESTRAINT, --swarms_per_restraint SWARMS_PER_RESTRAINT
Number of swarms to keep per restraint

--ds, -ds Enable dense sampling for restraints-swarm filtering

-V, -v, --version show version

✧ Simulation

usage: lightdock3.py [-h] [-f configuration_file] [-s SCORING_FUNCTION] [-sg GSO_SEED] [-t TRANSLATION_STEP] [-r ROTATION_STEP] [-V] [-c CORES] [--profile] [-mpi] [-ns NMODES_STEP] [-min] [--listscoring] [-l SWARM_LIST [SWARM_LIST ...]] setup_file steps

positional arguments:

setup_file Setup file name

steps number of steps of the simulation

optional arguments:

-h, --help show this help message and exit

-f configuration_file, --file configuration_file
algorithm configuration file

-s SCORING_FUNCTION, --scoring_function SCORING_FUNCTION
scoring function used

-sg GSO_SEED, --seed GSO_SEED
seed used in the algorithm

-t TRANSLATION_STEP, --translation_step TRANSLATION_STEP

```
                        translation step
  -r ROTATION_STEP, --rotation_step ROTATION_STEP
                        normalized rotation step
  -V, -v, --version     show version
  -c CORES, --cores CORES
                        number of cpu cores to use
  --profile             Output profiling data
  -mpi, --mpi           activates the MPI parallel execution
  -ns NMODES_STEP, --nmodes_step NMODES_STEP
                        normalized normal modes step
  -min, --min           activates the local minimization
  --listscoring         list all available scoring functions
  -l SWARM_LIST [SWARM_LIST ...], --list SWARM_LIST [SWARM_LIST ...]
                        List of swarms to simulate. Only simulate the swarm IDs specified by this list. For
                        example, to simulate only swarm 0: -l 0
```

For '-f configuration_file' in **lightdock3.py,** an example of is 'configuration_file' as follows:

```
# GlowWorm configuration file - algorithm parameters

[GSO]

# Rho
rho = 0.4
# Gamma
gamma = 0.6
# Initial Luciferin
initialLuciferin = 5.0
# Initial glowworm vision range (in Angstrom)
initialVisionRange = 15.0
# Max vision range (in Angstrom)
maximumVisionRange = 40.0
# Beta
beta = 0.16
# Max number of neighbors
maximumNeighbors = 5
```

✧ Generate conformations

```
usage: lgd_generate_conformations.py [-h] [--setup setup_file]
                                     receptor_structure ligand_structure
                                     lightdock_output glowworms

positional arguments:
```

```
  receptor_structure        receptor structures: PDB file or list of PDB files
  ligand_structure          ligand structures: PDB file or list of PDB files
  lightdock_output          lightdock output file
  glowworms                 number of glowworms
optional arguments:
  -h, --help                show this help message and exit
  --setup setup_file, -setup setup_file, -s setup_file
                            Simulation setup file
```

✧ Clustering structures intra-swarm

```
usage: lgd_cluster_bsas.py [-h] gso_output_file

positional arguments:
  gso_output_file   LightDock output file
optional arguments:
  -h, --help        show this help message and exit
```

✧ Script for Generation and Cluster in batch

```
usage: ant_thony.py [-h] [--cores CORES] tasks_file_name

positional arguments:
  tasks_file_name       A file containing a task for each line
optional arguments:
  -h, --help            show this help message and exit
  --cores CORES, -cores CORES, -c CORES
                        CPU cores to use
```

✧ Creating the best predictions rank by LightDock score

```
usage: lgd_rank.py [-h] [-c CLASHES_CUTOFF] [-f RESULT_FILE] [--ignore_clusters]
                   num_swarms steps

positional arguments:
  num_swarms                number of swarms to consider
  steps                     steps to consider
optional arguments:
  -h, --help                show this help message and exit
  -c CLASHES_CUTOFF, --clashes_cutoff CLASHES_CUTOFF
                            clashes cutoff
  -f RESULT_FILE, --file_name RESULT_FILE
                            lightdock output file to consider
  --ignore_clusters         Ignore cluster information
```

✧ Filtering

usage: lgd_filter_restraints.py [-h] [--cutoff CUTOFF] [--fnat FNAT] [--rnuc] [--lnuc] [--include_passive] [--only_passive] ranking_file restraints_file receptor_chains ligand_chains

positional arguments:

ranking_file	Path of ranking file to be used
restraints_file	File including restraints
receptor_chains	Chains on the receptor partner (1st molecule)
ligand_chains	Chains on the ligand partner (2nd molecule)

optional arguments:

-h, --help	show this help message and exit
--cutoff CUTOFF, -cutoff CUTOFF, -c CUTOFF	Interaction cutoff
--fnat FNAT, -fnat FNAT, -f FNAT	Structures with at least this fraction of native contacts
--rnuc	Is receptor molecule a nucleic acid?
--lnuc	Is ligand molecule a nucleic acid?
--include_passive, -include_passive	Include passive restraints into filtering
--only_passive, -only_passive	Filter only by passive restraints

* Once the clustering and filtering steps have finished, a new directory called "**filtered**" has been created, which contains any predicted structure. Inside the folder named "**filtered**", there is a file with the ranking of these structures by score called **"rank_filtered.list"**. For more information, please go to the website of LightDock (https://lightdock.org).

✧ Supported scoring functions

- ✧ **MM/GBSA**: Molecular Mechanics/Generalized Born Surface Area, a computational method used to estimate the binding free energy of the complex (e.g., protein-protein, protein-ligand, protein-peptide, *etc*).
- ✧ **MM/PBSA**: Molecular Mechanics Poisson-Boltzmann Surface Area is a computational method used to estimate the binding free energy between molecules (e.g., protein-protein, protein-ligand, protein-peptide, *etc*).
- ✧ **cpydock**: Implementation in C of the pyDock scoring function (also known as pyDockLite).
- ✧ **dfire**: Implementation of the DFIRE scoring function in Cython.
- ✧ **fastdfire**: Implementation of the DFIRE scoring function using the Python C-API, faster than dfire.
- ✧ **dfire2**: Implementation of the DFIRE2 scoring function using the Python C-API, despite a Cython version is also included for demonstrational purposes.
- ✧ **dna**: Implementation of the pyDockDNA scoring function (no desolvation) and custom Van der Waals weight for protein-DNA docking. Implemented using the Python C-API.
- ✧ **ddna**: Implementation of the DDNA scoring function as described in C. Zhang, S. Liu, Q. Zhu, and Y. Zhou, A knowledge-based energy function for protein-ligand, protein-protein and protein-DNA complexes, J. Med. Chem. 48, 2325-2335 (2005).

- ✧ **mj3h**: Pairwise contact energies for 20 types of residues, Mj3h.
- ✧ **pisa**: A statistical potential from the Improving ranking of models for protein complexes with side chain modeling and atomic potentials publication.
- ✧ **sd**: An electrostatics and Van der Waals based scoring function as described in the SwarmDock publication, but using AMBER94 force-field charges and parameters.
- ✧ **sipper**: Intermolecular pairwise propensities of exposed residues, SIPPER.
- ✧ **tobi**: TOBI potentials scoring function.
- ✧ **vdw**: A truncated Van der Waals (Lennard-Jones potential) as described in the original pyDock publication.

✧ The docking process mainly contains two steps: 'setup' and 'simulation'. The 'setup' is as follows:

```
#> molaical.exe -call run -c ldock -i lightdock3_setup.py R_protein.pdb P_peptide.pdb --noxt --noh --now -anm -sp -rst restraints.list
```

- ➢ **-call**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- ➢ **-c:** It will run molecular docking (protein-protein, protein-peptide, protein-DNA or protein-RNA) by calling "LightDock" if its value is "ldock".
- ➢ The 1st and 2nd arguments after "lightdock3_setup.py" are receptor and ligand files, respectively.
- ➢ **--noxt**: If this option is enabled, LightDock ignores OXT atoms. This is useful for several scoring functions that don't understand this special type of atom.
- ➢ **--noh**: If this option is enabled, LightDock ignores hydrogen atoms. This is relevant for dfire, fastdfire or dfire2 scoring functions (among others). Only removes atoms starting with H character, thus not recognizing types such as 1H, etc.
- ➢ **--now**: If this option is enabled, crystal waters will be removed.
- ➢ **--anm:** If this option is enabled, the ANM mode is activated and backbone flexibility is modeled using ANM (via ProDy).
- ➢ **-membrane:** When enabled, this flag considers the receptor partner is aligned to the Z-axis and filters out swarms which would not be compatible with a trans-membrane domain. To do so, it makes use of the residues provided as receptor restraints.
- ➢ **-transmembrane:** The argument "--transmembrane" has the opposite function of "--membrane". When enabled, this flag considers the receptor partner to be aligned to the Z-axis and filters out swarms that would not be inside the membrane.
- ➢ **-sp:** If enabled (False by default), writes in the init folder debugging extra files.
- ➢ **-r, -rst restraints_file:** If restraints_file is provided, residue restraints will be considered during the setup and the simulation. If restraints_file is not provided, the setup and the simulation will focus on the entire receptor.

In this example:

- ◆ It will produce the folder named 'init', which contains the molecules in PDB format that represent the glowworms of swarms for further docking.
- ◆ The files 'lightdock_R_protein.pdb' and 'lightdock_P_peptide.pdb' are re-generated from 'R_protein.pdb' and 'P_peptide.pdb' by LightDock.

✧ The 'simulation' is as follows:

```
#> molaical.exe -call run -c ldock -i lightdock3.py setup.json 100 -s fastdfire -c 12
```

- **-call**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- **-c:** It will run molecular docking (protein-protein, protein-peptide, protein-DNA, or protein-RNA) by calling "LightDock" if its value is "ldock".
- The 1st and 2nd arguments after "lightdock3.py" are the configuration file generated on the setup step, and the number of steps of the simulation (here, it is 100 steps), respectively.
- **-s SCORING_FUNCTION:** The default scoring function (DFIRE, fast C implementation fastdfire) using this flag. A name of a scoring function or a file containing the name and weight of multiple scoring functions are accepted.
- **-c CORES:** By default, the total number of available CPU cores on the hardware is used to run the simulation, but a different number of CPU cores can be specified via this option.

✧ Generate models, Clustering, Rank, and filter by MolAICal:

```
#> molaical.exe -call run -c sfile -i lrank.sh 1::=mempro.pdb 2::=P_peptide.pdb 3::=R 4::=P 5::=15 6::=restraints.list 7::=gso_100.out 8::=200 9::=generate_dock.list 10::=cluster_dock.list 11::=100 12::=5.0 13::=0.4 14::=rank_by_scoring.list 15::=0 16::=\"\" 17::=\"\" 18::=\"\"
```

```
#> molaical.exe -call run -c sfile -i lrank.sh 1::=mempro.pdb 2::=P_peptide.pdb 3::=R 4::=P 5::=15 6::=restraints.list
```

- **'-call'**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- **'::='**: Here, it is used to assign values to parameters in the specified order. Variables must immediately follow "::=" without any space.
- **'-c'**: It will run the VMD scripts of MolAICal if its value is "sfile".
- **'1::='**: It is the path of the first input molecule (receptor). Required parameters.
- **'2::='**: It is the path of the second input molecule (ligand). Required parameters.
- **'3::='**: Chains on the receptor partner (1st molecule). The default value is **'R'**.
- **'4::='**: Chains on the ligand partner (2nd molecule). The default value is **'P'**.
- **'5::='**: The number of used CPU cores. The default value is **12**.
- **'6::='**: File including restraints. The default value is **'restraints.list'**.
- **'7::='**: The simulated file in the specific simulation step (e.g., the gso_100.out corresponds to 100). The default file name is **'gso_100.out'**.
- **'8::='**: The number of the generated models. The default value is **200**.
- **'9::='**: The file for generating conformations. The default file name is **'generate_dock.list'**.
- **'10::='**: The file for conformations cluster. The default file name is **'cluster_dock.list'**.
- **'11::='**: Steps to consider for the rank step. The default value is **100**.
- **'12::='**: Interaction cutoff for the filter step. The default value is **5.0**.
- **'13::='**: Structures with at least this fraction of native contacts for the filter step. The default value is **0.4**.
- **'14::='**: Path of ranking file to be used. The default file name is **'rank_by_scoring.list'**.
- **'15::='**: The number of the deleted swarms. If the user deletes some generated swarms for their research purposes, it needs to add the number of the deleted swarms for traversing all swarms in other steps. The default

value is **0**.

- **'16::='**: Is Debug mode enabled? The value is 0 or 1, where 1 enables Debug mode and 0 disables it. The default value of Debug mode is 1.
- **'17::='**: More options for "lgd_filter_restraints.py". The default value is an empty string, and the empty string is represented as \"\". If this input contains multiple parameters, separate them using the comma character ','. MolAICal will automatically convert each comma ',' to a space " ".
- **'18::='**: More options for "lgd_generate_conformations.py". The default value is an empty string, and the empty string is represented as \"\". If this input contains multiple parameters, separate them using the comma character ','. MolAICal will automatically convert each comma ',' to a space " ".
- **'19::='**: More options for "lgd_rank.py". The default value is an empty string, and the empty string is represented as \"\". If this input contains multiple parameters, separate them using the comma character ','. MolAICal will automatically convert each comma ',' to a space " ".

✧ MM/GBSA or MM/PBSA calculation by MolAICal:

```
#> molaical.exe -call run -c sfile -i mmgbpbsa_batch.sh 1::=R 2::=P 3::=6 4::=961 5::=1000 6::=cal_mmgbpbsa.sh 7::=vmd_convert.list 8::=mmgbsa_file.list 9::=1 10::=\"\"
```

```
#> molaical.exe -call run -c sfile -i mmgbpbsa_batch.sh 1::=R 2::=P
```

- **'-call'**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- **'::='**: Here, it is used to assign values to parameters in the specified order. Variables must immediately follow "::=" without any space.
- **'-c'**: It will run the VMD scripts of MolAICal if its value is "sfile".
- **'1::='**: Chain of the first molecule. The default value is **'R'**.
- **'2::='**: Chain of the second molecule. The default value is **'P'**.
- **'3::='**: The number of used CPU cores. The default value is **12**.
- **'4::='**: The starting frame for analysis. The default value is **951**.
- **'5::='**: The end frame for analysis. The default value is **1000**.
- **'6::='**: MolAICal script for calculating MM/GBSA or MM/PBSA. The default file name is **'**cal_mmgbpbsa.sh**'**.
- **'7::='**: The file list that contains the converting jobs by VMD. Removing the membrane. The default file name is **'**vmd_convert.list**'**.
- **'8::='**: The file list that contains the job of MM/GB/PBSA jobs. The default file name is **'**mmgbsa_file.list**'**.
- **'9::='**: Is Debug mode enabled? The value is 0 or 1, where 1 enables Debug mode (meanwhile, save PDB and PSF files from the results of MM/GBSA based on the trajectory of the middle frame), and 0 disables it. The default value of Debug mode is 1.
- **'10::='**: The more options for the MolAICal script. The default value is an empty string, and the empty string is represented as \"\". If this input contains multiple parameters, separate them using the comma character ','. MolAICal will automatically convert each comma ',' to a space " ". If its value is **"cyc"**, it will assign the last residue number of the peptide to build the cyclic peptide in PDB and PSF format.

The contents of the default file **'6::=**cal_mmgbpbsa.sh**'** which can be modified according to the research purpose, are as follows:

```
# 1. fix autopsf
```

```
molaical.exe -call run -c fixpsf

# 2. generate psf
molaical.exe -call run -c vmdargs -i -pdb $molargs1 -dispdev text -args autopsf autopsf -mol 0 -prefix $molargs4

# 3. default to run 20 fs minimization and 40 fs MD simulation
molaical.exe -call run -c runnamd -i -dispdev text -args -s $molargs4_formatted_autopsf.psf -c $molargs4_formatted_autopsf.pdb -minimize $molargs6 -if_run 1

# 4. Cut last 40 steps for MM/GBSA calculation
molaical.exe -call run -c catdcd -i -dispdev text -args -out $molargs4_result.dcd -first $molargs5 -last $molargs6 -stride 1 $molargs4_formatted_autopsf_md.dcd

# 5. Calculate MM/GB/PBSA: "-pb 0 -gb 1" is MM/GBSA; "-pb 2" is MM/PBSA. Others are the same.
# molaical.exe -call run -c mmpbgbsa -i -dispdev text -args -s $molargs4_formatted_autopsf.psf -c $molargs4_formatted_autopsf.pdb -t $molargs4_result.dcd -cs "chain,$molargs2,$molargs3" -rs "chain,$molargs2" -ls "chain,$molargs3" -pb 2
molaical.exe -call run -c mmpbgbsa -i -dispdev text -args -s $molargs4_formatted_autopsf.psf -c $molargs4_formatted_autopsf.pdb -t $molargs4_result.dcd -cs "chain,$molargs2,$molargs3" -rs "chain,$molargs2" -ls "chain,$molargs3" -pb 0 -gb 1

# 6. delete file with command (in windows: del xxx, in linux: rm xxx)
molaical.exe -call run -c ecommand -i rm -rf *md.* *run.* *result.dcd *final.* *_wb* FFTW_* *_autopsf.pdb *_autopsf.psf *_autopsf.log *_formatted.pdb
```

✧ mrank.py: Extract and rank the results of LightDock, MM/GBSA, and MM/PBSA

```
1. Only print the rank results of MM/GB/PBSA with deleting the './mmgbpbsa /candidates'
#>molaical.exe -call run -c sfile -i mrank.py
2. Only print the rank results of MM/GB/PBSA without deleting the './mmgbpbsa /candidates'
#>molaical.exe -call run -c sfile -i mrank.py -n
3. Copy the top 2 molecules of the results to the './mmgbpbsa /candidates'
#> molaical.exe -call run -c sfile -i mrank.py -t 2
4. Copy the top 2 molecules of the results to the './filtered/candidates' (delete first, if it exists), which are the results of LightDock.
#>molaical.exe -call run -c sfile -i mrank.py -ft 2
5. Copy the top 2 molecules of the results to the './filtered/candidates' (do not delete, if it exists), which are the results of LightDock.
#>molaical.exe -call run -c sfile -i mrank.py -ft 2 -nd
```

mrank.py usages: Extract and rank candidate molecules.

```
optional arguments:
-h, --help                          show this help message and exit
-s ROOTDIR, --rootdir ROOTDIR       Results directory of MM/GB/PBSA (default: ./mmgbpbsa)
```

-p PATTERN, --pattern PATTERN	Log file pattern (default: "*_mmgbpbsa.log")
-c COLUMN, --column COLUMN	Column index to extract in file of --pattern (0-based, default: 3)
-o OUTFILE, --outfile OUTFILE	Output rank file name (default: final_rank_results.log)
-d CANDIDATES_DIR, --candidates_dir	CANDIDATES_DIR. Candidates folder name (default: candidates)
-t TOP_N, --top_n TOP_N	Number of top molecules to copy to candidates (default: 0) in the folder of --rootdir
-f, --from_outfile	Read results only from existing outfile instead of parsing logs
-y, --delete_candidates	Delete existing candidates folder in the folder of --rootdir if exists (default: True)
-n, --no-delete_candidates	Do not delete the existing candidates folder in the folder of --rootdir
-fd FILTERED_ROOT, --filtered_root	FILTERED_ROOT. Results directory of LightDock (default: ./filtered)
-fr FILTERED_RANK, --filtered_rank	FILTERED_RANK. Results rank file of LightDock (default: ./rank_filtered.list)
-ft FILTERED_TOP_N, --filtered_top_n	FILTERED_TOP_N. Number of top molecules to copy to candidates (default: 0) in the folder of --filtered_root
-df, --delete_filtered	Delete existing candidates folder in the folder of --filtered_root if exists (default: True)
-nd, --no-delete_filtered	Do not delete the existing candidates folder in the folder of --filtered_root

✧ Dealing with nucleic acids:

The command "reduce" (https://github.com/rlabduke/reduce) can be used to delete and add the hydrogen on molecules:

reduce arguments: [-flags] filename or -

Suggested usage:

reduce -FLIP myfile.pdb > myfileFH.pdb (do NQH-flips)

reduce -NOFLIP myfile.pdb > myfileH.pdb (do NOT do NQH-flips)

Flags:

-FLIP	add H and rotate and flip NQH groups
-NOFLIP	add H and rotate groups with no NQH flips
-Trim	remove (rather than add) hydrogens and skip all optimizations
-NOBUILD#.#	build with a given penalty often 200 or 999
-BUILD	add H, including His sc NH, then rotate and flip groups (except for pre-existing methionine methyl hydrogens)
-STRING	pass reduce a string object from a script, must be quoted
-DUMPATOMS FILE	dump the atoms, along with extra information about them,

	to FILE
usage: from within a script, reduce -STRING "_name_of_string_variable_"	
-Quiet	do not write extra info to the console
-REFerence	display citation reference
-Version	display the version of reduce
-Changes	display the change log
-Help	the more extensive description of command line arguments

The usage examples are below:

More help information about "reduce":

```
#> molaical.exe -call run -c ldock -i reduce -Help
```

Delete hydrogen:

```
#> molaical.exe -call run -c ldock -i reduce -Trim DNA_B.pdb > DNA_B_noH.pdb
```

For adding hydrogen, there is a "resname" issue in the nucleic acids (DNA or RNA): The "reduce" command appears to be sensitive to the position of the resname, requiring that within the allowed resname range for nucleic acids, the name must be right-aligned. This issue may be from the chain setting and save operations of VMD.

```
#> molaical.exe -call run -c sfile -i fix_pdb.tcl DNA_B_noH.pdb DNA_B_nh.pdb
```

Usages of "fix_pdb.tcl":

```
Usage: fix_pdb.tcl input.pdb output.pdb ?start_index end_index?

An example as below:
#> molaical.exe -call run -c sfile -i fix_pdb.tcl -args DNA_B_noH.pdb DNA_B_nh.pdb
```

- **1st:** the input molecule that needs to be fixed
- **2nd:** the output molecule after fixing
- **3rd:** the start index in any one line of PDB-format molecular file. The default is 17, which is the start index of resname.
- **4th:** the end index in any one line of PDB-format molecular file. The default is 19, which is the start index of resname.

Add hydrogen:

```
#> molaical.exe -call run -c ldock -i reduce -BUILD DNA_B_nh.pdb > DNA_B_H.pdb
```

For the further preparation of RNA and DNA molecules, the following three requirements still need to be met:

- Hydrogen atoms added by Reduce for nucleic acids are not fully name-compatible with the

AMBER94 force field. (**reduce_to_amber.py:** the script to rename and/or remove incompatible atom types).

- Moreover, Reduce appends terminal hydrogens (at the 3' and/or 5' ends) to RNA, which leads to conflicts with the AMBER94 force field. (**purge_prime_H.py:** the script to remove the terminal (3' and/or 5') hydrogens).
- Finally, the AMBER94 force field requires RNA residues to be labeled with an "R" prefix. (**retag_rna.py:** One script to add or remove the "R" prefix for RNA residues)

Renumbered the atoms (pdb_reatom**)**, there are no atom numbers for the added hydrogen by the above adding hydrogen command. "pdb_reatom" can re-number the whole atoms in the molecule.

```
#> molaical.exe -call run -c ldock -i pdb_reatom DNA_B_H.pdb > DNA.pdb
```

1) Add "R" before the original resname of DNA. The code in "retag_rna.py" is " *atom.residue_name = "R" + atom.residue_name* "

```
#> molaical.exe -call run -c sfile -i retag_rna.py RNA_h.pdb rna_pre.pdb
```

2) Update the H types. The code in "reduce_to_amber.py" is: " *atom.name = translation[atom.name]* ", which in translation contains the changed atom types.

```
#> molaical.exe -call run -c sfile -i reduce_to_amber.py rna_pre.pdb rna_pre2.pdb
```

3) Delete the hydrogens "HO'3" and "HO'5" which conflict with the AMBER94 force field. The code in "purge_prime_H.py" is: " *if atom.name in ("HO'3", "HO'5"): continue* ", which in translation contains the changed atom types.

```
#> molaical.exe -call run -c sfile -i purge_prime_H.py rna_pre2.pdb rna_tagged.pdb
```

4) The **"-r"** argument of "retag_rna.py" means to remove the added tag "R" for the setup.

```
#> molaical.exe -call run -c sfile -i retag_rna.py -r rna_tagged.pdb RNA.pdb
```

Attention should be paid to the different simulation methods used in DNA and RNA docking:

1) For DNA, open the folder DNA, and run setup:

```
#> cd DNA
#> molaical.exe -call run -c ldock -i lightdock3_setup.py protein.pdb DNA.pdb -anm --noxt -sp -rst ../restraints.list
#> molaical.exe -call run -c ldock -i lightdock3.py setup.json 100 -s dna -c 8
```

2) For RNA, open the folder DNA, and run setup:

```
#> cd ../RNA
#> molaical.exe -call run -c ldock -i lightdock3_setup.py protein.pdb RNA.pdb -anm --noxt -sp -rst ../restraints.list
## Simulation needs AMBER94 force field, which requires adding the R tag before "resname" of RNA.
```

```
#> molaical.exe -call run -c ecommand -i mv lightdock_RNA.pdb lightdock_rna_untagged.pdb
#> molaical.exe -call run -c sfile -i retag_rna.py lightdock_rna_untagged.pdb lightdock_RNA.pdb
```

```
#> molaical.exe -call run -c ldock -i lightdock3.py setup.json 100 -s dna -c 8
```

6.2.3 Peptide Representation

This part summarizes three commonly encountered formats for representing peptide and biopolymer information: BILN (Biopolymer Line Notation)[53], HELM (Hierarchical Editing Language for Macromolecules)[54], and FASTA. Each format has different design goals and typical uses: one is lightweight and human-readable, one is a formal standard for macromolecules, and one is a minimal sequence container widely used in bioinformatics. Each format's defining features, typical use cases, strengths, and limitations are described, and given one clear example as follows:

6.2.3.1 FASTA Format

Overview

FASTA is the legacy text-based format for representing nucleotide sequences or amino acid (protein) sequences. It is the universal standard for bioinformatics tools (e.g., BLAST) and simple sequence storage.

Characteristics and Syntax

- **Linearity**: Represents sequences strictly linearly from N-terminus to C-terminus.
- **Alphabet**: Uses the standard 20 single-letter IUPAC codes for natural amino acids.
- **Metadata**: The first line (header) begins with a greater-than symbol (>) followed by a description. The following lines contain the sequence data.

Scope and Limitations

- **Scope**: Ideal for natural, linear peptides and large proteins.
- **Limitations**: FASTA cannot inherently describe:
  - Cyclization (topology).
  - Branching or post-translational modifications (without non-standard hacks).
  - Complex chemical linkers or non-natural amino acids (unless 'X' is used ambiguously).

Example

A simple linear peptide consisting of Alanine, Glycine, and Cysteine.

```
>Sample_Peptide_01
AGC
```

6.2.3.2 HELM Format

Overview

Developed by the Pistoia Alliance, HELM (Hierarchical Editing Language for Macromolecules) is the modern industrial standard for exchanging complex biomolecular data. It treats molecules not just as sequences, but as hierarchical graphs.

Characteristics and Syntax

HELM uses a complex string format divided into four sections by the dollar sign ($). Format: Simple_Polymers $ Connections $Groups/Annotations$ Version

- **Monomer Definition:** Uses extended dictionaries to define non-natural monomers (e.g., dA for d-Alanine).
- **Explicit Connectivity:** Connectivity is defined in a separate block from the sequence, specifying exactly which atom (R-group) connects to which.

Scope and Applicability

- **Scope**: The gold standard for pharmaceutical registration and database storage. It handles Peptides, Oligonucleotides, and Antibody-Drug Conjugates (ADCs).
- **Applicability**: Designed for machine-parsing. It is generally too complex for humans to read or write fluently without software assistance (e.g., the Pistoia HELM Editor).

Example

A cyclic peptide where Alanine (A), Glycine (G), and Cysteine (C) are connected, and the C-terminus of Cysteine (at R2) is linked back to the N-terminus of Alanine (at R1) to form a ring.

PEPTIDE1{A.G.C}: Defines the linear sequence.

HELM String:

```
PEPTIDE1{A.G.C}$PEPTIDE1,PEPTIDE1,3:R2-1:R1$$$V2.0
```

- **Analysis:**
  -
  - PEPTIDE1,PEPTIDE1,3:R2-1:R1: Defines the edge. Monomer 3 (C) at attachment point R2 connects to Monomer 1 (A) at attachment point R1.

6.2.3.3 BILN Format

Overview

BILN (Boehringer Ingelheim Line Notation) is a newer format developed to bridge the gap between human readability and topological complexity. It serves as a shorthand notation that allows scientists to write complex structures linearly while retaining topological information.

Characteristics and Syntax

- **Inline Topology**: Unlike HELM, which separates connections into a different block, BILN embeds connection information directly next to the monomer.

- **Syntax**: Monomer(Ring_ID, Attachment_Point).
- **Flexibility**: It is often used as an input format for computational tools (like pyPept) to generate 3D conformers or convert to HELM/SMILES.

Scope and Applicability

- **Scope**: Specifically optimized for macrocyclic peptides, stapled peptides, and branched structures.
- **Applicability**: Excellent for "human-in-the-loop" design. A chemist can easily type a BILN string to describe a stapled peptide, whereas writing the equivalent HELM string manually is error-prone.

Example

The same cyclic peptide (A-G-C cyclized head-to-tail) used in the HELM example above.

BILN String:

```
A(1,1)-G-C(1,2)
```

- **Analysis:**
  - A(1,1): Alanine is part of Ring #1, using connection point 1 (N-term).
  - C(1,2): Cysteine is part of Ring #1, using connection point 2 (C-term).
  - The software parser sees that both share Ring ID "1" and automatically creates the bond between them.

Summary Comparison as follows:

Feature	FASTA	HELM	BILN
Primary Use	Biology Bioinformatics	Chemoinformatics Registration	Computational Modeling Design
Structure	Linear Sequence	Hierarchical Graph	Linear with Inline Topology
Cyclization	Not Supported	Explicit (Edge List)	Inline (Ring IDs)
Human Readability	High	Low	Medium-High
Machine Readability	High	Very High	High (via parsers)

6.2.4 Peptide Generation and Virtual Screening

6.2.4.1 Peptide Generation

MolAICal can generate the sequences and 3D structures of peptides with sequential traversal or random algorithms based on the BILN (default in MolAICal), HELM, and FASTA. The usages for peptides' generation are below:

```
usage: pep_gen.py [-h] -l LENGTH -n NUM_SEQUENCES [-s SEED_SEQUENCE]
                  [-fix FIXED_POSITIONS [FIXED_POSITIONS ...]] [-r] [-st]
```

```
[-nc] [-d →forward, backward, both, none]
[-f → fasta, biln, helm, all] [-cc]
[-dis DISULFIDE [DISULFIDE ...]] [-ad AUTO_DISULFIDE]
[-md MAX_DISULFIDE_PAIRS] [-aa AUTO_AMIDE_BOND]
[-ma MAX_AMIDE_PAIRS] [-b] [-bp BRANCH_POINTS]
[-bi BRANCH_POSITION] [-bd →forward, backward, both, none]
[-o OUTPUT] [-3d]
[-ss → base, alpha-helix, beta-sheet, beta-bridge, turn-coil [base, alpha-helix, beta-sheet,
      beta-bridge, turn-coil ...]]
[-rc RDKIT_CONFS] [-dir OUTPUT_3D_DIR] [-sp]
[-of → pdb, mol2, sdf [pdb, mol2, sdf ...]] [-prefix]
[-fprefix FILE_PREFIX] [-c CORES] [--quiet]
```

Generate peptide sequences in various formats with optional 3D structure generation. The optional arguments are below:

Optional arguments:

-h, --help	show this help message and exit
-l, --length LENGTH	Number of amino acid residues in the peptide
-n, --num-sequences NUM_SEQUENCES	Number of unique sequences to generate
-s, --seed-sequence SEED_SEQUENCE	Starting sequence (optional)
-fix, --fixed-positions FIXED_POSITIONS [FIXED_POSITIONS ...]	Positions (1-indexed) to keep fixed during generation
-r, --random	Generate sequences randomly (default: True)
-st, --systematic	Generate sequences by systematic substitution
-nc, --no-caps	Do not add ac/am caps to sequences
-d, --direction [forward, backward, both, none]	Direction for sequence extension from seed
-f, --format [fasta, biln, helm, all]	Output format for sequences (default: biln)
-cc, --cyclic	Generate cyclic peptides (BILN/HELM only)
-dis, --disulfide DISULFIDE [DISULFIDE ...]	Disulfide bond positions as pairs (e.g., 2 5 3 7)
-ad, --auto-disulfide AUTO_DISULFIDE	Automatically generate random disulfide bonds if sequence has >=2 cysteines (default: 0, disabled)
-md, --max-disulfide-pairs MAX_DISULFIDE_PAIRS	Maximum number of disulfide bond pairs to generate automatically (default: 1)
-aa, --auto-amide-bond AUTO_AMIDE_BOND	Automatically generate random amino-carboxyl condensation bonds (default: 0, disabled)
-ma, --max-amide-pairs MAX_AMIDE_PAIRS	Maximum number of amino-carboxyl bond pairs to generate automatically (default: 1)
-b, --branched	Generate branched peptides as multiple connected monomers (BILN/HELM only)
-bp, --branch-points BRANCH_POINTS	Number of branch points (dots) to insert in sequence (default: 2)
-bi, --branch-position BRANCH_POSITION	Position (1-indexed) for branch attachment (default: 3)
-bd, --branch-direction [forward,backward,both,none]	Direction for dot placement: forward (after seed), backward (before seed), both (before and after seed),

	none (random placement, default: none)
-o, --output OUTPUT	Output file for generated sequences
-3d, --generate-3d	Generate 3D structures using pyPept
-ss, --ss-types [base,alpha-helix,beta-sheet,beta-bridge,turn-coil ...]	Secondary structure types to generate (default: all)
-rc, --rdkit-confs RDKIT_CONFS	Number of RDKit random conformers to generate (default: 1)
-dir, --output-3d-dir OUTPUT_3D_DIR	Output directory for 3D structures
-sp, --save-png	Save PNG images of 3D structures (default: False)
-of, --output-3d-formats [pdb,mol2,sdf ...]	Output formats for 3D structures (default: pdb). Note: mol2 will be saved as SDF
-ch, --chain CHAIN	Chain name for PDB files (default: 'Z'). Must be a single character
-prefix, --use-peptide-prefix	Use "peptide_N" as filename prefix instead of sequence (prevents long filenames)
-fprefix, --file-prefix FILE_PREFIX	Custom prefix for output filenames when --use-peptide-prefix is set (default: peptide)
-c, --cores CORES	Number of CPU cores for parallel processing
--quiet, -q	Suppress all output messages (only errors will be shown)

The default format for peptide generation is BILN, and the default **chain name** is **'Z'** for the generated peptides. The example commands are as below:

1) Print help information for peptide generation in MolAICal
#> molaical.exe -call run -c pepgen -i --help

2) Generate 10 random linear peptides with 8 residues with C- and N- caps
#> molaical.exe -call run -c pepgen -i -l 8 -n 10 -o sequences.txt

3) Generate 10 random linear peptides with 8 residues without C- and N- caps
#> molaical.exe -call run -c pepgen -i -l 8 -n 10 -nc -o sequences.txt

3.1) Generate 10 random linear peptides with 8 residues without C- and N- caps and convert them into 3D structures
#> molaical.exe -call run -c pepgen -i -l 8 -n 10 -nc -3d

4) Based on the seed sequence, generate the sequence forward randomly
#> molaical.exe -call run -c pepgen -i -l 10 -n 50 -s ACDEA -d forward -f biln

4.1) Based on the seed sequence, generate the sequences forward randomly and convert them into 3D structures
#> molaical.exe -call run -c pepgen -i -l 10 -n 50 -s ACDEA -d forward -f biln -3d

5) Based on the seed sequence, generate the sequence backward randomly

#> molaical.exe -call run -c pepgen -i -l 10 -n 10 -s ACDEA -d backward -f biln

5.1) Based on the seed sequence, generate the sequence backward randomly and convert them into 3D structures
#> molaical.exe -call run -c pepgen -i -l 10 -n 10 -s ACDEA -d backward -f biln -3d

6) Based on the seed sequence, generate the sequence backward and forward randomly
#> molaical.exe -call run -c pepgen -i -l 10 -n 10 -s ACDEA -d both -f biln

6.1) Based on the seed sequence, generate the sequence backward and forward randomly and convert them into 3D structures
#> molaical.exe -call run -c pepgen -i -l 10 -n 10 -s ACDEA -d both -f biln -3d

7) Generate branched peptides as multiple connected monomers without caps
#> molaical.exe -call run -c pepgen -i -l 12 -n 10 -nc -b -bp 3 -bd forward

7.1) Generate branched peptides as multiple connected monomers without caps and convert them into 3D structures
#> molaical.exe -call run -c pepgen -i -l 12 -n 10 -nc -b -bp 3 -bd forward -3d

Note: When insufficient space is available, the code falls back to a simple split or no split at all, resulting in a number of points that does not equal the specified branch_points.

8) Generate branched peptides as multiple connected monomers in the BILN format based on seed sequence without caps
#> molaical.exe -call run -c pepgen -i -l 36 -n 10 -s CCCCC -nc --branched -bp 2 -d both -bd backward -f biln

8.1) Generate branched peptides as multiple connected monomers in the BILN format based on seed sequence without caps and convert them into 3D structures
#> molaical.exe -call run -c pepgen -i -l 36 -n 10 -s CCCCC -nc --branched -bp 2 -d both -bd backward -f biln -3d

9) Generate branched peptides as multiple connected monomers in the HELM format based on seed sequence without caps
#> molaical.exe -call run -c pepgen -i -l 36 -n 5 -s CCCCC -nc -b -bp 2 -d both -bd backward -f helm

9.1) Generate branched peptides as multiple connected monomers in the HELM format based on seed sequence without caps and convert them into 3D structures
#> molaical.exe -call run -c pepgen -i -l 36 -n 5 -s CCCCC -nc -b -bp 2 -d both -bd backward -f helm -3d

10) Generate branched peptides backward as multiple connected monomers in the HELM format

based on seed sequence without caps
#> molaical.exe -call run -c pepgen -i -l 36 -n 5 -s CCCCC -nc -b -bp 2 -d backward -bd backward -f helm -3d

11) Generate branched peptides forward as multiple connected monomers in the HELM format based on seed sequence without caps
#> molaical.exe -call run -c pepgen -i -l 36 -n 5 -s CCCCC -nc -b -bp 2 -d forward -bd forward -f helm -3d

12) Generate branched peptides as multiple connected monomers in the HELM format based on seed sequence without caps
#> molaical.exe -call run -c pepgen -i -l 36 -n 5 -s CCCCC -nc -b -bp 2 -bd none -f helm -3d

13) Within the peptide chain, bonds are formed through dehydration condensation between amino and carboxyl groups, not limited to R1 (–NH_2), R2 (–COOH), and R3; R3 may also contain additional amino or carboxyl groups
#> molaical.exe -call run -c pepgen -i -l 8 -n 10 -s KDEKED --auto-amide-bond 1 -f biln -3d -nc

14) Generate three disulfide bonds in each peptide using 10 CPU cores
#> molaical.exe -call run -c pepgen -i -l 12 -n 10 -s CCCCCCCC -ad 3 -md 3 -f biln -3d -nc -c 10

15) Generate peptide segments containing one disulfide bond and two amine-carboxyl bonds
#> molaical.exe -call run -c pepgen -i -l 12 -n 1 -s CKDEKEDKECDE --auto-disulfide 1 --auto-amide-bond 2 --max-amide-pairs 2 -3d

16) Randomly modify the peptide segment of length 16 using other amino acid residues, while keeping the amino acid residues at positions 1 and 3 unchanged
#> molaical.exe -call run -c pepgen -i -l 16 -n 10 -s CCCCCCCCCCCCCCCC -d none -fix 1 3 -f biln -3d -nc

17) Generate cyclic peptides; however, cyclic peptides are only supported in "base" and "turn-coil" formats, so the argument "-ss alpha-helix beta-sheet" will be ignored
#> molaical.exe -call run -c pepgen -i -l 8 -n 3 -cc -3d -ss alpha-helix beta-sheet -c 4 -nc

18) Generate cyclic peptides with one disulfide bond
#> molaical.exe -call run -c pepgen -i -l 10 -n 3 -cc -3d -s CCCCCCC -ad 1 -c 4 -nc

19) Generate cyclic peptides with one amino-carboxyl condensation bond without output information, and save a PNG format figure one by one
#> molaical.exe -call run -c pepgen -i -l 14 -n 3 -cc -3d -s CKDEKEDKECDE -aa 1 -c 3 -nc -q -sp

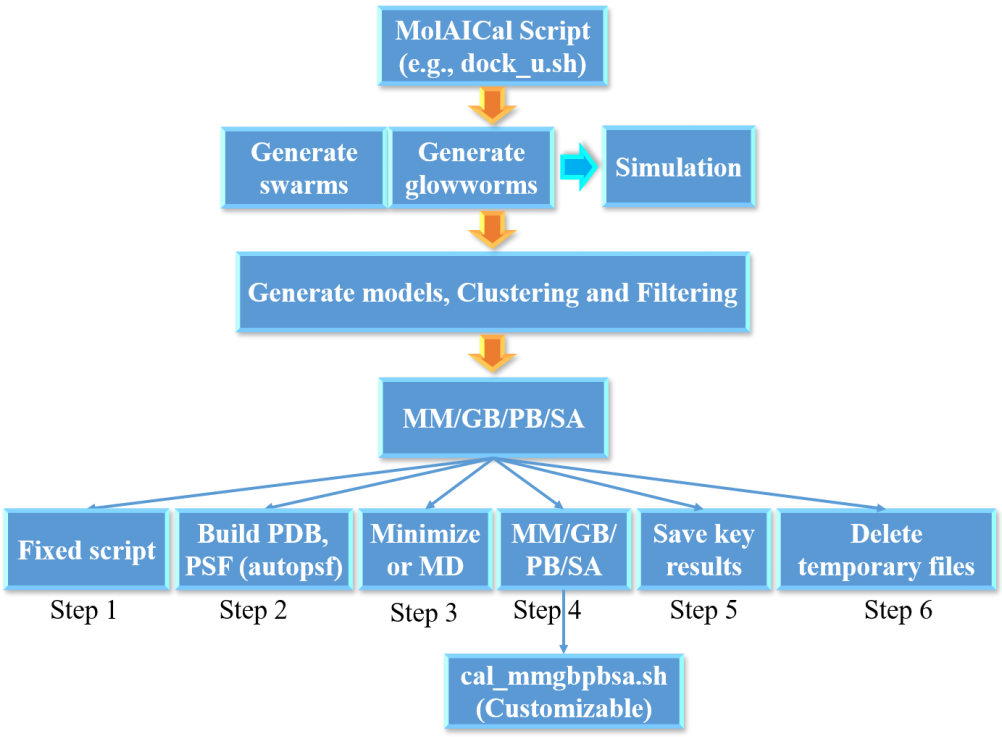
20) The argument "-st" will iterate through the 20 standard amino acids accordingly
#> molaical.exe -call run -c pepgen -i -l 8 -n 100 -cc -3d -st -c 4 -nc

6.2.4.2 Virtual screening of peptides by MolAICal and LightDock

The DFIRE scoring function in LightDock requires standard amino acid residues and their atom names; thus, it is essential to inspect and repair the receptor or ligand before performing docking with the DFIRE scoring function. The repair command is as follows:

```
#> molaical.exe -call run -c sfile -i lmfix_rec.py -i protein.pdb -o protein_fixed.pdb
```

MolAICal's virtual screening approach involves: creating a docking unit formatted as a MolAICal script file containing multiple tasks; each unit docks a single molecule (see Part I Figure 6.2.4.2). Building on this, multi-core parallel processing screens multiple molecules concurrently to achieve virtual screening.



Part I Figure 6.2.4.2 Docking unit.

✧ Virtual screening file "vs_ml.sh":

```
#> molaical.exe -call run -c sfile -i vs_ml.sh 1::=linear 2::=mempro.pdb 3::=R 4::=Z 5::=2 6::=8 7::=1 8::=0 9::=dock_u.sh 10::=restraints.list 11::=\"\"
```

```
#> molaical.exe -call run -c sfile -i vs_ml.sh 1::=linear 2::=mempro.pdb 3::=R 4::=Z
```

- **'-call'**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- **'::='**: Here, it is used to assign values to parameters in the specified order. Variables must immediately follow "::=" without any space.
- **'-c'**: It will run the scripts of MolAICal if its value is "sfile".

- **'1::='**: Work directory for storing molecules (e.g., peptides) to be screened. The default value is **'linear'**.
- **'2::='**: Receptor file. The default value is **'mempro.pdb'**.
- **'3::='**: Chain name of the first molecule (Corresponding: receptor file). The default value is **'R'**.
- **'4::='**: Chain name of the second molecule (Corresponding: ligand file). The default value is **'Z'**.
- **'5::='**: The number of CPU cores for the subtask (each docking job). The default value is **2**.
- **'6::='**: The number of CPU cores for the entire virtual screening job. The default value is **1**. This parameter determines the number of parallel pipelines to start. **Multiple parallel pipelines are highly memory-intensive. It is recommended to set this value to 1,** meaning only one pipeline will be activated. Unless available memory is very abundant, consider setting a higher number of pipelines. To further improve computational efficiency, the number of parallel cores can be increased for subtasks within the pipeline, for example: 5::=10.
- **'7::='**: Check whether the docking job is completed. Its value is 0 or 1. If its value is 1, it will check the results in the MM/GB/PB/SA step. If its value is 0, it will check the results in the first virtual screening step. The default value is 1.
- **'8::='**: Debug mode. Its value is 0 or 1. If its value is 1, it will do a test for the job to check the errors. If its value is 0, it will not do the debug and directly run the whole job. The default value is 0.
- **'9::='**: The docking script file in the MolAICal script format. The default value is **'dock_unit.sh'**.
- **'10::='**: File including restraints. The default value is **'restraints.list'**.
- **'11::='**: The directory for output results. The default folder is **'dresults'**.
- **'12::='**: The more options for the docking script file in **'9::='**. The default value is an empty string, and the empty string is represented as \"\". If this input contains multiple parameters, separate them using the comma character ','. MolAICal will automatically convert each comma ',' to a space " ". It has some special values:
  - If **"createdir"**, it only creates the folders for virtual screening;
  - If **"startvs"**, it only runs virtual screening jobs;
  - If **"continuevs"**, it will handle the remaining unfinished virtual screening tasks;
  - If **"sn_{num}"**, it will set the ${num} of **parallel pipelines** which corresponds to **'6::='**;
  - Otherwise, run all jobs.

The docking unit script (e.g., dock_u.sh) follows the format below. Lines starting with "#" denote comments, while executable commands typically begin with "molaical.exe" to run MolAICal tasks.

```
# Generate swarms
# 1.1 no memebrane
# molaical.exe -call run -c ldock -i lightdock3_setup.py $molargs1 $molargs2 --noxt --noh --now -rst $molargs3
# 1.2. memebrane
molaical.exe -call run -c ldock -i lightdock3_setup.py $molargs1 $molargs2 -membrane --noxt --noh --now -rst $molargs3

# 2. Run simulations
molaical.exe -call run -c ldock -i lightdock3.py setup.json 100 -s fastdfire -c $molargs6 -min

# 3. Generate models, Clustering, Rank, and filter
molaical.exe -call run -c sfile -i lrank.sh 1::=$molargs1 2::=$molargs2 3::=$molargs4 4::=$molargs5 5::=$molargs6 6::=$molargs3

# 4. Get the top candidate from the above results
```

```
molaical.exe -call run -c sfile -i mrank.py -ft 1

# 5. MM/GBSA
molaical.exe -call run -c sfile -i mmgbpbsa_batch.sh 1::=$molargs4 2::=$molargs5

# 6. Get the top candidate from MM/GBSA results
molaical.exe -call run -c sfile -i mrank.py -t 1
```

In the MM/GB/PB/SA step (e.g., step 5 in the above content), during initial execution, a parameter file named **"cal_mmgbpbsa.sh"** is automatically generated. Subsequent runs will not overwrite an existing **"cal_mmgbpbsa.sh"** file, which will be used to take precedence directly. Users may customize parameters within **"cal_mmgbpbsa.sh"**, which **adheres** to **MolAICal's script format,** enabling **batch execution** of MolAICal commands line-by-line (command-line calls typically begin with molaical.exe).

The **"cal_mmgbpbsa.sh"** remains a MolAICal script file, designed for line-by-line execution with content typically structured as follows:

```
# 1. fix autopsf
molaical.exe -call run -c fixpsf

# 2. generate psf
molaical.exe -call run -c vmdargs -i -pdb $molargs1 -dispdev text -args autopsf autopsf -mol 0 -prefix $molargs4

# 3. default to run 20 fs minimization and 40 fs MD simulation
molaical.exe -call run -c runnamd -i -dispdev text -args -s $molargs4_formatted_autopsf.psf -c $molargs4_formatted_autopsf.pdb -minimize $molargs6 -if_run 1

# 4. Cut last 40 steps for MM/GBSA calculation
molaical.exe -call run -c catdcd -i -dispdev text -args -out $molargs4_result.dcd -first $molargs5 -last $molargs6 -stride 1 $molargs4_formatted_autopsf_md.dcd

# 5. Save a psf and pdb file after minimization or MD simulation
molaical.exe -call run -c vmdargs -i -psf $molargs4_formatted_autopsf.psf -dcd $molargs4_result.dcd -args none set sel [atomselect top all frame [expr {int\\\(\\\($molargs6-$molargs5\\\)/2\\\)}]], \\\$sel writepdb $molargs4_minmd_out.pdb, \\\$sel writepsf $molargs4_minmd_out.psf

# 6. Calculate MM/GB/PBSA: "-pb 0 -gb 1" is MM/GBSA; "-pb 2" is MM/PBSA. Others are the same.
molaical.exe -call run -c mmpbgbsa -i -dispdev text -args -s $molargs4_formatted_autopsf.psf -c $molargs4_formatted_autopsf.pdb -t $molargs4_result.dcd -cs "chain,$molargs2,$molargs3" -rs "chain,$molargs2" -ls "chain,$molargs3" -pb 0 -gb 1

# 7. Delete file with command (in windows: del xxx, in linux: rm xxx)
molaical.exe -call run -c ecommand -i rm -rf *md.* *run.* *result.dcd *final.* *_wb* FFTW_* *_autopsf.pdb *_autopsf.psf *_autopsf.log *_formatted.pdb
```

✧ Rank program file "rank_ppn.py" for virtual screening results:

```
usage: rank_ppn.py [-h] [-m {both,filtered,mmgbpbsa}] [-p PERCENTAGE]
                   [-fs FILTERED_SCORE_FILE] [-fm FILTERED_MERGED]
                   [-fo FILTERED_OUTPUT] [-ms MMGBPBSA_SCORE_FILE]
                   [-mm MMGBPBSA_MERGED] [-mo MMGBPBSA_OUTPUT]
                   root_directory
Molecular Ranking and Extraction Tool
positional arguments:
root_directory                  Root directory to search for candidate subdirectories
optional arguments:
-h, --help                      show this help message and exit
-m, --mode {both,filtered,mmgbpbsa}
                                Processing mode (default: both)
-p, --percentage PERCENTAGE     Percentage of top molecules to extract (default: 5.0)
-fs, --filtered-score-file FILTERED_SCORE_FILE
                                Filtered score filename to search for (default:
                                rank_filtered_candidates.list)
-fm, --filtered-merged FILTERED_MERGED
                                Filtered merged output filename (default: total_filtered_rank.dat)
-fo, --filtered-output FILTERED_OUTPUT
                                Filtered results output directory (default: vs_filtered_results)
-ms, --mmgbpbsa-score-file MMGBPBSA_SCORE_FILE
                                MMGBPBSA score filename to search for (default:
                                final_rank_results.log)
-mm, --mmgbpbsa-merged MMGBPBSA_MERGED
                                MMGBPBSA merged output filename (default:
                                total_mmgbpbsa_rank.dat)
-mo, --mmgbpbsa-output MMGBPBSA_OUTPUT
                                MMGBPBSA results output directory (default:
                                vs_mmgbpbsa_results)
```

The example commands for ranking results are as below:

1) Print help information for ranking results in MolAICal
#> molaical.exe -call run -c sfile -i rank_ppn.py --help

2) Process both filtered and MMGBPBSA with default settings, and get the top 10% results
#> molaical.exe -call run -c sfile -i rank_ppn.py dresults -p 10

3) Process only filtered candidates with a custom percentage (here it is 20%)
#> molaical.exe -call run -c sfile -i rank_ppn.py dresults -m filtered -p 20

4) Process only MMGBPBSA with custom filenames
#> molaical.exe -call run -c sfile -i rank_ppn.py dresults -m mmgbpbsa -ms final_rank_results.log

5) Process only MMGBPBSA with custom directory of output results
#> molaical.exe -call run -c sfile -i rank_ppn.py dresults -m mmgbpbsa -mo mmgbpbsa -p 20

6) Custom output directories and merged filenames
#> molaical.exe -call run -c sfile -i rank_ppn.py dresults -fo my_filtered_results -mm my_mmgbpbsa_rank.dat

6.2.4.3 Virtual screening of peptides by MolAICal

LightDock depends on the amino acid types supported by the scoring function. For example, DFIRE only supports the following amino acid types:

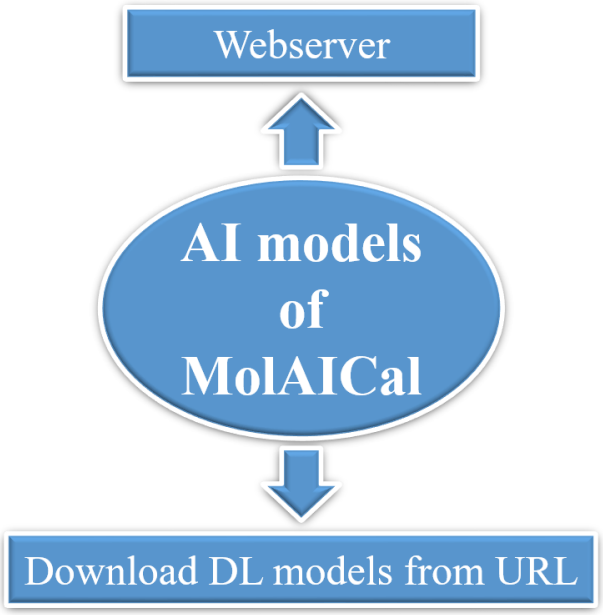
```
"ALA": ["N", "CA", "C", "O", "CB"],
"CYS": ["N", "CA", "C", "O", "CB", "SG"],
"ASP": ["N", "CA", "C", "O", "CB", "CG", "OD1", "OD2"],
"GLU": ["N", "CA", "C", "O", "CB", "CG", "CD", "OE1", "OE2"],
"PHE": ["N", "CA", "C", "O", "CB", "CG", "CD1", "CD2", "CE1", "CE2", "CZ"],
"GLY": ["N", "CA", "C", "O"],
"HIS": ["N", "CA", "C", "O", "CB", "CG", "ND1", "CD2", "CE1", "NE2"],
"ILE": ["N", "CA", "C", "O", "CB", "CG1", "CG2", "CD1"],
"LYS": ["N", "CA", "C", "O", "CB", "CG", "CD", "CE", "NZ"],
"LEU": ["N", "CA", "C", "O", "CB", "CG", "CD1", "CD2"],
"MET": ["N", "CA", "C", "O", "CB", "CG", "SD", "CE"],
"ASN": ["N", "CA", "C", "O", "CB", "CG", "OD1", "ND2"],
"PRO": ["N", "CA", "C", "O", "CB", "CG", "CD"],
"GLN": ["N", "CA", "C", "O", "CB", "CG", "CD", "OE1", "NE2"],
"ARG": ["N", "CA", "C", "O", "CB", "CG", "CD", "NE", "CZ", "NH1", "NH2"],
"SER": ["N", "CA", "C", "O", "CB", "OG"],
"THR": ["N", "CA", "C", "O", "CB", "OG1", "CG2"],
"VAL": ["N", "CA", "C", "O", "CB", "CG1", "CG2"],
"TRP": ["N", "CA", "C", "O", "CB", "CG", "CD1", "CD2", "CE2", "NE1", "CE3", "CZ3", "CH2", "CZ2"],
"TYR": ["N", "CA", "C", "O", "CB", "CG", "CD1", "CD2", "CE1", "CE2", "CZ", "OH"],
"MMB": ["BJ"]
```

but when using MolAICal's default screening method, it is not limited to specific amino acid types. The virtual screening way is the same with 3.2.4. Command introduction for virtual screening.

Part II. Artificial intelligence

In this part, the excellent models **under the permitted license** or trained models by ourselves are collected into MolAICal to make the best of artificial intelligence (AI), such as deep learning for drug design. The deep learning model will continue to be updated if there are better models than the current models. I believe the above classical algorithm and the below AI models can promote the drug design progress.

Due to the big size of deep learning models, two ways are chosen for MolAICal (see Part II Figure 2): one is based on the web server, the users can utilize functions of deep learning online, directly. The other is to download the deep learning models from the cloud server, and then, put them into the folder "MolAICal-xxx/mtools/AIModels" from the URL below:
https://lzueducn-my.sharepoint.com/:f:/g/personal/baiqf11_lzu_edu_cn/EpypiZ9_QDxIjoEjt0tl7osBVrssSaQ5x9Ulxue57ksUkg?e=2FYRVu
For more detailed procedures, please check the tutorial part: https://molaical.github.io/tutorial.html



Part II Figure 2. Two ways for AI models in MolAICal.

1. Protein fold

AlphaFold v2.0 is a very excellent deep learning framework for predicting the protein structure from a one-dimensional sequence[55]. AlphaFold code is licensed under the Apache 2.0 License, and the AlphaFold parameters are made available for non-commercial use only under the terms of the CC

BY-NC 4.0 license. (More detailed license: https://github.com/deepmind/alphafold#license-and-disclaimer). AlphaFold is a very excellent tool for protein folding. MolAICal provides a web server URL for protein fold based on a reduced AlphaFold version of the BFD database and with no ensembling (non-commercial use license under the terms of the CC BY-NC 4.0 license). If users want to use "casp14" option for predicting protein structure with high quality, users can send the protein sequence to us for further academic cooperation or visit the official site for protein fold prediction:
https://colab.research.google.com/github/sokrypton/ColabFold/blob/main/AlphaFold2.ipynb

MolAICal supplies this function via the web server, whose URL can be obtained by the following command:

-model: specified AI model function.

1) Getting the latest URL via MolAICal.
#> molaical.exe -model alphafold

2. Binding affinity prediction

Deep learning methods can be used to train deep learning models for binding affinity prediction. To let researchers use deep learning models for binding affinity prediction, two deep learning models named OnionNet[56] and Pafnucy[57] are introduced under the permitted licenses. OnionNet uses GNU General Public License v3.0 (https://choosealicense.com/licenses/gpl-3.0). Pafnucy uses the BSD 3-Clause License (https://choosealicense.com/licenses/bsd-3-clause). The source codes of OnionNet and Pafnucy are modified to calculate the binding affinity between the receptor and ligands easily.

2.1. Binding affinity prediction by OnionNet and MolAICal

To calculate the binding affinity between proteins and ligands by OnionNet, it needs to merge the files of proteins and ligands. MolAICal provides commands to calculate the binding affinities and to merge the files of proteins and ligands based on the trained OnionNet model as follows.

-model: specified AI model function.
-r: path of receptor file.
-l: path of ligand file.
-c: merged complex file that contains protein and ligand files.
-f: list file containing ligand names.

The example commands are as below:

1) Merge one protein and one ligand
#> molaical.exe -model mergeon -r E:/protein.pdb -l E:/ligand.mol2.pdb -c E:/complex.pdb

2) Merge one protein and many ligands
#> molaical.exe -model mergeon -r E:/GCGRNoLigand.pdb -f E:/list.txt

Finally, onionnet models can be run for binding affinity prediction when files are merged.

Notice: Currently, the below command is only supported to run on the Linux system.

1) Calculate pKx (pKa or pKi) for binding affinity prediction
molaical.exe -model onionnet -i /home/feng/onionnet/input_PDB_testing.dat -o result.csv

2.2. Binding affinity prediction by Pafnucy and MolAICal

Pafnucy is another excellent deep learning model for binding affinity prediction. It can calculate the pKx (pKa or pKi) between proteins and ligands. MolAICal provides commands to calculate the binding affinities between proteins and ligands based on the trained Pafnucy model in the **Linux system** as follows:

The example commands are as below:

1) Calculating binding affinity between a ligand and a protein pocket using default HDF filename
#> molaical.exe -model pafnucy -l lig_1.mol2 -p pocket.mol2 -o res.csv

2) Calculate binding affinity between a ligand and a protein pocket with appointed HDF filename
#> molaical.exe -model pafnucy -l lig_1.mol2 -p pocket.mol2 -t tmp.hdf -o res.csv

3) Calculate binding affinity between many ligands and a protein pocket
#> molaical.exe -model pafnucy -l "lig_1.mol2 lig_2.mol2" -p pocket.mol2 -t tmp.hdf -o res.csv

4) If there are different pockets for each ligand, list them all in the same order as ligands. And then, MolAICal can calculate binding affinities between “lig_1.mol2-pocket1.mol2” and “lig_2.mol2-pocket2.mol2”
#> molaical.exe -model pafnucy -l "lig_1.mol2 lig_2.mol2" -p “pocket1.mol2 pocket2.mol2” -t tmp.hdf -o res.csv

3. Molecular generation

3.1. Molecular generation for drug-like molecules

This function can generate drug-like and FDA-like molecules that can be used for drug virtual screening. Molecular generation can be carried out by the basic version of MolAICal. The example commands are as below:

-dock: value is AI, which means performing drug virtual screening by AI.
-s: selection of the deep learning model. Values are “ZINCMol”, “FDAMol”, “FDAFrag”.
-n: number of generated molecules by the deep learning model.
-nf: number of generated molecules in one folder. It avoids very huge of molecules in one folder.
-nc: number of CPU cores.
-g: switch AI molecular generations. “on” means generation. “off” means no generation.
-b: switch molecular format conversion. “on” means conversion. “off” means no conversion.
-v: switch virtual screening (VS) based on docking. “on” means VS. “off” means no VS.

The example commands are as below:

1) Generating FDA-like molecules.
#> molaical.exe -dock AI -s FDAMol -n 30 -nf 10 -nc 4 -g on -b on -v off

Note: It will generate a file named tmpGenVS.dat that contains 30 molecular SMILES strings, and 3 folders named tmpGenMols1, tmpGenMols2, and tmpGenMols3. Users can generate many more molecules according to their needs. The generated 3D molecules are in PDBQT format by default. If users need to change the molecular format, they can refer to “Molecular format conversion” part in the MolAICal manual.

2) Generating FDA fragment-like molecules.
#> molaical.exe -dock AI -s FDAFrag -n 30 -nf 10 -nc 4 -g on -b off -v off

Note: It will generate a file named tmpGenVS.dat that contains 30 molecular SMILES strings. Users can generate many more molecules according to their needs.

3) Generating drug-like molecules.
#> molaical.exe -dock AI -s ZINCMol -n 30 -nf 10 -nc 4 -g on -b on -v off

Note: It will generate a file named tmpGenVS.dat that contains 30 molecular SMILES strings, and 3 folders named tmpGenMols1, tmpGenMols2, and tmpGenMols3. Users can use generated molecules to construct their own database. The generated 3D molecules are in PDBQT format by default. If users need to change the molecular format, they can refer to “Molecular format conversion” part in the MolAICal manual.

3.2. Generating similar ligands based on a known ligand

This function can generate novel scaffolds and groups based on a known ligand. It can be used to modify or design new drugs. The deep learning generative model trained by ligdream[58] is used to produce the novel ligands starting from a supplied ligand based on the variational autoencoder (VAE), convolutional, and recurrent networks. The ligdream is under GNU Affero General Public License v3.0 (https://www.gnu.org/licenses/agpl-3.0.en.html). MolAICal supplies this function via the web server, whose URL can be obtained by the following command:

-model: specified AI model function.

1) Getting the latest URL via MolAICal.
#> molaical.exe -model ligdream

4. ADMET and molecular properties

About 40% of drug candidates have failed in the past due to toxicity and unacceptable efficacy. In this case, the predictions of absorption, distribution, metabolism, excretion, and toxicity (ADMET) play an important role in the success of a drug candidate. Here, the interface package of the FP-ADMET[59] tool is integrated into MolAICal for drug ADMET evaluation under the permitted GNU General Public License v3.0 (http://choosealicense.com/licenses/gpl-3.0/). In addition, MolAICal will add and develop new methods for molecular properties prediction.

4.1. ADMET calculation by FP-ADMET and MolAICal

FP-ADMET employs fingerprint-based machine learning models for predicting 58 ADMET items. These 58 ADMET items can be understandable and interpretable by checking the related documents that are named “ADMETModels-doc.pdf”, “FP-ADMET.pdf”, and “FP-ADMET-SI.pdf” in the tutorial document directory of FP-ADMET. 58 ADMET items of FP-ADMET are listed as follows:

1: Anticommensal Effect on Human Gut Microbiota
2: Blood–brain-barrier penetration
3: Oral Bioavailability
4: AMES Mutagenecity
5: Metabolic Stability
6: Rat Acute LD50
7: Drug-Induced Liver Inhibition
8: HERG Cardiotoxicity
9: Haemolytic Toxicity

10: Myelotoxicity

11: Urinary Toxicity

12: Human Intestinal Absorption

13: Hepatic Steatosis

14: Breast Cancer Resistance Protein Inhibition

15: Drug-Induced Choleostasis

16: Human multidrug and toxin extrusion Inhibition

17: Toxic Myopathy

18: Phospholipidosis

19: Human Bile Salt Export Pump Inhibition

20: Organic anion transporting polypeptide 1B1 binding

21: Organic anion transporting polypeptide 1B3 binding

22: Organic anion transporting polypeptide 2B1 binding

23: Phototoxicity human

24: Phototoxicity in vitro

25: Respiratory Toxicity

26: P-glycoprotein Inhibition

27: P-glycoprotein Substrate

28: Mitochondrial Toxicity

29: Carcinogenecity

30: DMSO Solubility

31: Human Liver Microsomal Stability

32: Human Plasma Protein Binding

33: hERG Liability

34: Organic Cation Transporter 2 Inhibition

35: Drug-induced Ototoxicity

36: Rhabdomyolysis

37: T1/2 Human

38: T1/2 Mouse

39: T1/2 Rat

40: Cytotoxicity HepG2 cell line

41: Cytotoxicity NIH 3T3 cell line

42: Cytotoxicity HEK 293 cell line

43: Cytotoxicity CRL-7250 cell line

44: Cytotoxicity HaCat cell line

45: CYP450 1A2 Inhibition

46: CYP450 2C19 Inhibition

47: CYP450 2C9 Inhibition

48: CYP450 2D6 Inhibition

49: CYP450 3A4 Inhibition

50: pKa dissociation constant

51: logD Distribution coefficient (pH 7.4)

52: logS

53: Drug affinity to human serum albumin

54: MDCK permeability

55: 50% hemolytic dose

56: Skin penetration

57: CYP450 2C8 Inhibition

58: Aqueous Solubility (in phosphate saline buffer)

MolAICal supplies an interface package for calculating ADMET. The command parameters are as below:

-model: specified AI model function. Here, it is "fpadmet".

-s: select the running type for ADMET calculation. It has two string values: "all" and "single". "all" means MolAICal will calculate all items of ADMET for all drugs. And "single" means MolAICal will calculate a specified item of ADMET for all drugs.

-i: input a file that contains ligand SMILES strings.

-n: if "single" mode is selected in "-s", it should choose a specified item number of ADMET from the above 58 items.

-d: whether to delete the temporarily generated files. It contains two values: "on" (it will delete temporarily generated files) and "off" (it will not delete temporarily generated files). The default value is "on". If "off" is selected, one temporarily generated file contains one item prediction of ADMET for all input molecular SMILES strings.

-o: output result file.

Currently, this function of MolAICal can only be run on the **Linux operating system**. The example commands are as below:

1) Calculating all items of ADMET based on a file that contains SMILES strings with deleting temporarily generated files

#> molaical.exe -model fpadmet -s all -i mols.smi -o results.dat

Interpretations of results: It will generate a file named "results.dat" that contains all items of ADMET of the ligand. Open the file "results.dat", and it will show 58 items of results for one drug. For example, predicting Blood–brain-barrier penetration (item 2) and pKa dissociation constant (item 50) are shown below :

```
##############################
2: Blood-brain-barrier penetration
Predicted Confidence Credibility
lig1 Yes 0.93 0.32

……

50: pKa dissociation constant
Predicted quantile_0.025 quantile_0.975
```

lig1 5.19 -1.60 13.06
##############################

"lig1" is a drug or ligand name. The label "Yes" of compound "lig1" indicates that this compound is predicted to be suitable for blood brain barrier penetration. A confidence value of 0.93 indicates that the classifier is quite certain, and the prediction is possible to be a single label. A relatively low value of credibility (0.32) suggests that compounds like "lig1" are not sufficiently represented in the training set, and users should deal with the prediction with caution. For the regression example of item 50, a 95% prediction interval (predictions at 0.025 and 97.5 percentiles for pKa) is computed and gives a range for the predictions on individual observations. As the original paper on FP-ADMET said, narrow prediction intervals indicate a lower uncertainty associated with the prediction[59]. For more detailed interpretations of predicted results, please check the files named "ADMETModels-doc.pdf", "FP-ADMET.pdf", and "FP-ADMET-SI.pdf" in the folder "doc" along with the ADMET tutorial document.

2) Calculating all items of ADMET based on a file that contains SMILES strings without deleting temporarily generated files
#> molaical.exe -model fpadmet -s all -i mols.smi -d off -o results.dat

Notice: the temporarily generated files have the same interpretations as the files in the example below **3)**.

3) Calculating a specified item of ADMET from the above 58 items based on a file that contains SMILES strings

#> molaical.exe -model fpadmet -s single -i mols.smi -n 1

Interpretations of results: It will generate a file named "1.dat" due to selecting task item 1. If task item 2 is chosen, it will produce a file named "2.dat". If task item 3 is chosen, it will produce a file named "3.dat"… So the prefix name of the generated result file is the same as the selected task item. Open file "1.dat", the content of "1.dat" is like this:

##############################
Predicted Confidence Credibility
lig1 Negative 0.77 0.31
lig2 Negative 0.90 0.59
lig5 Negative 0.80 0.35
lig6 Negative 0.91 0.64
lig10 Negative 0.96 0.86
##############################

It only shows the calculated results of "1: Anticommensal Effect on Human Gut Microbiota" item of ADMET for all molecules.

4.2. Calculations of ADME and molecular properties by MolAICal

In this part, based on the library of adme-pred-py (https://github.com/ikmckenz/adme-pred-py, AGPL-3.0 license) with open-source Bayesian models[60], MolAICal focuses and updates on the brief drug ADME information and related molecular properties prediction continuously. The detailed parameters for molecular properties calculation are listed below:

-model: specified AI model function. Here, it is "admepro".
-s: select the running type for the calculations of ADME and molecular properties. Currently, it has 13 integer values that are **0-12**. The detailed calculated content for every number is as follows:
0: output all calculated molecular properties information
1: output brief general ADME information of a ligand
2: output basic molecular properties, the parameters are listed as follows:
Number of Hydrogen Bond Donors
Number of Hydrogen Bond Acceptors
Wildman-Crippen Molar Refractivity
Molecular weight
Number of atoms
Number of carbon atoms
Number of heteroatoms
Number of rings
Number of rotatable bonds
Log of partition coefficient (MolLogP)[61]
Topological polar surface area (TPSA)

3: output Druglikeness evaluation by Egan model with 95% confidence interval[62].
4: output Druglikeness evaluation by Ghose approach in looser qualifying range[63].
5: output Druglikeness evaluation by Ghose approach in tighter preferred range[63].
6: output Druglikeness evaluation by Lipinski's rule of five[3]
7: output Druglikeness evaluation by Muegge Simple Selection Criteria[64].
8: output Druglikeness evaluation based on molecular Properties[65]. Druglikeness rules satisfy: i. polar surface area <= 140 angstroms squared AND # rot. bonds <= 10 OR ii. sum of H-bond donors and acceptors <= 12 AND # rot. bonds <= 10 (https://github.com/ikmckenz/adme-pred-py/blob/master/src/adme_pred.py).
9: output BOILED-Egg value for Gastrointestinal (GI) absorption[66].
10: output BOILED-Egg value for blood–brain barrier (BBB) permeant[66].
11: output Brenk filter[67]. Brenk's Structural Alert filter finds fragments "putatively toxic, chemically reactive, metabolically unstable or to bear properties responsible for poor pharmacokinetics." (https://github.com/ikmckenz/adme-pred-py/blob/master/src/adme_pred.py)
12: output PAINS filter[68].

-i: input molecular SMILES strings.

-n: if “single” mode is selected in “-s”, it should choose a specified item number of ADMET from the above 58 items.
-o: output results file.

The example commands are as below:

#> molaical.exe -model admepro -s 0 -i "O=C(C)Oc1ccccc1C(=O)O"

Notice: Due to the character recognition problem in the command line, the SMILES string for input should have double quotation marks.

Part III. Virtual Environment Interface

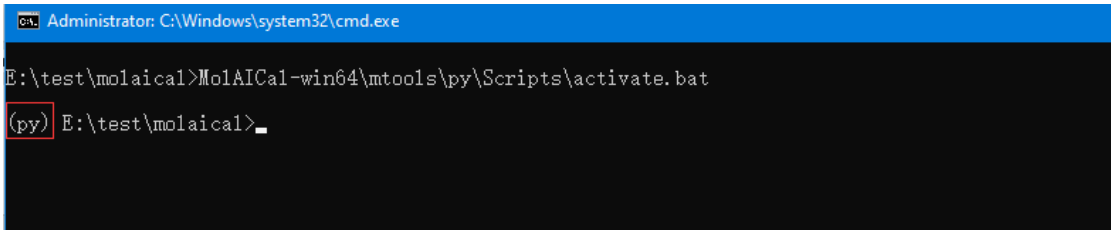
MolAICal supplies a virtual conda environment to use some functions such as Python script and RDKit (BSD-3-Clause License) for Computer-Aided Drug Design (CADD). Users can use these wanted functions without installing additional software.

1. Go into virtual environment

For Windows, open DOS, and use the below command:

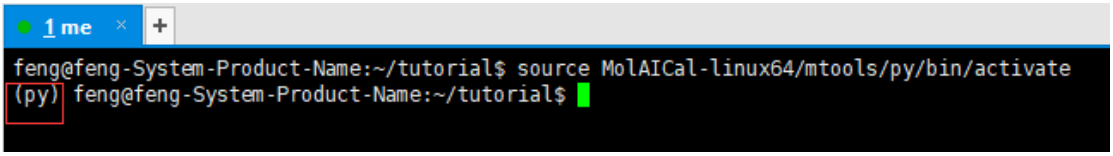
#> <Your installed path of MolAICal>\mtools\py\Scripts\activate.bat

It will be like:



For Linux, open the Linux terminal console, and use the following command:
#> source <Your installed path of MolAICal>/mtools/py/bin/activate

It will be like:



2. Using RDKit

2.1. Checking the version of RDKit by using as below command

```
#> python
>>> from rdkit import rdBase
>>> rdBase.rdkitVersion
```

Then it will show like: '2019.03.2'

```
feng@feng-System-Product-Name:~/test$ source MolAICal-linux64/mtools/py3/bin/activate
(py3) feng@feng-System-Product-Name:~/test$ python
Python 3.6.8 |Anaconda, Inc.| (default, Dec 30 2018, 01:22:34)
[GCC 7.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from rdkit import rdBase
>>> rdBase.rdkitVersion
'2019.03.2'
>>>
```

Notice: the version of RDKit will be updated with the update of MolAICal. This is just an example, users may show another version rather than '2019.03.2'.

2.2. An example of RDKit by using virtual environment of MolAICal

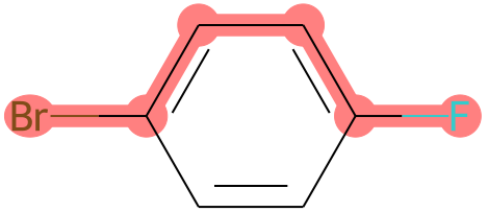
This example is about highlighting appointed atoms and bonds. First of all, run the command as follows:

```
#> python
```

And then, running commands as below:

```
>>> from rdkit import Chem
>>> from rdkit.Chem.Draw import rdMolDraw2D
>>> smi = 'c1cc(Br)ccc1F'
>>> mol = Chem.MolFromSmiles(smi)
>>> patt = Chem.MolFromSmarts('FccccBr')
>>> hit_ats = list(mol.GetSubstructMatch(patt))
>>> hit_bonds = []
>>> for bond in patt.GetBonds():
...     aid1 = hit_ats[bond.GetBeginAtomIdx()]
...     aid2 = hit_ats[bond.GetEndAtomIdx()]
...     hit_bonds.append(mol.GetBondBetweenAtoms(aid1,aid2).GetIdx())
...
>>> d = rdMolDraw2D.MolDraw2DCairo(500, 500)
>>> rdMolDraw2D.PrepareAndDrawMolecule(d, mol, highlightAtoms=hit_ats,
highlightBonds=hit_bonds)
>>> d.WriteDrawingText("./atom_labels_1.png")
>>> exit()
```

Open generated image “atom_labels_1.png”, it is like as below:



Appendix

1. Configuration file for simple radii measurement

pdbpath	D:/GramicidinA/1JNO.pdb
cpoint	0.1625 -0.629 -1.838
vector	0.00 0.00 1.00
sample	0.25
conpar	0.15
endrad	5.0
selatoms	5.1
numpt	300
surColor	blue
lineColor	red
lineWidth	2
searchNum	2000
material	Glass2

2. Configuration file for radii measurement with CHARMM ff

pdbpath	D:/GramicidinA/1JNO.pdb
psfpath	D:/GramicidinA/1JNO.psf
cpoint	0.1625 -0.629 -1.838
vector	0.00 0.00 1.00

sample	0.25
conpar	0.15
endrad	5.0
selatoms	5.1
numpt	300
surColor	blue
lineColor	red
lineWidth	2
searchNum	2000
material	Glass2

REFERENCES

1. Kumar, A.; Jha, A., Chapter 7 - Drug Development Strategies. In *Anticandidal Agents*, Kumar, A.; Jha, A., Eds. Academic Press: 2017; pp 63-71.
2. Chen, H.; Engkvist, O.; Wang, Y.; Olivecrona, M.; Blaschke, T., The rise of deep learning in drug discovery. *Drug Discov. Today* **2018,** *23* (6), 1241-1250.
3. Lipinski, C. A.; Lombardo, F.; Dominy, B. W.; Feeney, P. J., Experimental and computational approaches to estimate solubility and permeability in drug discovery and development settings. *Adv Drug Deliv Rev* **2001,** *46* (1-3), 3-26.
4. Dahlin, J. L.; Nissink, J. W.; Strasser, J. M.; Francis, S.; Higgins, L.; Zhou, H.; Zhang, Z.; Walters, M. A., PAINS in the assay: chemical mechanisms of assay interference and promiscuous enzymatic inhibition observed during a sulfhydryl-scavenging HTS. *J Med Chem* **2015,** *58* (5), 2091-113.
5. Dana, D.; Gadhiya, S. V.; St Surin, L. G.; Li, D.; Naaz, F.; Ali, Q.; Paka, L.; Yamin, M. A.; Narayan, M.; Goldberg, I. D.; Narayan, P., Deep Learning in Drug Discovery and Medicine; Scratching the Surface. *Molecules* **2018,** *23* (9).
6. De Cao, N.; Kipf, T., MolGAN: An implicit generative model for small molecular graphs. *arXiv preprint arXiv:1805.11973* **2018**.
7. Guimaraes, G. L.; Sanchez-Lengeling, B.; Outeiral, C.; Farias, P. L. C.; Aspuru-Guzik, A., Objective-reinforced generative adversarial networks (ORGAN) for sequence generation models. *arXiv preprint arXiv:1705.10843* **2017**.
8. Shultz, M. D., Two Decades under the Influence of the Rule of Five and the Changing Properties of Approved Oral Drugs. *J. Med. Chem.* **2019,** *62* (4), 1701-1714.
9. Smith, B. R.; Eastman, C. M.; Njardarson, J. T., Beyond C, H, O, and N! Analysis of the elemental composition of U.S. FDA approved drug architectures. *J Med Chem* **2014,** *57* (23), 9764-73.
10. Trott, O.; Olson, A. J., AutoDock Vina: improving the speed and accuracy of docking with a new scoring function, efficient optimization, and multithreading. *J. Comput. Chem.* **2010,** *31* (2), 455-61.

11. Wang, R.; Fang, X.; Lu, Y.; Yang, C. Y.; Wang, S., The PDBbind database: methodologies and updates. *J Med Chem* **2005,** *48* (12), 4111-9.
12. Morris, G. M.; Huey, R.; Lindstrom, W.; Sanner, M. F.; Belew, R. K.; Goodsell, D. S.; Olson, A. J., AutoDock4 and AutoDockTools4: Automated docking with selective receptor flexibility. *J Comput Chem* **2009,** *30* (16), 2785-91.
13. Rester, U., From virtuality to reality - Virtual screening in lead discovery and lead optimization: a medicinal chemistry perspective. *Current opinion in drug discovery & development* **2008,** *11* (4), 559-68.
14. Quiroga, R.; Villarreal, M. A., Vinardo: A Scoring Function Based on Autodock Vina Improves Scoring, Docking, and Virtual Screening. *PLoS One* **2016,** *11* (5), e0155183.
15. Bai, Q. F.; Xu, T. Y.; Huang, J. Z.; Pérez-Sánchez, H., Geometric deep learning methods and applications in 3D structure-based drug design. *Drug Discovery Today* **2024,** *29* (7).
16. Hou, T.; Wang, J.; Li, Y.; Wang, W., Assessing the performance of the MM/PBSA and MM/GBSA methods. 1. The accuracy of binding free energy calculations based on molecular dynamics simulations. *J. Chem. Inf. Model.* **2011,** *51* (1), 69-82.
17. Hou, T.; Wang, J.; Li, Y.; Wang, W., Assessing the performance of the molecular mechanics/Poisson Boltzmann surface area and molecular mechanics/generalized Born surface area methods. II. The accuracy of ranking poses generated from docking. *J. Comput. Chem.* **2011,** *32* (5), 866-77.
18. Jurrus, E.; Engel, D.; Star, K.; Monson, K.; Brandi, J.; Felberg, L. E.; Brookes, D. H.; Wilson, L.; Chen, J. H.; Liles, K.; Chun, M. J.; Li, P.; Gohara, D. W.; Dolinsky, T.; Konecny, R.; Koes, D. R.; Nielsen, J. E.; Head-Gordon, T.; Geng, W. H.; Krasny, R.; Wei, G. W.; Holst, M. J.; McCammon, J. A.; Baker, N. A., Improvements to the APBS biomolecular solvation software suite. *Protein Sci* **2018,** *27* (1), 112-128.
19. Li, C.; Jia, Z.; Chakravorty, A.; Pahari, S.; Peng, Y. H.; Basu, S.; Koirala, M.; Panday, S. K.; Petukh, M.; Li, L.; Alexov, E., DelPhi Suite: New Developments and Review of Functionalities. *Journal of Computational Chemistry* **2019,** *40* (28), 2502-2508.
20. Li, L.; Li, C. A.; Sarkar, S.; Zhang, J.; Witham, S.; Zhang, Z.; Wang, L.; Smith, N.; Petukh, M.; Alexov, E., DelPhi: a comprehensive suite for DelPhi software and associated resources. *Bmc Biophys* **2012,** *5*.
21. Glykos, N. M., Software news and updates. Carma: a molecular dynamics analysis program. *J. Comput. Chem.* **2006,** *27* (14), 1765-8.
22. Andricioaei, I.; Karplus, M., On the calculation of entropy from covariance matrices of the atomic fluctuations. **2001,** *115* (14), 6289-6292.
23. Metropolis, N.; Rosenbluth, A. W.; Rosenbluth, M. N.; Teller, A. H.; Teller, E., Equation of state calculations by fast computing machines. *The journal of chemical physics* **1953,** *21* (6), 1087-1092.
24. Bai, Q.; Perez-Sanchez, H.; Zhang, Y.; Shao, Y.; Shi, D.; Liu, H.; Yao, X., Ligand induced change of beta2 adrenergic receptor from active to inactive conformation and its implication for the closed/open state of the water channel: insight from molecular dynamics simulation, free energy calculation and Markov state model analysis. *Phys. Chem. Chem. Phys.* **2014,** *16* (30), 15874-85.
25. Humphrey, W.; Dalke, A.; Schulten, K., VMD: visual molecular dynamics. *J. Mol. Graph.* **1996,** *14* (1), 33-8, 27-8.
26. Halgren, T. A., Merck molecular force field. I. Basis, form, scope, parameterization, and performance of MMFF94. *Journal of Computational Chemistry* **1996,** *17* (5-6), 490-519.

27. Baell, J.; Walters, M. A., Chemistry: Chemical con artists foil drug discovery. *Nature* **2014,** *513* (7519), 481-3.
28. O'Boyle, N. M.; Banck, M.; James, C. A.; Morley, C.; Vandermeersch, T.; Hutchison, G. R., Open Babel: An open chemical toolbox. *J. Cheminform.* **2011,** *3*, 33.
29. Ertl, P.; Schuffenhauer, A., Estimation of synthetic accessibility score of drug-like molecules based on molecular complexity and fragment contributions. *J Cheminform* **2009,** *1* (1), 8.
30. Kochev, N.; Avramova, S.; Angelov, P.; Jeliazkova, N., Computational prediction of synthetic accessibility of organic molecules with Ambit-synthetic accessibility tool. *J Org. Chem. Ind. J* **2018,** *14* (2), 123.
31. Kalgutkar, A. S.; Gardner, I.; Obach, R. S.; Shaffer, C. L.; Callegari, E.; Henne, K. R.; Mutlib, A. E.; Dalvie, D. K.; Lee, J. S.; Nakai, Y.; O'Donnell, J. P.; Boer, J.; Harriman, S. P., A comprehensive listing of bioactivation pathways of organic functional groups. *Curr Drug Metab* **2005,** *6* (3), 161-225.
32. Kalgutkar, A. S.; Soglia, J. R., Minimising the potential for metabolic activation in drug discovery. *Expert Opinion on Drug Metabolism & Toxicology* **2005,** *1* (1), 91-142.
33. Polykovskiy, D.; Zhebrak, A.; Sanchez-Lengeling, B.; Golovanov, S.; Tatanov, O.; Belyaev, S.; Kurbanov, R.; Artamonov, A.; Aladinskiy, V.; Veselov, M.; Kadurin, A.; Johansson, S.; Chen, H.; Nikolenko, S.; Aspuru-Guzik, A.; Zhavoronkov, A., Molecular Sets (MOSES): A Benchmarking Platform for Molecular Generation Models. *Frontiers in Pharmacology* **2020,** *Volume 11 - 2020*.
34. Ertl, P.; Roggo, S.; Schuffenhauer, A., Natural product-likeness score and its application for prioritization of compound libraries. *Journal of Chemical Information and Modeling* **2008,** *48* (1), 68-74.
35. Ivanenkov, Y. A.; Zagribelnyy, B. A.; Aladinskiy, V. A., Are We Opening the Door to a New Era of Medicinal Chemistry or Being Collapsed to a Chemical Singularity? *Journal of Medicinal Chemistry* **2019,** *62* (22), 10026-10043.
36. Wang, R.; Fang, X.; Lu, Y.; Wang, S., The PDBbind database: collection of binding affinities for protein-ligand complexes with known three-dimensional structures. *J Med Chem* **2004,** *47* (12), 2977-80.
37. Li, Y.; Su, M.; Liu, Z.; Li, J.; Liu, J.; Han, L.; Wang, R., Assessing protein-ligand interaction scoring functions with the CASF-2013 benchmark. *Nat. Protoc.* **2018,** *13* (4), 666-680.
38. Pettersen, E. F.; Goddard, T. D.; Huang, C. C.; Couch, G. S.; Greenblatt, D. M.; Meng, E. C.; Ferrin, T. E., UCSF Chimera--a visualization system for exploratory research and analysis. *J. Comput. Chem.* **2004,** *25* (13), 1605-12.
39. Kim, R.; Skolnick, J., Assessment of programs for ligand binding affinity prediction. *J Comput Chem* **2008,** *29* (8), 1316-31.
40. Karney, C. F.; Ferrara, J. E.; Brunner, S., Method for computing protein binding affinity. *J Comput Chem* **2005,** *26* (3), 243-51.
41. Yap, C. W., PaDEL-descriptor: an open source software to calculate molecular descriptors and fingerprints. *J Comput Chem* **2011,** *32* (7), 1466-74.
42. Moriwaki, H.; Tian, Y. S.; Kawashita, N.; Takagi, T., Mordred: a molecular descriptor calculator. *J Cheminform* **2018,** *10* (1), 4.
43. Halgren, T. A., Merck molecular force field. II. MMFF94 van der Waals and electrostatic parameters for intermolecular interactions. *Journal of Computational Chemistry* **1996,** *17* (5-6), 520-552.
44. Bajusz, D.; Rácz, A.; Héberger, K., Why is Tanimoto index an appropriate choice for

fingerprint-based similarity calculations? *Journal of Cheminformatics* **2015,** *7* (1), 20.
45. Ballester, P. J.; Richards, W. G., Ultrafast shape recognition to search compound databases for similar molecular shapes. *J. Comput. Chem.* **2007,** *28* (10), 1711-23.
46. Irwin, J. J.; Shoichet, B. K., ZINC--a free database of commercially available compounds for virtual screening. *J. Chem. Inf. Model.* **2005,** *45* (1), 177-82.
47. Le Guilloux, V.; Schmidtke, P.; Tuffery, P., Fpocket: An open source platform for ligand pocket detection. *BMC Bioinformatics* **2009,** *10*.
48. Krivák, R.; Hoksza, D., P2Rank: machine learning based tool for rapid and accurate prediction of ligand binding sites from protein structure. *J. Cheminform.* **2018,** *10* (1), 39.
49. Jiménez-García, B.; Roel-Touris, J.; Romero-Durana, M.; Vidal, M.; Jiménez-González, D.; Fernández-Recio, J., LightDock: a new multi-scale approach to protein-protein docking. *Bioinformatics* **2018,** *34* (1), 49-55.
50. Roel-Touris, J.; Bonvin, A. M. J. J.; Jiménez-García, B., LightDock goes information-driven. *Bioinformatics* **2020,** *36* (3), 950-952.
51. Roel-Touris, J.; Jimenez-Garcia, B.; Bonvin, A. M. J. J., Integrative modeling of membrane-associated protein assemblies. *Nature communications* **2020,** *11* (1).
52. Krishnanand, K. N.; Ghose, D., Glowworm swarm optimization for simultaneous capture of multiple local optima of multimodal functions. *Swarm Intelligence* **2009,** *3* (2), 87-124.
53. Fox, T.; Bieler, M.; Haebel, P.; Ochoa, R.; Peters, S.; Weber, A., BILN: A Human-Readable Line Notation for Complex Peptides. *J. Chem. Inf. Model.* **2022,** *62* (17), 3942-3947.
54. Zhang, T. H.; Li, H. L.; Xi, H. L.; Stanton, R. V.; Rotstein, S. H., HELM: A Hierarchical Notation Language for Complex Biomolecule Structure Representation. *J. Chem. Inf. Model.* **2012,** *52* (10), 2796-2806.
55. Jumper, J.; Evans, R.; Pritzel, A.; Green, T.; Figurnov, M.; Ronneberger, O.; Tunyasuvunakool, K.; Bates, R.; Zidek, A.; Potapenko, A.; Bridgland, A.; Meyer, C.; Kohl, S. A. A.; Ballard, A. J.; Cowie, A.; Romera-Paredes, B.; Nikolov, S.; Jain, R.; Adler, J.; Back, T.; Petersen, S.; Reiman, D.; Clancy, E.; Zielinski, M.; Steinegger, M.; Pacholska, M.; Berghammer, T.; Bodenstein, S.; Silver, D.; Vinyals, O.; Senior, A. W.; Kavukcuoglu, K.; Kohli, P.; Hassabis, D., Highly accurate protein structure prediction with AlphaFold. *Nature* **2021**.
56. Zheng, L.; Fan, J.; Mu, Y., OnionNet: a Multiple-Layer Intermolecular-Contact-Based Convolutional Neural Network for Protein-Ligand Binding Affinity Prediction. *ACS Omega* **2019,** *4* (14), 15956-15965.
57. Stepniewska-Dziubinska, M. M.; Zielenkiewicz, P.; Siedlecki, P., Development and evaluation of a deep learning model for protein-ligand binding affinity prediction. *Bioinformatics* **2018,** *34* (21), 3666-3674.
58. Skalic, M.; Jimenez, J.; Sabbadin, D.; De Fabritiis, G., Shape-Based Generative Modeling for de Novo Drug Design. *J. Chem. Inf. Model.* **2019,** *59* (3), 1205-1214.
59. Venkatraman, V., FP-ADMET: a compendium of fingerprint-based ADMET prediction models. *J Cheminform* **2021,** *13* (1), 75.
60. Clark, A. M.; Dole, K.; Coulon-Spektor, A.; McNutt, A.; Grass, G.; Freundlich, J. S.; Reynolds, R. C.; Ekins, S., Open Source Bayesian Models. 1. Application to ADME/Tox and Drug Discovery Datasets. *J Chem Inf Model* **2015,** *55* (6), 1231-45.
61. Wildman, S. A.; Crippen, G. M., Prediction of Physicochemical Parameters by Atomic Contributions. *Journal of Chemical Information and Computer Sciences* **1999,** *39* (5), 868-873.

62. Egan, W. J.; Merz, K. M., Jr.; Baldwin, J. J., Prediction of drug absorption using multivariate statistics. *J Med Chem* **2000,** *43* (21), 3867-77.
63. Ghose, A. K.; Viswanadhan, V. N.; Wendoloski, J. J., A knowledge-based approach in designing combinatorial or medicinal chemistry libraries for drug discovery. 1. A qualitative and quantitative characterization of known drug databases. *J Comb Chem* **1999,** *1* (1), 55-68.
64. Muegge, I.; Heald, S. L.; Brittelli, D., Simple selection criteria for drug-like chemical matter. *J Med Chem* **2001,** *44* (12), 1841-6.
65. Veber, D. F.; Johnson, S. R.; Cheng, H. Y.; Smith, B. R.; Ward, K. W.; Kopple, K. D., Molecular properties that influence the oral bioavailability of drug candidates. *J Med Chem* **2002,** *45* (12), 2615-23.
66. Daina, A.; Zoete, V., A BOILED-Egg To Predict Gastrointestinal Absorption and Brain Penetration of Small Molecules. *ChemMedChem* **2016,** *11* (11), 1117-21.
67. Brenk, R.; Schipani, A.; James, D.; Krasowski, A.; Gilbert, I. H.; Frearson, J.; Wyatt, P. G., Lessons learnt from assembling screening libraries for drug discovery for neglected diseases. *ChemMedChem* **2008,** *3* (3), 435-44.
68. Baell, J. B.; Holloway, G. A., New substructure filters for removal of pan assay interference compounds (PAINS) from screening libraries and for their exclusion in bioassays. *J. Med. Chem.* **2010,** *53* (7), 2719-40.